6

Język PHP

6.1. Wprowadzenie

PHP (ang. *Hypertext Preprocessor*) to skryptowy język programowania najczęściej wykorzystywany do tworzenia aplikacji internetowych. Skrypty pisane w PHP są umieszczane w kodzie HTML i wykonywane po stronie serwera. Klient otrzymuje tylko wynik wykonania skryptu.

Oprogramowanie PHP umożliwia przetwarzanie danych z formularzy, dynamiczne generowanie zawartości stron internetowych, wysyłanie i odbieranie ciasteczek (cookies). Obsługuje wiele protokołów sieciowych, w tym SMTP, POP3, IMAP, NNTP. Współpracuje z większością dostępnych systemów operacyjnych, między innymi z różnymi wersjami Linuksa, systemem Windows i macOS, oraz z różnymi systemami baz danych. Obsługuje wiele serwerów HTTP (Apache, IIS).

PHP jest rozpowszechniany na zasadach open source, w związku z tym dostępny jest jego pełny kod źródłowy. Istnieją również liczne rozszerzenia i narzędzia, z których można korzystać, by zwiększyć jego możliwości.

Wyróżnia się trzy główne obszary, w których używany jest język PHP. Są to:

- Skrypty po stronie serwera — jest to główne zastosowanie PHP. Aby tworzyć skrypty, potrzebne są: parser PHP, serwer WWW oraz przeglądarka internetowa.
- Skrypty wywoływane z wiersza poleceń — skrypty są tworzone z wykorzystaniem parsera PHP i wywoływane w linii poleceń. Mogą być użyte również do prostego przetwarzania tekstu.
- Aplikacje po stronie klienta — zaawansowane funkcje PHP umożliwiają pisanie aplikacji desktopowych.

Kod zapisany w skryptach PHP jest przetwarzany za pomocą silnika Zend. Nadzoruje on wykonywanie wszystkich operacji dotyczących przetwarzania kodu PHP. PHP integruje się z serwerem WWW jako jego moduł lub skrypt CGI. Serwerem WWW najczęściej jest serwer Apache lub serwer IIS.

6.1.1. Opis języka

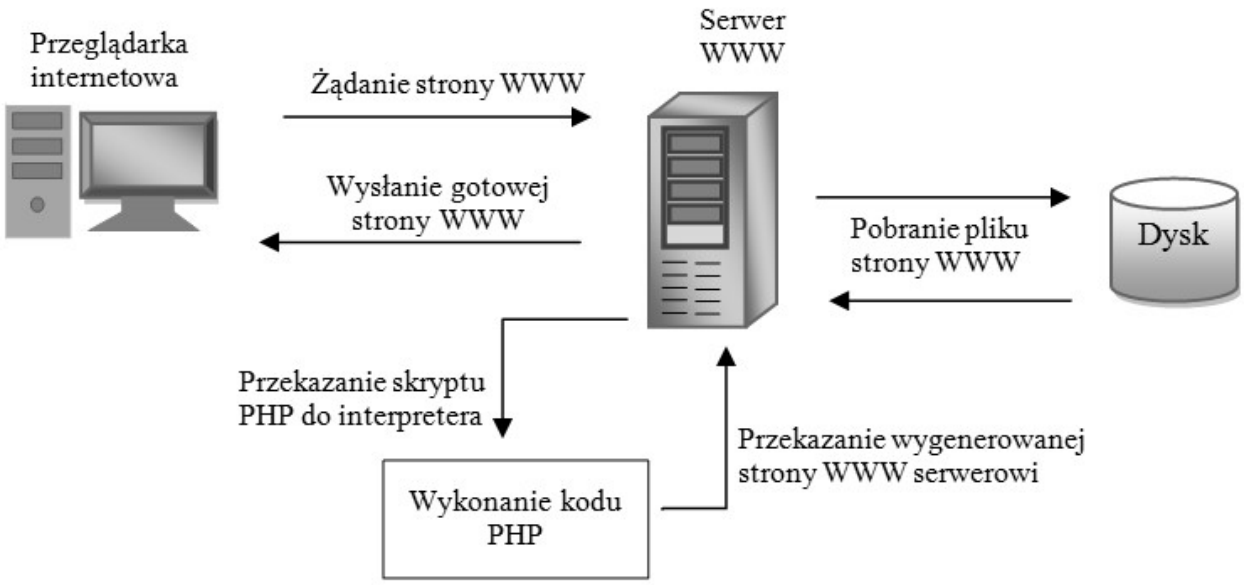
PHP jest skryptowym językiem programowania, za pomocą którego można tworzyć aplikacje WWW. Używając PHP, można:

- generować dynamicznie zawartość strony internetowej,
- tworzyć i edytować pliki na serwerze,
- pobierać dane z formularzy,
- odbierać i wysyłać cookies,
- wykonywać operacje na bazie danych,
- sterować dostępem użytkowników do stron witryny internetowej,
- szyfrować dane.

6.1.2. PHP w HTML

Skrypty pisane w języku PHP są umieszczane wewnątrz dokumentów HTML. Umownie dokumenty HTML zawierające osadzony kod PHP nazywa się skryptami.

Strony internetowe wygenerowane na podstawie skryptu PHP są zarządzane przez serwer WWW. Gdy w przeglądarce ma zostać wyświetlona taka strona, serwer WWW pobiera plik zawierający żądany skrypt i przekazuje go do interpretera PHP. Interpreter odnajduje w pliku skrypt PHP (jest on specjalnie oznaczony w kodzie HTML) i próbuje go wykonać. Wykonany skrypt jest zwracany do serwera WWW w postaci kodu HTML. Cała zawartość strony internetowej w kodzie HTML jest wysyłana do przeglądarki internetowej. Proces przetwarzania kodu PHP został pokazany na rysunku 6.1.



Rysunek 6.1. Proces przetwarzania kodu PHP

6.2. Struktura języka PHP

Skrypty PHP są plikami tekstowymi, dlatego do ich pisania można wykorzystać dowolny edytor tekstu. Aby pliki tekstowe były prawidłowo rozpoznawane przez serwer WWW, muszą mieć rozszerzenie *.php*.

6.2.1. Blok instrukcji PHP

Umieszczając skrypt PHP w dokumencie HTML, należy zaznaczyć w kodzie, gdzie zaczynają się i kończą polecenia PHP. Do tego celu służą specjalne znaczniki otwierające i zamykające (tabela 6.1).

Tabela 6.1. Znaczniki bloku PHP

Rodzaj znacznika	Znacznik otwierający	Znacznik zamykający
Standardowy	`<?php`	`?>`
Skrócony	`<?`	`?>`

Konfiguracje serwerów są różne, zatem jeżeli tworzony jest kod, który będzie przenoszony między serwerami, najbezpieczniejsze jest używanie znaczników standardowych. Jest to też zalecany sposób umieszczania skryptów w kodzie HTML.

Przykład 6.1

```
<?
echo "Mój pierwszy skrypt PHP";
?>
```

Przykład 6.2

```
<?php
echo "Mój pierwszy skrypt PHP";
?>
```

Użyta w przykładach instrukcja `echo` służy do wyświetlania danych. Instrukcja `echo` może być stosowana w sposób podany w przykładach 6.1 i 6.2 lub z użyciem nawiasów okrągłych. Obydwa sposoby jej zapisu są równoważne. Natomiast wyświetlany ciąg znaków zawsze musi być ujęty w cudzysłowy lub apostrofy.

Przykład 6.3

```
<?php
echo("Mój pierwszy skrypt PHP");
?>
```

Tak samo jak instrukcja `echo` działa instrukcja `print`. Może być ona używana wymiennie z instrukcją `echo`. Ciągi znaków przekazane do instrukcji `print` muszą być umieszczone w cudzysłowach lub apostrofach.

Przykład 6.4

```
<?php
print("Mój pierwszy skrypt PHP");
?>
```

Polecenia języka PHP muszą kończyć się średnikiem. Wyjątkiem są instrukcje zawierające inne instrukcje oraz instrukcje kończące blok kodu. Brak średnika na końcu polecenia powoduje sygnalizację błędu składni w kodzie PHP.

6.2.2. Blok PHP w kodzie HTML

Kod zapisany w języku PHP może zostać wstawiony bezpośrednio do kodu HTML.

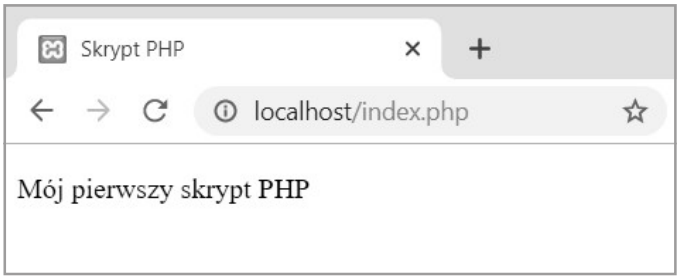
Przykład 6.5

```
<!DOCTYPE HTML>
<html>
<head>
<title>Skrypt PHP</title>
<meta charset="UTF-8">
</head>
<body>
<p>
<?php
echo "Mój pierwszy skrypt PHP";
?>
</p>
</body>
</html>
```

W przykładzie 6.5 w dokumencie HTML został umieszczony kod języka PHP. Aby zobaczyć efekt działania skryptu, należy go uruchomić w przeglądarce. W tym celu treść dokumentu zapisujemy w katalogu głównym serwera WWW (*/opt/lampp/htdocs/* dla pakietu LAMPP w systemie Linux, *C:\xampp\htdocs* dla pakietu XAMPP w systemie Windows), w pliku *index.php*. Następnie wpisujemy w przeglądarce adres *http://localhost/index.php* lub *http://127.0.0.1/index.php*.

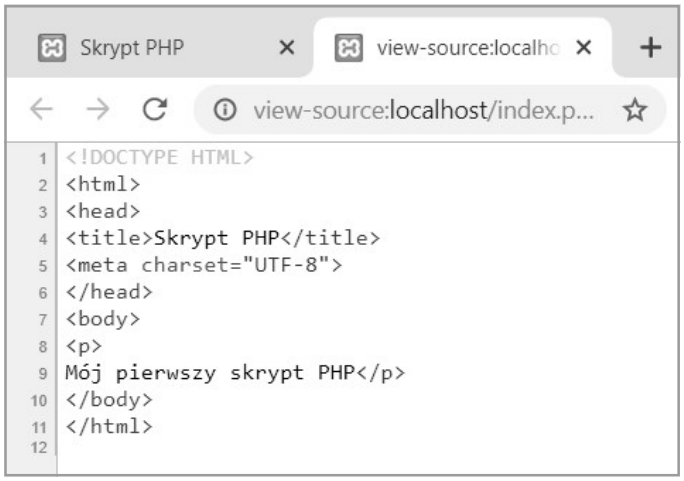
Jeżeli treść dokumentu została zapisana w innym katalogu, na przykład w katalogu *htdocs* został utworzony katalog *pliki_php*, to w przeglądarce należy wpisać adres *http://localhost/pliki_php/index.php*.

Jeśli wyświetlona strona wygląda tak jak na rysunku 6.2, to skrypt działa prawidłowo.



Rysunek 6.2. Efekt działania skryptu

Można teraz obejrzeć kod źródłowy wyświetlonej strony. W tym celu należy w jej obszarze kliknąć prawym przyciskiem myszy i wybrać opcję *Źródło*, *Wyświetl źródło strony* lub *Źródło strony*. Zostanie pokazany kod HTML wyświetlonej strony (rysunek 6.3).



Rysunek 6.3. Kod źródłowy strony

Widać, że w kodzie nie ma znaczników PHP. Pozostał jedynie kod HTML. Tak właśnie działają języki skryptowe wykonywane po stronie serwera. Ich zadaniem jest wygenerowanie kodu, który będzie mogła zinterpretować przeglądarka internetowa. W przedstawionym przykładzie w miejsce skryptu PHP została wstawiona treść, która może być w poprawny sposób wyświetlona przez przeglądarkę internetową. Kod PHP został wykonany na serwerze, a wynik został zwrócony do przeglądarki jako zwykły HTML.

6.3. Składnia języka PHP

Podstawową zasadą podczas tworzenia skryptów PHP jest oddzielanie kolejnych instrukcji znakiem średnika. Instrukcje mogą być pisane pojedynczo w jednym wierszu lub w kilku.

6.3.1. Komentarze

W skrypcie PHP można umieszczać komentarze. Są one widoczne w kodzie źródłowym skryptu, natomiast w trakcie jego przetwarzania są usuwane i nie są wyświetlane w przeglądarce. Występują trzy rodzaje komentarzy:

- komentarz blokowy,
- komentarz jednowierszowy,
- komentarz jednowierszowy uniksowy.

Komentarz blokowy zaczyna się od znaków `/*`, a kończy się znakami `*/`. Wszystko, co znajduje się między tymi znakami, jest ignorowane podczas przetwarzania skryptu.

Komentarz jednowierszowy zaczyna się od znaków `//` i kończy się w bieżącej linii skryptu. Wszystko, co znajduje się między znakami `//` a końcem linii, jest ignorowane podczas przetwarzania skryptu.

Komentarz jednowierszowy uniksowy jest podobny do komentarza jednowierszowego. Zaczyna się od znaku `#` i ciągnie się do końca linii.

6.3.2. Zmienne

Zmienne w programowaniu służą do przechowywania danych i wyników wykonywanych operacji. Wartością zmiennej może być liczba, ciąg znaków, tablica, obiekt. Zmienna musi mieć nazwę, przez którą można odwoływać się do niej w skrypcie. Nazwa może być dowolna, ale musi spełniać następujące warunki:

- musi zaczynać się od litery lub znaku podkreślenia,
- może składać się jedynie z liter, cyfr i znaku podkreślenia,
- w nazwach rozróżniane są małe i duże litery,
- w nazwach można stosować polskie litery.

Przed nazwą zmiennej należy umieścić znak `$`.

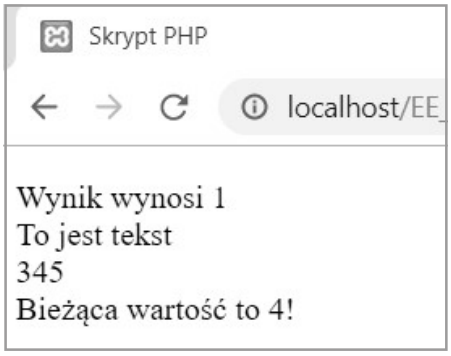
W języku PHP zmienna jest tworzona przy pierwszym jej użyciu. Nie trzeba wcześniej deklarować zmiennej ani określać jej typu.

Przykład 6.6

```
<?php
$x = 1;
$nazwa_1 = "tekst";
$liczba7 = 345;
$ilość = 4;
echo "Wynik wynosi $x <br>";
echo "To jest $nazwa_1 <br>";
```

```
echo "$liczba7 <br>";
echo "Bieżąca wartość to $ilość!";
?>
```

Wynik działania kodu został pokazany na rysunku 6.4.



Rysunek 6.4. Rezultat wykonania kodu z przykładu

Występujący w przykładzie zapis:

```
echo "Bieżąca wartość to $ilość!";
```

jest równoważny zapisowi:

```
echo "Bieżąca wartość to " . $ilość . "!";
```

Zakres zmiennych

Zmienne mogą być deklarowane w dowolnym miejscu skryptu.

Zmienne deklarowane wewnątrz skryptu lub funkcji są zmiennymi lokalnymi i istnieją tylko w obrębie danego skryptu albo funkcji. Do zmiennych lokalnych nie można odwoływać się w innych skryptach lub funkcjach.

Zmienne zadeklarowane poza funkcją mają zasięg globalny i można uzyskać do nich dostęp tylko poza funkcją.

Zmienne predefiniowane

W PHP istnieje grupa zmiennych predefiniowanych, nazywanych zmiennymi superglobalnymi. Przechowują one informacje dotyczące konfiguracji bieżącego skryptu i najczęściej mają postać tablicy. Ich wartość jest ustalana na podstawie zmiennych środowiskowych serwera WWW. Są dostępne we wszystkich skryptach. W wielu przypadkach są niezbędne do poprawnego tworzenia aplikacji internetowych. Należą do nich:

- `$_GET` — jest to tablica zawierająca zmienne przesyłane do skryptu za pomocą metody `GET`.
- `$_POST` — jest to tablica zawierająca zmienne przesyłane do skryptu za pomocą metody `POST`.
- `$_COOKIE` — jest to tablica zawierająca zmienne przekazane z serwera do skryptu za pomocą cookies.
- `$_FILES` — jest to tablica zawierająca zmienne przekazane do skryptu podczas przesyłania plików do serwera.
- `$_SERVER` — jest to tablica zawierająca zmienne przekazane do skryptu przez serwer WWW. Są to dane takie jak wersja serwera, ścieżka do pliku, adres skryptu, wysłane nagłówki.
- `$_ENV` — jest to tablica zawierająca wartości zmiennych środowiskowych serwera.
- `$_REQUEST` — jest to tablica zawierająca zmienne przekazane do skryptu przez użytkownika. Obejmuje dane z `$_GET`, `$_POST` oraz `$_COOKIE`.

- `$_SESSION` — jest to tablica zawierająca zmienne zarejestrowane w bieżącej sesji.
- `$GLOBALS` — jest to tablica zawierająca odniesienie do każdej zmiennej utworzonej przez użytkownika, która ma zasięg globalny dla danego skryptu.

6.3.3. Typy danych

W większości języków programowania każda zmienna ma swój typ. Określa on, jakiego rodzaju wartości mogą zostać przypisane do zmiennej. W języku PHP typ danych określany jest w zależności od kontekstu, w jakim zmienna została użyta.

Występujące w PHP typy danych można podzielić na trzy rodzaje.

Typy skalarne, czyli typy proste. Należą do nich:

- typ `boolean`,
- typ `integer`,
- typ `float` lub `double`,
- typ `string`.

Typy złożone. Należą do nich:

- typ `array` (tablicowy),
- typ `object` (obiektowy).

Typy specjalne. Należą do nich:

- typ `resource`,
- typ `null`.

Typ boolean

Jest to typ logiczny. Przyjmuje jedną z dwóch wartości: prawda (`true`) lub fałsz (`false`). Wartości typu logicznego są wykorzystywane przy budowaniu wyrażeń logicznych, porównywaniu danych, sprawdzaniu warunków.

Typ integer

Jest to typ liczb całkowitych. Może służyć do przedstawiania dodatnich i ujemnych liczb całkowitych. Liczby mogą zostać zapisane w formacie dziesiętnym, ósemkowym (oktalnym) lub szesnastkowym (heksadecymalnym). Domyślnie stosowany jest format dziesiętny. Liczby w formacie ósemkowym należy poprzedzić znakiem `0` (zero). Liczby w formacie szesnastkowym należy poprzedzić znakami `0x`. Przy zapisie szesnastkowym można używać dużych i małych liter od `A` do `F`. Ich zakres jest zależny od platformy sprzętowo-systemowej, z reguły są to wartości od -2^{31} do $2^{31}-1$. W przypadku przekroczenia zakresu wartość jest konwertowana na typ `float`.

Przykład 6.7

`537` — dodatnia liczba całkowita,

`-451` — ujemna liczba całkowita,

`032` — dodatnia liczba całkowita zapisana w formacie ósemkowym,

`-021` — ujemna liczba całkowita zapisana w formacie ósemkowym,

`0xFF` — dodatnia liczba całkowita zapisana w formacie szesnastkowym,

`-0x0c` — ujemna liczba całkowita zapisana w formacie szesnastkowym.

Typ float

Jest to liczba zmiennoprzecinkowa (zmiennopozycyjna, rzeczywista). Może przedstawiać zarówno dodatnie, jak i ujemne liczby rzeczywiste. Ich zakres jest zależny od platformy sprzętowo-systemowej, z reguły są to wartości od $-1{,}8 \times 10^{308}$ do $1{,}8 \times 10^{308}$. Do zapisu liczby można używać notacji wykładniczej.

Przykład 6.8

`1,47; -2,346; 0,17E2; -1,0E-2`

Typ string

Jest to typ łańcuchowy, który służy do zapamiętywania ciągu znaków. Pojedynczy znak ciągu jest zapamiętywany w jednym bajcie. Łańcuch znaków można utworzyć, korzystając z jednego z czterech sposobów:

- używając znaków apostrofu,
- używając znaków cudzysłowu,
- używając składni `heredoc`,
- używając składni `nowdoc`.

Użycie znaków apostrofu

Ciąg znakowy można utworzyć poprzez ujęcie go w znaki apostrofu. Taki ciąg pojawi się na ekranie w większości przypadków w niezmienionej postaci. Jeżeli apostrof zostanie wykorzystany jako jeden ze znaków ciągu, to należy poprzedzić go znakiem `\`.

Przykład 6.9

```
<?php
echo 'To jest symbol apostrofu \' użyty w kodzie PHP';
?>
```

Przykład 6.10

```
<?php
$z = 'To jest tekst';
echo $z;
?>
```

Użycie znaków cudzysłowu

Drugim sposobem deklaracji ciągu znakowego jest ujęcie go w znaki cudzysłowu. Jeśli w takim ciągu wystąpi odwołanie do zmiennej, zostanie ono zamienione na wartość przypisaną do zmiennej.

Przykład 6.11

```
<?php
$x = "To jest tekst";
echo "$x";
?>
```

Użycie składni heredoc

Jeżeli wykorzystywana jest składnia `heredoc`, ciąg znaków trzeba rozpocząć od sekwencji <<<. Po tej sekwencji musi wystąpić identyfikator i nowy wiersz, w którym wprowadzany jest ciąg znaków. Ten sam identyfikator musi zostać użyty na końcu ciągu znakowego i zapisany w nowej linii. Nazwa identyfikatora powinna być tworzona według zasad, które obowiązują przy tworzeniu nazw zmiennych. Linia zawierająca identyfikator kończący ciąg nie może zawierać żadnych innych znaków oprócz tego identyfikatora i średnika.

Przykład 6.12

```
<?php
$napis = "napis";
$tekst = <<<TX
  Tutaj rozpoczyna się $napis
TX;
echo $tekst;
?>
```

Jeżeli w ciągu znaków wystąpi odwołanie do zmiennej, podobnie jak w przypadku składni z cudzysłowami, zostanie ono zamienione na wartość przypisaną do zmiennej. Ten sposób tworzenia napisów jest wykorzystywany, kiedy są one długie i składają się z wielu linii.

Przykład 6.13

```
<!DOCTYPE HTML>
<html>
<head>
<title>Skrypt PHP</title>
<meta charset="UTF-8">
</head>
<body>
<?php
$imie = "Anna";
$jezyk1 = "PHP";
$jezyk2 = "JavaScript";
$tekst = <<<TX
  Mam na imię $imie.
  Uczę się programować w języku $jezyk1.
  Umiem już programować w $jezyk2.
TX;
echo $tekst;
?>
</body>
</html>
```

Użycie składni nowdoc

Składnia `nowdoc` jest podobna do składni `heredoc`. Różnica w definicji typu polega na umieszczeniu identyfikatora pomiędzy znakami apostrofu. Wszystkie wymagania dotyczące identyfikatora są takie same jak dla `heredoc`. Różnica w działaniu polega na tym, że gdy w ciągu znaków wystąpi odwołanie do zmiennej, nie jest ona zamieniana na odpowiadającą jej wartość.

Przykład 6.14

```
<?php
$napis = "napis";
$tekst = <<<'PC'
  Tutaj rozpoczyna się $napis
PC;
echo $tekst;
?>
```

Typ array

Tablice przechowują zbiory danych najczęściej jednego typu, a dostęp do nich jest możliwy przez indeks. Tablice mogą być jednowymiarowe lub wielowymiarowe. Wartościami elementów tablic mogą być tablice, dlatego możliwe jest tworzenie tablic wielowymiarowych.

Struktury tablicowe w PHP są tworzone na bieżąco przez przypisanie wartości do poszczególnych indeksów albo za pomocą funkcji `array()`. W PHP można tworzyć tablice zwykłe, które są indeksowane kolejnymi liczbami całkowitymi (tablice indeksowane), oraz tablice asocjacyjne, które są indeksowane kluczem.

Są dwa sposoby tworzenia **tablic indeksowanych**: przez automatyczne tworzenie indeksu lub przez jego ręczne przypisanie.

Funkcja `array` dla tablic indeksowanych tworzonych przez automatyczne przypisanie indeksu ma postać:

```
$tab = array(t1, t2, ..., tn);
```

lub

```
$tab = [t1, t2, ..., tn];
```

Powstała zmienna jest zmienną typu tablicowego, natomiast `t1`, `t2`, …, `tn` to kolejne wartości znajdujące się w tablicy.

Składnia tworzenia tablicy indeksowanej poprzez ręczne przypisanie indeksu ma postać:

```
$tab[0] = t1;
$tab[1] = t2;
$tab[2] = t3;
...
$tab[n - 1] = tn;
```

Numer indeksu może zostać pominięty. Wtedy nawiasy kwadratowe powinny pozostać puste; PHP nada te indeksy automatycznie.

Niezależnie od tego, jak tablica została utworzona, można do niej dodać kolejny element. Jeżeli podczas przypisywania wartości do następnej pozycji tablicy nawiasy kwadratowe pozostaną puste, zostanie jej nadany kolejny indeks. Stosując tę metodę, można uniknąć sytuacji, w której indeksy zostaną źle ponumerowane. Jeżeli w ten sposób będą dodawane wpisy do nowej tablicy, to pierwszy z nich będzie miał indeks równy `0`.

Przykład 6.15

```
<?php
$ks = array("Dżuma", "Potop", "Obcy");
echo $ks[1];
?>
```

Przykład 6.16

```
<?php
$im[0] = "Jan";
$im[1] = "Paweł";
$im[2] = "Michał";
echo $im[2];
?>
```

Przykład 6.17

```
<?php
$tab[] = 1;
$tab[] = 2;
$tab[] = 3;
echo $tab[2];
?>
```

Tablice asocjacyjne, czyli tablice skojarzeniowe (ang. *associative arrays*), to tablice, w których zamiast indeksów liczbowych używa się identyfikatorów (tzw. kluczy). W takich tablicach przechowywane są pary danych: unikatowy klucz i wartość. Dostęp do wartości uzyskuje się poprzez podanie wartości klucza. Różnicą między tablicą indeksowaną a tablicą asocjacyjną jest wartość klucza. Dla tablic indeksowanych klucz jest liczbą, dla tablic asocjacyjnych jest on ciągiem znaków mających określone znaczenie.

Polecenie `array` dla tablic asocjacyjnych ma postać:

```
$tab_a = array(
  klucz1 => a1,
  klucz2 => a2,
  ...,
  kluczn => an
);
```

lub

```
$tab_a[klucz1] = a1;
$tab_a[klucz2] = a2;
  ...,
$tab_a[kluczn] = an;
```

Przykład 6.18

```
<?php
$osoba = array("nazwisko" => "Kowalski", "imię" => "Jan", "wiek" => 27);
echo $osoba["nazwisko"];
?>
```

Przykład 6.19

```
<?php
$osoba["nazwisko"] = "Kowalski";
$osoba["imię"] = "Jan";
$osoba["wiek"] = 27;
echo $osoba["nazwisko"];
?>
```

W języku PHP można tworzyć **tablice wielowymiarowe**. Są to tablice, które zawierają w sobie inne tablice.

Przykład 6.20

```
<?php
$dane = array(
  array("nazwisko" => "Kowalski",
    "imię" => "Jan",
    "wiek" => 27),
  array("nazwisko" => "Nowak",
    "imię" => "Paweł",
    "wiek" => 24),
  array("nazwisko" => "Górka",
    "imię" => "Tomasz",
    "wiek" => 29)
  );
?>
```

Każdy z elementów tablicy `$dane` to tablica asocjacyjna składająca się z trzech elementów: `nazwisko`, `imię`, `wiek`. Aby wyświetlić zawartość tej tablicy, należy użyć indeksu elementu głównego razem z nazwą klucza elementu wewnętrznego.

Przykład 6.21

```
echo $dane[2]["imię"];
```

Wynikiem będzie wyświetlenie tekstu `Tomasz`.

Typ object

Jest to typ służący do przechowywania obiektów. W języku PHP obiekt musi być jawnie zadeklarowany.

Typ resource

Jest to specjalny typ, wskazujący, że zmienna przechowuje odwołanie do zasobu zewnętrznego (aplikacji lub pliku) utworzonego za pomocą specjalnych funkcji.

Typ null

Typ `null` jest przeznaczony dla zmiennych, które zostały zadeklarowane, ale nie przypisano im żadnych wartości. Wielkość liter przy deklaracji tego typu nie ma znaczenia. Poprawne są zapisy: `NULL`, `null`, `Null` i inne.

Deklaracja typu

Język PHP jest językiem słabo typowanym. Typ danych jest automatycznie kojarzony ze zmienną w momencie przypisywania jej wartości. Dlatego możliwe jest na przykład dodawanie ciągu do liczby całkowitej.

W PHP 7 została dodana deklaracja typu. Umożliwia to określenie oczekiwanego typu argumentów podczas deklarowania funkcji. W przypadku niezgodności typów zostanie wygenerowany błąd krytyczny.

Zmiana typu zmiennej

W języku PHP podczas przypisywania zmiennej nowej wartości ponownie ustalany jest jej typ. Czasami zachodzi jednak konieczność zmiany typu danych. Można to zrobić za pomocą rzutowania (`cast`) — wtedy efekt jest jednorazowy — lub za pomocą funkcji `settype()` — wówczas efekt jest trwały.

Rzutowanie typów odbywa się przez podanie nowego typu w nawiasie przed zmienną lub wartością, której typ należy zmienić. Dozwolone są określone typy rzutowań. Są to:

- `integer` — rzutowanie do typu całkowitego,
- `float` (`double`) — rzutowanie do typu rzeczywistego,
- `string` — rzutowanie do ciągu tekstowego,
- `array` — rzutowanie do tablicy,
- `object` — rzutowanie do obiektu.

Przykład 6.22

```
<?php
$x = 23.75;
$y = (integer) $x;
echo "$x <br>";
echo "$y";
?>
```

Zmianę typu w sposób trwały przeprowadza się za pomocą funkcji `settype()`. Funkcja ta ma dwa argumenty. Pierwszym jest nazwa zmiennej, której nadajemy nowy typ, drugim parametr określający nowy typ zmiennej. Parametr ten może przyjmować wartości: `integer`, `float`, `string`, `array`, `object`.

```
settype($zmienna, 'nowy typ');
```

Jeżeli zmiana typu przebiegła pomyślnie, funkcja zwraca wartość `true`. Jeśli zmiana typu nie powiodła się, zwraca wartość `false`.

Przykład 6.23

```
<?php
$x = 97.234;
echo "Zadeklarowana wartość zmiennej \$x: $x <br>";
settype($x, 'string');
echo "Wartość zmiennej \$x po zmianie typu na string: $x <br>";
settype($x, 'integer');
echo "Wartość zmiennej \$x po zmianie typu na integer: $x <br>";
?>
```

Ćwiczenie 6.1

Utwórz trzy zmienne. Pierwszej przypisz wartość liczby całkowitej, drugiej wartość liczby rzeczywistej, trzeciej ciąg znaków. Utwórz w języku PHP skrypt, który będzie wyświetlał wartości tych zmiennych na ekranie.

Ćwiczenie 6.2

Utwórz cztery zmienne. Przypisz im wartości typu `string`. Do tworzenia zmiennych wykorzystaj po kolei każdy z czterech sposobów: użyj znaków apostrofu, znaków cudzysłowu, składni `heredoc` oraz składni `nowdoc`. Utwórz w języku PHP skrypt, który będzie wyświetlał wartości tych zmiennych na ekranie. Przeanalizuj sposób wyświetlania zmiennych i ich wartości.

6.3.4. Stałe

W języku PHP występują stałe, czyli identyfikatory, których wartości nie ulegają zmianie. Stałe, podobnie jak zmienne, są identyfikowane w skrypcie przez nazwę. Nazwa musi spełniać warunki podobne do tych, jakie dotyczą nazwy zmiennej. Jednak w nazwach stałych w odróżnieniu od nazw zmiennych nie używa się znaku dolara (`$`). Zgodnie z przyjętą konwencją nazwy stałych zawsze pisane są wielkimi literami. Zasięg stałych jest globalny i można odwoływać się do nich w każdym miejscu skryptu. Nie można tworzyć stałych poprzez zwykłe przypisanie. Do ich definiowania służy funkcja `define()`, która ma dwa argumenty: nazwę stałej oraz przypisaną do niej wartość. Stałe raz zdefiniowane nie mogą być zmieniane ani usuwane. Mogą zawierać tylko wartości skalarne (typ `boolean`, `integer`, `float` lub `string`).

Definicja stałej ma postać:

```
define("NAZWA_STAŁEJ", wartość);
```

Przykład 6.24

```
<?php
define("WIEK", "21");
echo "Mamy wiek " . WIEK;
?>
```

Stałe predefiniowane

W języku PHP istnieje grupa stałych predefiniowanych, które towarzyszą każdemu uruchamianemu skryptowi. Część z nich jest dostępna dzięki różnym rozszerzeniom i można z nich korzystać, gdy te rozszerzenia zostały wczytane. Przykładowe stałe predefiniowane to:

- `TRUE` — stała zawierająca logiczną wartość prawdy,
- `FALSE` — stała zawierająca logiczną wartość fałszu,
- `PHP_VERSION` — stała reprezentująca aktualnie używaną wersję parsera PHP,
- `PHP_OS` — stała zawierająca nazwę systemu operacyjnego, na którym uruchamiany jest parser PHP.

Istnieje grupa tak zwanych **magicznych stałych**, które zmieniają się w zależności od miejsca ich użycia. Należą do nich:

- `__FILE__` — stała zawierająca nazwę pliku ze skryptem, który jest aktualnie przetwarzany,
- `__LINE__` — stała zawierająca numer linii w skrypcie, która aktualnie jest przetwarzana,
- `__DIR__` — stała zawierająca nazwę katalogu pliku,
- `__FUNCTION__` — stała zawierająca nazwę funkcji,

- `__CLASS__` — stała zawierająca nazwę klasy,
- `__METHOD__` — stała zawierająca nazwę metody.

6.3.5. Operatory i wyrażenia

Operatory służą do wykonywania działań na zmiennych. W języku PHP operatory można podzielić na:

- arytmetyczne,
- porównania,
- bitowe,
- logiczne,
- przypisania,
- łańcuchowe,
- inkrementacji i dekrementacji,
- pozostałe.

Operatory arytmetyczne

Służą do wykonywania operacji arytmetycznych (tabela 6.2).

Tabela 6.2. Operatory arytmetyczne

Operator	Działanie	Przykład
`+`	dodawanie	`$a + $b`
`-`	odejmowanie	`$a - $b`
`*`	mnożenie	`$a * $b`
`/`	dzielenie	`$a / $b`
`%`	dzielenie modulo (reszta z dzielenia)	`$a % $b`
`**`	potęgowanie	`$a ** $b`

Operatory porównania

Operatory porównania porównują argumenty; wynikiem tego porównania jest wartość logiczna `true` (prawda) lub `false` (fałsz). Operatory porównania zostały przedstawione w tabeli 6.3.

Tabela 6.3. Operatory porównania

Operator	Działanie	Przykład
==	Wynik `true`, gdy argumenty są równe.	`$a == $b`
!=	Wynik `true`, gdy argumenty są różne.	`$a != $b`
===	Wynik `true`, gdy argumenty są tego samego typu i są równe.	`$a === $b`
!==	Wynik `true`, gdy argumenty są różne lub są różnych typów.	`$a !== $b`
>	Wynik `true`, gdy argument pierwszy jest większy od drugiego.	`$a > $b`
<	Wynik `true`, gdy argument pierwszy jest mniejszy od drugiego.	`$a < $b`
>=	Wynik `true`, gdy argument pierwszy jest większy od drugiego lub mu równy.	`$a >= $b`
<=	Wynik `true`, gdy argument pierwszy jest mniejszy od drugiego lub mu równy.	`$a <= $b`
<>	Wynik `true`, gdy argumenty są różne.	`$a <> $b`

Operatory bitowe

Operatory bitowe umożliwiają wykonanie operacji na poszczególnych bitach liczb (tabela 6.4).

Tabela 6.4. Operatory bitowe

Operator	Działanie	Przykład
&	iloczyn bitowy (AND)	`$a & $b`
|	suma bitowa (OR)	`$a | $b`
~	negacja bitowa (NOT)	`~$a`
^	bitowa różnica symetryczna	`$a ^ $b`
>>	przesunięcie bitowe w prawo	`$a >> n`
<<	przesunięcie bitowe w lewo	`$a << n`

Operatory logiczne

Operatory logiczne wykonują operacje na argumentach, które mają wartość logiczną (`true` lub `false`). Operatory logiczne zostały przedstawione w tabeli 6.5.

Tabela 6.5. Operatory logiczne

Operator	Działanie	Przykład
`and`	iloczyn logiczny	`$a and $b`
`&&`	iloczyn logiczny	`$a && $b`
`or`	suma logiczna	`$a or $b`
`||`	suma logiczna	`$a || $b`
`!`	negacja logiczna (NOT)	`!$a`
`xor`	różnica symetryczna	`$a xor $b`

Wynik iloczynu logicznego przyjmuje wartość `true` tylko wtedy, gdy obydwa argumenty mają wartość `true`. W pozostałych przypadkach wynik przyjmuje wartość `false`.

Wynik sumy logicznej przyjmuje wartość `false` tylko wtedy, gdy obydwa argumenty mają wartość `false`. W pozostałych przypadkach wynik przyjmuje wartość `true`.

Negacja logiczna zmienia wartość argumentu na przeciwną.

Wynik różnicy symetrycznej przyjmuje wartość `true` tylko wtedy, gdy obydwa argumenty mają różną wartość. Gdy obydwa argumenty mają równą wartość, wynikiem jest `false`.

Operatory przypisania

Operatory przypisania są wykorzystywane do przypisywania wartości argumentom znajdującym się po lewej stronie operatora (tabela 6.6). Oprócz prostego przypisania w języku PHP istnieje zestaw operacji łączących przypisanie z innymi operacjami, na przykład arytmetyczną, bitową lub łańcuchową. Przykładowo zapis: `$a += 4` oznacza w praktyce to samo co zapis: `$a = $a + 4`. Stosowanie takich skróconych zapisów upraszcza tworzenie bardziej rozbudowanych skryptów. W języku PHP występuje cała grupa operatorów tego typu.

Tabela 6.6. Niektóre operatory przypisania

Operator	Przykład	Znaczenie
`=`	`$x = 10`	`$x = 10`
`+=`	`$x += 5`	`$x = $x + 5`
`-=`	`$x -= 5`	`$x = $x - 5`
`*=`	`$x *= 5`	`$x = $x * 5`

Operator	Przykład	Znaczenie
/=	$x /= 5	$x = $x / 5
%=	$x %= 5	$x = $x % 5

Operatory łańcuchowe

Są dwa sposoby zapisu operatora łańcuchowego (konkatenacji). Obydwa pozwalają na łączenie ciągów znakowych i ich wykorzystanie daje ten sam rezultat. Niezależnie od tego, jaki jest typ danych, które będą łączone za pomocą tych operatorów, dane te są traktowane jako ciągi znaków i rezultat ich łączenia jest zawsze ciągiem znaków (tabela 6.7).

Operator łańcuchowy jest oznaczany pojedynczą kropką.

Tabela 6.7. Operatory łańcuchowe

Operator	Działanie	Przykład
.	łączenie łańcuchów znakowych	$x = "moje "."miasto";
.=	dołączanie do łańcucha znakowego	$x = "moje "; $x .= "miasto";

Przykład 6.25

```
<?php
$osoba["nazwisko"] = "Kowalski";
$osoba["imie"] = "Jan";
$osoba["wiek"] = 27;
echo $osoba["nazwisko"] . " " . $osoba["imie"]
. " ma " . $osoba["wiek"] . " lat.";
?>
```

Operatory inkrementacji i dekrementacji

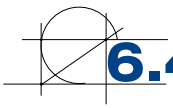
Operatory **inkrementacji** (zwiększania) i **dekrementacji** (zmniejszania) służą do zwiększania lub zmniejszania wartości zmiennej o jeden.

Operator inkrementacji powoduje zwiększenie wartości o jeden. Może występować w postaci przedrostkowej (++$x) lub przyrostkowej ($x++). Operacja $x++ zwiększa wartość zmiennej po jej wykorzystaniu, natomiast ++$x przed jej wykorzystaniem. Oba operatory zwiększają wartość zmiennej, ale nie są równoważne.

Operator dekrementacji działa analogicznie, tylko zamiast zwiększać wartości zmiennych, zmniejsza je.

Przykład 6.26

```
<?php
$x = 7;
echo $x++; echo "<br>";
echo ++$x; echo "<br>";
$v = $x;
$t = $x++;
$z = $x;
$y = ++$x;
echo "Wartość zmiennej \$v = $v <br>";
echo "Wartość zmiennej \$t = $t <br>";
echo "Wartość zmiennej \$z = $z <br>";
echo "Wartość zmiennej \$y = $y <br>";
?>
```



6.4. Instrukcje sterujące

Instrukcje sterujące służą do sterowania kolejnością wykonywania poleceń zapisanych w skrypcie. W języku PHP istnieje cała grupa instrukcji, które pozwalają na zmianę tej kolejności w zależności od spełnienia lub niespełnienia określonych warunków.

6.4.1. Instrukcja warunkowa

Instrukcja warunkowa może występować w kilku wersjach. Najprostsza ma postać:

```
if (warunek) {
   instrukcja1;
   instrukcja2;
   instrukcjaN;
}
```

i działa w następujący sposób: jeśli warunek jest spełniony, wykonywany jest ciąg instrukcji zapisanych między nawiasami `{}`. Jeśli warunek nie jest spełniony, program pomija ten ciąg instrukcji i kontynuuje wykonywanie kolejnych instrukcji. Instrukcja, która jest wykonywana, gdy warunek jest spełniony, może być instrukcją prostą (jedną instrukcją) lub złożoną (blokiem instrukcji).

Przykład 6.27

```
<?php
$x = 11;
$y = 7;
if ($y > $x)
  echo "Wartość zmiennej y jest większa od wartości zmiennej x.";
if ($y < $x)
  echo "Wartość zmiennej x jest większa od wartości zmiennej y.";
?>
```

Jeżeli trzeba sprawdzić dwa wykluczające się warunki, można użyć rozbudowanej instrukcji warunkowej w postaci:

```
if (warunek)
  instrukcja1;
else
  instrukcja2;
```

instrukcja1 i *instrukcja2* mogą być instrukcjami prostymi lub złożonymi. Gdy w bloku `if` lub `else` występuje wiele instrukcji, należy je umieścić w nawiasach klamrowych. Jeżeli w takim przypadku spełniony jest warunek, wykonywana jest *instrukcja1*, w przeciwnym razie *instrukcja2*. Po wykonaniu jednej z tych instrukcji program kontynuuje działanie.

Przykład 6.28

```
<?php
$x = 11;
$y = 7;
if ($y > $x)
  echo "Wartość zmiennej y jest większa od wartości zmiennej x.";
else
  echo "Wartość zmiennej x jest większa od wartości zmiennej y.";
?>
```

Kolejna wersja instrukcji warunkowej pozwala sprawdzać wiele warunków. W tej wersji po `if` może wystąpić kilka następnych bloków `elseif`. Instrukcja ma postać:

```
if (warunek1)
  instrukcja1;
elseif (warunek2)
  instrukcja2;
```

```
  ..
elseif (warunekn)
  instrukcjan;
else
  instrukcjan + 1;
```

Działanie instrukcji polega na sprawdzeniu warunku *warunek1*. Jeżeli jest on spełniony, wykonana zostanie *instrukcja1*. W przeciwnym razie sprawdzany jest *warunek2* i jeżeli jest spełniony, wykonana zostanie *instrukcja2* itd. Jeżeli żaden warunek nie będzie spełniony, wykonana zostanie *instrukcjan* + 1.

Przykład 6.29

```
<?php
$x = 11;
$y = 7;
$z = $x + $y;
if ($z < 0)
  echo "Wartość zmiennej z jest ujemna.";
elseif ($z < 10)
  echo "Wartość zmiennej z jest zawarta w zakresie od 0 do 10.";
elseif ($z < 20)
  echo "Wartość zmiennej z jest zawarta w zakresie od 10 do 20.";
elseif ($z < 30)
  echo "Wartość zmiennej z jest zawarta w zakresie od 20 do 30.";
else
  echo "Wartość zmiennej z jest większa od 30.";
?>
```

6.4.2. Instrukcja switch

Kolejną instrukcją sterującą przepływem jest instrukcja `switch`. Jest to instrukcja wyboru. Wyrażenie występujące w tej konstrukcji jest porównywane z listą szukanych wartości aż do znalezienia pasującej wartości.

```
switch (wyrażenie) {
   case wartość1:
      instrukcja1;
      break;
   case wartość2:
```

```
        instrukcje2;
        break;
    case wartość3:
        instrukcje3;
        break;
    default:
        instrukcje4;
}
```

Działanie instrukcji wygląda następująco: sprawdź wartość wyrażenia (*wyrażenie*) i jeżeli wynikiem jest *wartość1*, wykonaj *instrukcje1* i wyjdź z bloku `switch` (polecenie `break`). Jeśli wynikiem jest *wartość2*, to wykonaj *instrukcje2* i wyjdź z bloku `switch`. Jeżeli wynikiem jest *wartość3*, to wykonaj *instrukcje3* i wyjdź z bloku `switch`. Jeśli wartość jest inna, wykonaj *instrukcje4* i zakończ blok `switch`.

Przykład 6.30

```
<?php
$kolor = "red";
switch ($kolor) {
  case "red":
    echo "Dominującym kolorem jest czerwony!";
    break;
  case "blue":
    echo "Dominującym kolorem jest niebieski!";
    break;
  case "green":
    echo "Dominującym kolorem jest zielony!";
    break;
  default:
    echo "Brak dominującego koloru!";
}
?>
```

6.4.3. Operator warunkowy

Operator warunkowy działa podobnie jak instrukcja `if`. Zwraca wartość jednego z dwóch wyrażeń rozdzielonych dwukropkiem. Ma on postać:

```
warunek ? wartość1 : wartość2
```

i działa w następujący sposób: jeżeli warunek jest prawdziwy, to jako wartość całego wyrażenia podstaw *wartość1*, w przeciwnym razie jako wartość całego wyrażenia podstaw *wartość2*.

Przykład 6.31

```
<?php
$x = 11;
$wynik = ($x < 0) ? "ujemna" : "dodatnia";
echo "Wartość zmiennej x jest $wynik.";
?>
```

6.4.4. Pętle

Pętle są używane do wykonywania powtarzających się czynności. W języku PHP występują następujące rodzaje pętli: `for`, `while`, `do ... while`, `foreach`.

Pętla for

Pętla typu `for` służy do budowania pętli, gdy został podany licznik jej wykonań oraz warunek, który musi być spełniony, aby kolejny raz wykonać pętlę. Oto składnia instrukcji:

```
for (wyrażenie początkowe; wyrażenie warunkowe; wyrażenie modyfikujące) {
  instrukcje do wykonania;
}
```

- *wyrażenie początkowe* — inicjuje zmienną, która jest używana jako licznik pętli,
- *wyrażenie warunkowe* — określa warunek, który musi być spełniony, aby pętla została wykonana kolejny raz,
- *wyrażenie modyfikujące* — modyfikuje zmienną, która jest licznikiem.

Pętlę `for` wykorzystuje się zwykle wtedy, gdy znamy liczbę wykonywanych powtórzeń.

Przykład 6.32

```
<?php
for ($i = 1; $i <= 5; $i++) {
  echo "Pętla wykonana $i raz/y <br>";
}
?>
```

Pętla while

Pętla `while` jest zwykle wykorzystywana wtedy, gdy liczba wykonywanych powtórzeń nie jest znana, a zakończenie pętli zależy od spełnienia określonego warunku. Składnia instrukcji jest następująca:

```
while (warunek) {
  instrukcje;
}
```

Blok instrukcji jest wykonywany w pętli, dopóki wyrażenie warunkowe jest prawdziwe. Konstrukcja mówi: dopóki wyrażenie warunkowe jest prawdziwe, wykonuj instrukcje.

Przykład 6.33

```
<?php
$i = 0;
while ($i++ < 5) {
  echo "Pętla wykonana $i raz/y </br>";
}
?>
```

Pętla do ... while

Pętla `do ... while` jest odmianą pętli `while`. Jej składnia jest następująca:

```
do {
  instrukcje;
}
while (warunek);
```

Konstrukcja mówi: wykonuj instrukcje, dopóki wyrażenie warunkowe jest prawdziwe. W pętli `do ... while` blok instrukcji jest wykonywany co najmniej raz, nawet jeżeli warunek zapisany w wyrażeniu warunkowym jest fałszywy, ponieważ najpierw wykonywany jest ciąg instrukcji, a dopiero potem sprawdzany jest warunek.

Przykład 6.34

```
<?php
$i = 1;
do {
  echo("Pętla wykonana $i raz/y </br>");
}
while ($i++ < 5);
?>
```

Pętla foreach

Pętla `foreach` umożliwia dostęp do elementów tablicy lub właściwości obiektu. Może występować w postaci:

```
foreach ($tablica as $wartość ){
  instrukcje;
}
```

lub

```
foreach ($tablica as $klucz => $wartość){
  instrukcje;
}
```

W drugim przypadku oprócz wartości argumentu tablicy otrzymujemy wartość aktualnego klucza.

Przykład 6.35

```
<?php
$tab = array(
  1 => 'biały',
  2 => 'czarny',
  3 => 'niebieski',
  4 => 'zielony'
);
foreach ($tab as $x){
  echo "$x <br>";
};
?>
```

Jeżeli na listingu chcemy uzyskać nazwy indeksów, musimy zastosować drugą konstrukcję.

Przykład 6.36

```
<?php
$tab = array(
  1 => 'biały',
  2 => 'czarny',
  3 => 'niebieski',
  4 => 'zielony'
```

```
);
foreach ($tab as $kl => $x){
  echo "$kl = $x <br>";
};
?>
```

Instrukcja break

Instrukcja `break` jest instrukcją modyfikującą zachowanie pętli. Służy do przerywania jej wykonywania. Można ją stosować niezależnie od rodzaju pętli.

Przykład 6.37

```
<?php
$i = 0;
while (true) {
  echo("Wypisz $i <br>");
  if ($i >= 20) break;
  $i++;
}
?>
```

Pętla `while` będzie wykonywana, dopóki `$i < 20`, ale jeżeli wartość zmiennej `$i` osiągnie `20`, nastąpi przerwanie wykonywania powtarzających się instrukcji i wyjście z pętli.

Instrukcja continue

Instrukcja `continue`, podobnie jak `break`, służy do modyfikowania zachowania pętli. Po jej napotkaniu następuje przerwanie wykonywania bieżącej iteracji i przejście na początek pętli.

Przykład 6.38

```
<?php
for ($i = 0; $i <= 30; $i++) {
  if (($i % 3) != 0) continue;
  echo "$i; ";
}
?>
```



Rysunek 6.5.
Skrypt wyświetla liczby z zakresu od 0 do 30 podzielne przez 3

Podany kod wyświetli liczby z zakresu od 0 do 30 podzielne przez 3 (rysunek 6.5). Jeżeli wynik dzielenia przez 3 nie jest liczbą całkowitą, następuje przerwanie wykonywania pętli i powrót na jej początek (liczby niepodzielne przez 3 nie będą wyświetlane).

6.4.5. Naprzemienne bloki kodu PHP i HTML

Definiując blok PHP w kodzie HTML, należy korzystać ze znaczników początku i końca PHP. Jeżeli wewnątrz bloku PHP wystąpi kod HTML, który będzie wykonywany opcjonalnie — tylko wtedy, gdy zostanie spełniony określony warunek — to można zdefiniować przełączanie się do trybu HTML wewnątrz bloku warunkowego PHP.

Przykład 6.39

```
<!DOCTYPE HTML>
<html>
<head>
<title>Bloki PHP</title>
<meta charset="UTF-8">
</head>
<?php
$tx = true;
if ($tx) {
?>
<table align="left" border= "1" width="400"
hspace="40" vspace="20" cellspacing="4" >
<tr><td> Nazwisko</td><td> Imię</td><td> Telefon</td></tr>
<tr><td> Nowak</td><td> Adam</td><td> 692399123</td></tr>
<tr><td> Kowalski</td><td> Jan</td><td> 628345621</td></tr>
<tr><td> Górniak</td><td> Mateusz</td><td> 638231484</td></tr>
</table>
<?php
}
?>
</body>
</html>
```

W podanym przykładzie przejście do HTML nastąpi tylko wtedy, gdy warunek `if` będzie spełniony (rysunek 6.6). Jeżeli zmienna `$tx` będzie miała wartość `false`, tabela nie zostanie wyświetlona na stronie.

Nazwisko	Imię	Telefon
Nowak	Adam	692399123
Kowalski	Jan	628345621
Górniak	Mateusz	638231484

Rysunek 6.6. Wynik wykonania kodu HTML i PHP

Ćwiczenie 6.3

Napisz skrypt, który będzie wyświetlał liczby podzielne przez 5 z przedziału od 150 do 100. Liczby powinny być wyświetlane od wartości największej do najmniejszej.

Rozwiązanie

```
<!DOCTYPE HTML>
<html>
<head>
<title>Liczby podzielne przez 5</title>
<meta charset="utf-8">
</head>
<body>
<?php
for ($i = 150; $i >= 100; $i-=5) {
  echo $i . ', ';
}
?>
</body>
</html>
```

Ćwiczenie 6.4

Napisz skrypt, który umieści w tablicy oceny semestralne z pięciu przedmiotów, a następnie wyświetli dane z tablicy w postaci: nazwa przedmiotu i ocena z tego przedmiotu.

Rozwiązanie

```
<!DOCTYPE HTML>
<html>
<head>
<title>Przedmioty i oceny</title>
<meta charset="UTF-8">
</head>
<body>
<?php
$oceny = array(
  "J. polski" => "2",
  "Matematyka" => "4",
```

```
  "Geografia" => "3",
  "Historia" => "5",
  "J. angielski" => "5"
);
foreach ($oceny as $kl => $x) {
  echo "$kl = $x <br>";
};
?>
</body>
</html>
```

Ćwiczenie 6.5

Połącz skrypt PHP z ćwiczenia 6.4 z kodem HTML, tak aby nazwy przedmiotów i oceny z tych przedmiotów były wyświetlane w dwukolumnowej tabeli z nagłówkiem.

Rozwiązanie

```
<!DOCTYPE HTML>
<html>
<head>
<title>Oceny</title>
<meta charset="utf-8">
<style>
table {width: 200px}
table, tr, td {
  border: 1px solid;
  border-collapse: collapse;
}
</style>
</head>
<body>
<table>
<?php
$oceny = array (
  "Przedmiot" => "Ocena",
  "J. polski" => "2",
  "Matematyka" => "4",
```

```
  "Geografia" => "3",
  "Historia" => "5",
  "J. angielski" => "5"
);
foreach ($oceny as $kl => $x) {
  echo '<tr><td>' . $kl . '</td><td>' . $x . '</td></tr>';
}
?>
</table>
</body>
</html>
```

Ćwiczenie 6.6

Utwórz tabelę mnożenia 10 × 10, która będzie wyświetlała wynik mnożenia, na przykład w podany na rysunku 6.7 sposób.

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
1	1	2	3	4	5	6	7	8	9	10
2	2	4	6	8	10	12	14	16	18	20
3	3	6	9	12	15	18	21	24	27	30
4	4	8	12	16	20	24	28	32	36	40
5	5	10	15	20	25	30	35	40	45	50
6	6	12	18	24	30	36	42	48	54	60
7	7	14	21	28	35	42	49	56	63	70
8	8	16	24	32	40	48	56	64	72	80
9	9	18	27	36	45	54	63	72	81	90
10	10	20	30	40	50	60	70	80	90	100

Rysunek 6.7. Tabliczka mnożenia

Rozwiązanie

```
<!DOCTYPE HTML>
<html>
<head>
<title>Tabliczka mnożenia</title>
<meta charset="UTF-8">
<style>
table {
  width: 400px
}
```

```
table, tr, td, th {
  border: 1px solid;
  border-collapse: collapse;
  text-align: center;
}
</style>
</head>
<body>
<table>
<tr>
<th></<th>
<?php
$wiersz = 10;
$kolumna = 10;
for ($i = 1; $i <= $kolumna; $i++){
  echo "<th>" . $i . "</th>";
}
?>
</tr>
<?php
for ($i = 1; $i <= $wiersz; $i++){
  echo "<tr>";
  echo "<th>" . $i . "</th>";
  for ($j = 1; $j <= $kolumna; $j++){
    echo "<td>";
    echo $i * $j;
    echo "</td>";
  }
  echo "</tr>";
}
?>
</table>
</body>
</html>
```

6.5. Funkcje

Funkcja jest ciągiem instrukcji stanowiącym blok kodu, który może być wielokrotnie wykorzystany w różnych programach lub w różnych miejscach programu. Funkcja jest wywoływana przez podanie jej nazwy i listy argumentów. Po zakończeniu działania funkcja często zwraca pewną wartość.

Istnieją dwa rodzaje funkcji. Funkcje, które zostały wbudowane w język, oraz funkcje zdefiniowane przez programistę.

6.5.1. Definiowanie funkcji

W języku PHP można definiować własne funkcje. Funkcję definiujemy za pomocą słowa kluczowego `function`, po którym następuje nazwa funkcji, a dalej w nawiasach okrągłych powinny zostać wymienione argumenty funkcji oddzielone przecinkami. W nawiasach klamrowych jest zapisywane ciało funkcji. Definicja funkcji ma postać:

```
function nazwa($argument1, $argument2, ...) {
  instrukcje
}
```

Jeżeli funkcja ma argumenty, są one zapisane w nawiasach okrągłych jako lista zmiennych oddzielonych przecinkami. Jeżeli funkcja nie ma argumentów, nawiasy okrągłe powinny pozostać puste. Nazwa funkcji powinna być tworzona zgodnie z zasadami obowiązującymi dla nazw zmiennych i powinna odzwierciedlać to, co dana funkcja będzie robiła. Nazwa funkcji nie może zaczynać się od znaku `$`.

Przykład 6.40

```
<?php
function napis() {
  echo "<h2>Programuj w PHP!</h2>";
}
napis();
?>
```

Przykład 6.41

```
<?php
function dodaj($a, $b) {
  $c = $a + $b;
  echo "Wynik dodawania $a i $b to " . $c;
}
dodaj(15, 37);
?>
```

6.5.2. Zwracanie wartości przez funkcje

Jeżeli chcemy, aby funkcja zwracała wartość, która będzie wykorzystywana w innych działaniach, należy użyć słowa kluczowego `return`. Zatrzymuje ono działanie funkcji i zwraca wskazaną wartość do bloku kodu, z którego funkcja została wywołana. Definicja funkcji ze słowem kluczowym `return` ma postać:

```
function nazwa($argument1, $argument2, ...) {
  instrukcje
  return wartość;
}
```

Słowo kluczowe `return` podane wewnątrz funkcji bez żadnego argumentu powoduje przerwanie działania funkcji.

Przykład 6.42

```
<?php
function dodaj($a, $b) {
  $c = $a + $b;
  return $c;
}
$suma = dodaj(14, 7);
echo "Wynik dodawania to $suma";
?>
```

6.5.3. Zasięg zmiennych

Zasięg zmiennej to obszar, w którym można odwoływać się bezpośrednio do zmiennej. Zmienna może być:

1. lokalna,
2. globalna.

Zmienne globalne

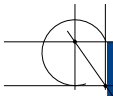
Zmienne **globalne** są widoczne w całym skrypcie. Są to zmienne, które zostały zdefiniowane poza funkcją. Można z nich korzystać w każdym miejscu skryptu z wyjątkiem wnętrza funkcji.

Przykład 6.43

```
<?php
$zm = 1;
function pokaz() {
```

```
  echo "Wartość zmiennej \$zm wynosi $zm. <br>";
}
pokaz();
?>
```

W podanym przykładzie wartość zmiennej `$zm` nie zostanie wyświetlona. Zmienna ma zasięg globalny, ale została zdefiniowana poza funkcją `pokaz()`, w związku z tym funkcja ta nie ma do niej dostępu. Jest to przydatna cecha języka pozwalająca uniknąć konfliktów wynikających ze stosowania takich samych nazw zmiennych używanych w różnych obszarach skryptu. Funkcje wykorzystujące dane zewnętrzne powinny pobierać je w postaci argumentów.



UWAGA

Taki sposób działania zmiennych globalnych jest charakterystyczny dla języka PHP. W większości języków programowania do zmiennych globalnych można odwoływać się z dowolnego miejsca.

Instrukcja global

Jeżeli istnieje konieczność odwołania się do zmiennej globalnej wewnątrz funkcji, można wykorzystać instrukcję `global`. Przed odwołaniem się do zmiennej należy umieścić konstrukcję:

```
global $nazwa zmiennej;
```

Za pomocą jednej instrukcji `global` można zadeklarować w funkcji wiele zmiennych globalnych.

```
global $zm1, $zm2, $zm3;
```

Przykład 6.44

```
<?php
$zm = 1;
function pokaz() {
  global $zm;
  echo "Wartość zmiennej \$zm wynosi $zm. <br>";
}
pokaz();
?>
```

Inną metodą odwołania się do zmiennej globalnej jest odwołanie się do superglobalnej tablicy `$GLOBALS`. Zawiera ona odwołania do wszystkich zdefiniowanych w skrypcie

zmiennych globalnych. Nazwy tych zmiennych są indeksami tablicy `$GLOBALS`. Odwołanie do tablicy ma postać:

```
$GLOBALS['nazwa zmiennej']
```

Przykład 6.45

```
<?php
$zm = 7;
function pokaz() {
  echo "Wartość zmiennej \$zm wynosi ";
  echo $GLOBALS['zm'];
  echo ". <br>";
}
pokaz();
?>
```

Przy posługiwaniu się zmiennymi globalnymi należy zachować szczególną ostrożność, ponieważ każda zmiana ich wartości będzie widoczna w całym skrypcie.

Zmienne lokalne

Zmienne **lokalne** mają zasięg lokalny i są definiowane wewnątrz funkcji. Ich zasięg dotyczy tylko funkcji, w której zostały zdefiniowane, i poza nią nie są widoczne.

Przykład 6.46

```
<?php
function tekst() {
  $tx = "Tekst1";
}
echo "Tekst zapisany w zmiennej to: $tx. <br>";
?>
```

W podanym przykładzie zmienna `$tx` ma zasięg lokalny, została zdefiniowana wewnątrz funkcji `tekst()` i poza tą funkcją nie istnieje. Próba uruchomienia tego kodu spowoduje wyświetlenie komunikatów o błędach.

Zmienne statyczne

Zmienne statyczne to zmienne lokalne funkcji, które zachowują swoją wartość między wywołaniami funkcji. Domyślnie zmienna lokalna jest tworzona w chwili wywołania funkcji i jest widoczna w obrębie tej funkcji. Gdy funkcja zakończy działanie, zmienna znika. Przy ponownym wywołaniu funkcji zmienna jest tworzona i przypisywana jest jej wartość początkowa.

Przykład 6.47

```
<?php
function funk() {
  $i = 1;
  echo "Funkcja wywołana $i raz(y) <br>";
  $i++;
}
funk();
funk();
funk();
?>
```

W podanym przykładzie przy każdym wywołaniu funkcji `funk()` zmienna `$i` będzie przyjmowała wartość `1`, co spowoduje wyświetlenie tego samego komunikatu, mówiącego, że funkcja została wywołana jeden raz (rysunek 6.8).

Funkcja wywołana 1 raz(y)
Funkcja wywołana 1 raz(y)
Funkcja wywołana 1 raz(y)

Rysunek 6.8. Efekt zadeklarowania w treści funkcji zmiennej lokalnej

Jeżeli przy tworzeniu zmiennej zostanie zadeklarowane, że jest to zmienna statyczna, jej wartość zostanie zachowana między kolejnymi wywołaniami funkcji. Deklaracja zmiennej statycznej ma postać:

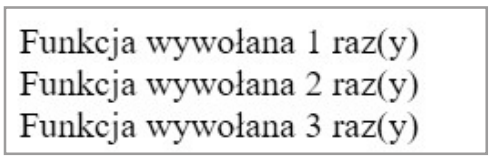
```
static $nazwa zmiennej = wartość;
```

Przykład 6.48

```
<?php
function funk() {
  static $i = 1;
  echo "Funkcja wywołana $i raz(y) <br>";
  $i++;
}
funk();
funk();
funk();
?>
```

W podanym przykładzie zmienna `$i` została zadeklarowana jako zmienna statyczna, dlatego przy pierwszym wywołaniu funkcji zostanie utworzona zmienna statyczna, a przy opuszczaniu funkcji wartość tej zmiennej zostanie zapamiętana. Przy ponownym wywołaniu funkcji instrukcja przypisująca początkową wartość zmiennej `$i` będzie

ignorowana, a stosowana będzie wartość zapamiętana przy poprzednim wywołaniu funkcji (rysunek 6.9).



Rysunek 6.9. Efekt zadeklarowania w treści funkcji zmiennej statycznej

6.5.4. Argumenty funkcji

Argumenty mogą być przekazywane do funkcji na dwa sposoby:

- przez wartość,
- za pomocą referencji.

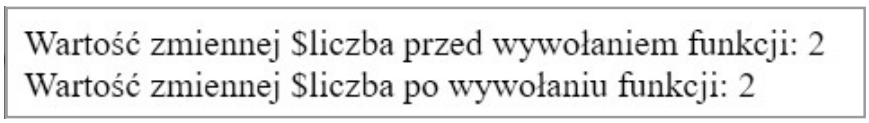
Argumenty przekazywane przez wartość

Domyślnym sposobem przekazywania argumentów do funkcji jest przekazywanie przez wartość. Ten sposób przekazywania był stosowany w prezentowanych do tej pory przykładach. W praktyce oznaczało to, że do funkcji przekazywane są kopie argumentów źródłowych i wszystkie operacje wykonywane są na kopiach. Zostanie to zilustrowane w przykładzie 6.49.

Przykład 6.49

```
<?php
function wart($liczba) {
  $liczba += 3;
}
$liczba = 2;
echo "Wartość zmiennej \$liczba przed wywołaniem funkcji: $liczba <br>";
wart($liczba);
echo "Wartość zmiennej \$liczba po wywołaniu funkcji: $liczba <br>";
?>
```

W przykładzie została zdefiniowana funkcja `wart()` z argumentem `$liczba` (wartość przekazanej zmiennej jest zwiększana o `3`). Przed jej wywołaniem zmienna `$liczba` została ustawiona na wartość `2` i wyświetlona w celu przetestowania jej stanu. Następnie została wywołana funkcja `wart()`. Po jej wykonaniu ponownie została wyświetlona zmienna `$liczba`, aby sprawdzić jej aktualny stan. Wszystkie operacje wewnątrz funkcji odbywały się na kopii zmiennej, natomiast wartość oryginalnej zmiennej nie uległa zmianie (rysunek 6.10).



Rysunek 6.10. Wartość oryginalnej zmiennej nie ulega zmianie

Argumenty przekazywane za pomocą referencji

Jeżeli modyfikowanie kopii przekazanego do funkcji argumentu nie wystarcza, można zastosować przekazywanie argumentów za pomocą referencji. Wtedy funkcja będzie modyfikowała argumenty oryginalne. Przy takim definiowaniu argumentów należy umieścić przed nimi znak `&` (ampersand).

Przykład 6.50

```
<?php
function wart(&$liczba) {
  $liczba += 3;
}
$liczba = 2;
echo "Wartość zmiennej \$liczba przed wywołaniem funkcji: $liczba <br>";
wart($liczba);
echo "Wartość zmiennej \$liczba po wywołaniu funkcji: $liczba <br>";
?>
```

W tym przykładzie wartość zmiennej `$liczba` będzie inna przed wywołaniem funkcji `wart()` i inna po jej wywołaniu (rysunek 6.11).

Wartość zmiennej $liczba przed wywołaniem funkcji: 2
Wartość zmiennej $liczba po wywołaniu funkcji: 5

Rysunek 6.11. Wartość oryginalnej zmiennej ulega zmianie

Domyślne argumenty funkcji

W języku PHP można tworzyć funkcje, których argumenty będą miały określone wartości domyślne. Mechanizm ten pozwala na wywołanie funkcji bez podawania jej argumentów. Jako argumenty zostaną wstawione wartości zdefiniowane podczas tworzenia funkcji. Jeżeli natomiast argumenty będą podane, funkcja zostanie wywołana z tymi argumentami. Definicja funkcji z argumentami domyślnymi ma postać:

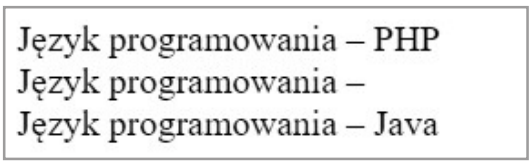
```
function nazwa ($argument1 = wartość, $argument2 = wartość,...) {
  instrukcje
}
```

Przykład 6.51

```
<?php
function progr($język = "PHP") {
  return "Język programowania - $język";
}
```

```
echo progr();
echo "<br>";
echo progr(null);
echo "<br>";
echo progr("Java");
?>
```

Utworzona w przykładzie funkcja ma zdefiniowany argument domyślny. Przy pierwszym wywołaniu funkcja zostanie wykonana z domyślną wartością argumentu, przy drugim wartością argumentu będzie `null`, przy trzecim wartość argumentu zostanie ustawiona zgodnie z argumentem wywołania funkcji (rysunek 6.12).



Rysunek 6.12. Wynik wywołania funkcji z różnymi argumentami

Przykład 6.52

```
<?php
function styl($tekst, $kolor = "red") {
  echo "<p style=\"color: $kolor\">" . $tekst . "</p>";
}
styl("Tytuł", "blue");
styl("Rozdział 1.", "green");
styl("Treść");
?>
```



Rysunek 6.13. Wynik wywołania funkcji z różnymi argumentami

W podanym przykładzie funkcja `styl()` ma dwa argumenty. Pierwszy argument `$tekst` przekazuje treść tekstu do wyświetlenia, drugi `$kolor` przekazuje kolor, w jakim tekst powinien zostać wyświetlony (rysunek 6.13). Wartość domyślna argumentu `$kolor` została ustawiona na `red`. Jeżeli przy wywołaniu funkcji kolor tekstu nie zostanie określony, tekst będzie wyświetlany w kolorze przekazanym jako argument domyślny (`red`).

Wartości domyślne można przypisać dowolnej liczbie argumentów. Ale jeżeli wartość domyślna zostanie przypisana określonemu argumentowi, to wszystkie następne również muszą mieć wartości domyślne.

6.6. Funkcje wbudowane

W języku PHP istnieje duża grupa predefiniowanych funkcji, które są przydatne podczas tworzenia aplikacji internetowych. Funkcje te zostały podzielone na grupy tematyczne — ułatwia to znalezienie funkcji wykonującej określone zadania.

6.6.1. Funkcje tablic

Cała grupa funkcji wbudowanych dotyczy operacji wykonywanych na tablicach. Do najpopularniejszych należą `count()` i `sizeof()`, które zliczają liczbę elementów w tablicy.

Przykład 6.53

```
<?php
$tab = array("e1", "e2", "e3", "e4", "e5");
$dl = count($tab);
for ($i = 0; $i < $dl; $i++) {
  echo $tab[$i], " ";
}
?>
```

Funkcje sortowania

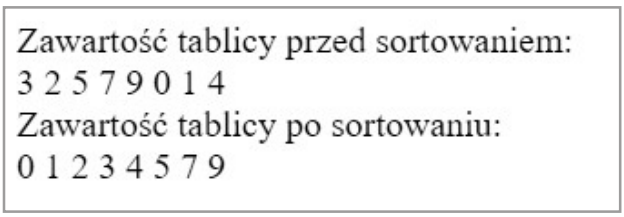
Sortowanie polega na ustawianiu elementów tablicy w określonym porządku, najczęściej rosnącym lub malejącym. Do sortowania tablic można wykorzystać kilka funkcji w zależności od tego, jakie sortowanie będzie realizowane oraz jakiego typu tablic będzie ono dotyczyło.

- `sort()` — sortuje elementy tablicy indeksowanej w kolejności od najmniejszego do największego.
- `rsort()` — sortuje elementy tablicy indeksowanej w kolejności od największego do najmniejszego.
- `asort()` — sortuje elementy tablicy asocjacyjnej według zawartości, w kolejności od najmniejszego do największego.
- `arsort()` — sortuje elementy tablicy asocjacyjnej według zawartości, w kolejności od największego do najmniejszego.
- `ksort()` — sortuje elementy tablicy asocjacyjnej według klucza, w kolejności od najmniejszego do największego.
- `krsort()` — sortuje elementy tablicy asocjacyjnej według klucza, w kolejności od największego do najmniejszego.

Przykład 6.54

```
<?php
$tab = array(3, 2, 5, 7, 9, 0, 1, 4);
echo "Zawartość tablicy przed sortowaniem: <br>";
foreach ($tab as $x) {
  echo "$x ";
}
sort($tab);
echo "<br>Zawartość tablicy po sortowaniu: <br>";
foreach ($tab as $x) {
  echo "$x ";
}
?>
```

Wynik interpretacji kodu został pokazany na rysunku 6.14.



Rysunek 6.14. Wyświetlenie danych pobranych z tablicy

Funkcje wyszukiwania

`in_array()` — funkcja ma dwa argumenty. Pierwszym jest poszukiwana wartość, drugim przeszukiwana tablica. Jeżeli szukana wartość zostanie znaleziona, funkcja zwróci wartość `true`, w przeciwnym razie zwróci wartość `false`.

Ćwiczenie 6.7

Utwórz tablicę 20-elementową. Wartości tablicy to dowolne liczby całkowite. Napisz skrypt obliczający, ile jest w tablicy liczb parzystych, a ile nieparzystych.

Ćwiczenie 6.8

Utwórz tablicę asocjacyjną składającą się z 15 elementów. Nazwy kluczy to: l1, l2 itd. Wartości to losowo podane liczby całkowite. Wyświetl pary klucz–liczba dla liczb, które są podzielne przez 3 i 4.

6.6.2. Funkcje daty i czasu

Podczas pisania skryptów często wymagane jest podanie daty lub czasu. Język PHP dysponuje dużą grupą funkcji, których zadaniem jest wykonywanie operacji na dacie i czasie.

Funkcja time()

Funkcja `time()` zwraca informacje na temat bieżącej daty i czasu. Nie ma żadnych argumentów. Informacje na temat bieżącej daty i czasu są zwracane w postaci liczby.

Odpowiada ona liczbie sekund, które upłynęły od godziny 00:00:00 1 stycznia 1970 roku do bieżącej daty. Jest to tak zwany *znacznik_czasu* (timestamp).

Przykład 6.55

```
<?php
echo time();
?>
```

Taki zapis daty i czasu pozwala na wykonywanie działań na dacie, na przykład zwiększanie daty o jeden dzień, czasu o jedną godzinę, minutę lub sekundę. Wystarczy do znacznika czasu dodać wartość określającą liczbę sekund, jakie upłynęły, aby otrzymać interesujący nas wynik. Następnie za pomocą funkcji PHP można uzyskany wynik przekształcić na bardziej czytelną postać.

Funkcja getdate()

Drugą funkcją związaną z przekazywaniem informacji o dacie i czasie jest funkcja getdate(). Ma ona postać:

```
getdate([znacznik_czasu])
```

Argument *znacznik_czasu* jest opcjonalny. Jeżeli nie zostanie podany, działania wykonywane przez funkcję będą się odnosić do daty bieżącej. Wynikiem wykonania funkcji jest tablica asocjacyjna zawierająca dane dotyczące daty i czasu. Indeksy oraz wartości tej tablicy dotyczą poszczególnych elementów wchodzących w skład daty i czasu. Zostały one przedstawione w tabeli 6.8.

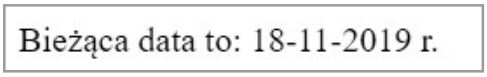
Tabela 6.8. Indeksy tablicy zwracanej przez funkcję getdate()

Nazwa indeksu	Znaczenie indeksu	Opis
seconds	Liczba sekund	od 0 do 59
minutes	Liczba minut	od 0 do 59
hours	Godzina	od 0 do 23
mday	Dzień miesiąca	od 1 do 31
wday	Dzień tygodnia w postaci liczby	od 0 (niedziela) do 6 (sobota)
mon	Miesiąc w postaci liczby	od 1 do 12
year	Rok w postaci czterocyfrowej	na przykład 2005
yday	Numer kolejnego dnia roku	od 0 do 365
weekday	Nazwa dnia tygodnia (po angielsku)	na przykład Monday
month	Nazwa miesiąca (po angielsku)	na przykład January
0	Aktualny znacznik czasu	

Przykład 6.56

```
<?php
$data = getdate();
$dzien = $data["mday"];
$miesiac = $data["mon"];
$rok = $data["year"];
if ($dzien < 10) $dzien = "0" . $dzien;
if ($miesiac < 10) $miesiac = "0" . $miesiac;
echo "Bieżąca data to: $dzien-$miesiac-$rok r.";
?>
```

W podanym przykładzie zmiennej `$data` została przypisana tablica zwrócona przez funkcję `getdate()`. Wartości indeksów `mday`, `mon`, `year` zostały przypisane zmiennym `$dzien`, `$miesiac`, `$rok`. Gdy wartości zmiennych `$dzien` i `$miesiac` będą mniejsze od `10`, zostanie do nich dodany znak `0`. Po połączeniu wszystkich zmiennych w jeden ciąg jest on wyświetlony instrukcją `echo` (rysunek 6.15).



Rysunek 6.15. Wykorzystanie funkcji daty do wyświetlenia bieżącej daty

Funkcja date()

Funkcją, która pozwala na formatowanie w odpowiedni sposób daty i czasu, jest funkcja `date()`. Ma ona postać:

```
date(format[, znacznik_czasu])
```

Parametr *`format`* określa, jakie informacje na temat daty i czasu powinny zostać zwrócone. Parametr *`znacznik_czasu`* zawiera opisany już znacznik czasu. Jest on opcjonalny i określa interesującą nas datę. Jeżeli zostanie pominięty, funkcja zwróci datę bieżącą.

Parametr *`format`* jest ciągiem znaków składającym się ze znaczników. Opis niektórych znaczników został podany w tabeli 6.9.

Tabela 6.9. Niektóre znaczniki formatujące funkcji date()

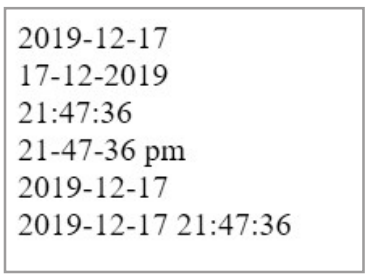
Znacznik	Znaczenie	Opis
`a`	Użycie określenia „przed południem” (am) lub „po południu” (pm)	am, pm
`A`	Użycie określenia „przed południem” (AM) lub „po południu” (PM)	AM, PM
`c`	Data i czas zgodne z formatem ISO 8601	2013-03-03T15:25:20

Znacznik	Znaczenie	Opis
d	Dzień miesiąca w formacie z zerem na początku	od 01 do 31
D	Dzień tygodnia w formacie trzyliterowego skrótu	Mon, Tue
F	Pełna nazwa miesiąca	January
g	Godzina w formacie dwunastogodzinnym bez zera na początku	od 1 do 12
G	Godzina w formacie dwudziestoczterogodzinnym bez zera na początku	od 1 do 24
H	Godzina w formacie dwudziestoczterogodzinnym z zerem na początku	od 01 do 24
i	Liczba minut z zerem na początku	od 01 do 59
l	Nazwa dnia tygodnia	Monday
m	Miesiąc w postaci liczby dwucyfrowej z zerem na początku	od 01 do 12
s	Liczba sekund z zerem na początku	od 01 do 59
Y	Rok w postaci czterech znaków	2013

Przykład 6.57

```
<?php
echo date("Y-m-d") . "<br>";
echo date("d-m-Y") . "<br>";
echo date("G:i:s") . "<br>";
echo date("H-i-s a") . "<br>";
echo date("Y-m-d") . "<br>";
echo date("Y-m-d G:i:s") . "<br>";
?>
```

Funkcja `date()` zwraca angielskie nazwy miesięcy i dni tygodnia. Przykład 6.57 pokazuje użycie tej funkcji z różnymi znacznikami formatującymi. Na rysunku 6.16 widoczny jest rezultat wykonania kodu.



Rysunek 6.16. Rezultat użycia funkcji date() z różnymi znacznikami formatującymi

Funkcja mktime()

Funkcja `mktime()` zwraca znacznik czasu daty podanej jako argument funkcji. Może mieć od zera do sześciu argumentów w postaci liczb całkowitych. Są to kolejno:

- godzina,
- minuta,
- sekunda,
- miesiąc,
- dzień miesiąca,
- rok.

Znacznik czasu, który zwróci funkcja `mktime()`, może zostać wykorzystany w funkcjach `getdate()` i `date()`.

Przykład 6.58

```
<?php
$czas = mktime(16, 30, 0, 10, 24, 2020);
echo "Data: dzień, miesiąc, rok, godzina:minuta <br>";
echo date("d-m-Y G:i", $czas) . "<br>";
echo "Data: rok, miesiąc, dzień, godzina:minuta:sekunda <br>";
echo date("Y-m-d G:i:s", $czas);
?>
```

Wynik interpretacji kodu został pokazany na rysunku 6.17.

Data: dzień, miesiąc, rok, godzina:minuta
24-10-2020 16:30
Data: rok, miesiąc, dzień, godzina:minuta:sekunda
2020-10-24 16:30:00

Rysunek 6.17. Wykorzystanie funkcji mktime() do ustawienia znacznika czasu

Ćwiczenie 6.9

Napisz skrypt, który na podstawie danych pobranych z tablicy zwracanej przez funkcję `getdate()` wyświetli bieżącą datę. W dacie zostanie podana nazwa miesiąca w postaci tekstu w języku polskim.

Ćwiczenie 6.10

Napisz skrypt, który będzie wyświetlał bieżący dzień tygodnia w podanej postaci: Dzisiaj jest [*dzień tygodnia*].

Ćwiczenie 6.11

Napisz skrypt wyświetlający liczbę dni, które upłynęły od początku bieżącego roku, oraz liczbę dni, które pozostały do końca bieżącego roku.

6.6.3. Funkcje formatowania ciągów

Gdy pracuje się z ciągami znakowymi, często trzeba formatować je tak, aby były wyświetlane w określony sposób. Do tego celu można wykorzystywać funkcje formatujące. W języku PHP istnieje wiele funkcji, za pomocą których możemy ustalić określony wygląd ciągu.

Funkcja nl2br()

Jeżeli wyświetlamy w przeglądarce blok tekstu, który zawiera znaki końca linii, to przeglądarka nie uwzględni tych znaków. Można ominąć ten problem, dodając znaczniki `<br>` lub `<p>`. Ale nie zawsze takie rozwiązanie jest możliwe, szczególnie wtedy, gdy tekst jest wczytywany z pliku lub bazy danych. W takiej sytuacji można wykorzystać funkcję:

```
nl2br("ciąg_znaków")
```

Funkcja ta dla wybranego bloku tekstu przed każdym znakiem końca linii automatycznie wstawi znacznik `<br>` i zwróci przetworzony tekst.

Przykład 6.59

```
<!DOCTYPE HTML>
<html>
<head>
<title>Funkcja nl2br</title>
<meta charset="UTF-8">
<style>
p {
  font-weight: bold;
}
</style>
</head>
<body>
<?php
$tekst = <<<TX
  Na cóż czekamy, zebrani na rynku?
  Dziś mają tu przyjść barbarzyńcy.
  Dlaczego taka bezczynność w senacie?
  Senatorowie siedzą-czemuż praw nie uchwalą?
TX;
```

```
echo "<p>Tekst przed użyciem funkcji nl2br():</p>";
      echo $tekst."<br>";
      echo "<p>Tekst po użyciu funkcji nl2br():</p>";
      echo nl2br($tekst);
      ?>
      </body>
      </html>
```

Wynik interpretacji kodu został pokazany na rysunku 6.18.

Tekst przed użyciem funkcji nl2br():

Na cóż czekamy, zebrani na rynku? Dziś mają tu przyjść barbarzyńcy. Dlaczego taka bezczynność w senacie? Senatorowie siedzą

Tekst po użyciu funkcji nl2br():

Na cóż czekamy, zebrani na rynku?
Dziś mają tu przyjść barbarzyńcy.
Dlaczego taka bezczynność w senacie?
Senatorowie siedzą – czemuż praw nie uchwalą?

Rysunek 6.18. Wynik zastosowania do tekstu funkcji nl2br()

Funkcja wordwrap()

Do formatowania tekstu w postaci kolumny o określonej szerokości można wykorzystać funkcję `wordwrap()`. Dzieli ona ciąg podany jako argument na linie o maksymalnej długości 75 znaków. Do rozdzielenia linii domyślnie jest używany znak `\n`. Oprócz argumentu określającego ciąg źródłowy funkcja ma jeszcze trzy opcjonalne argumenty. Są to: liczba wskazująca maksymalną długość linii, ciąg znaków zastosowany do rozdzielenia linii oraz argument podziału słów dłuższych niż zadeklarowana maksymalna długość linii.

Przykład 6.60

```
<?php
$tekst = <<<TX
Na cóż czekamy, zebrani na rynku? Dziś mają tu przyjść barbarzyńcy.
Dlaczego taka bezczynność w senacie? Senatorowie siedzą-
czemuż praw nie uchwalą?
TX;
echo wordwrap($tekst);
?>
```

W podanym przykładzie funkcja `wordwrap()` została wywołana z jednym argumentem, a więc linie zostaną rozdzielone znakiem `\n`, który nie jest interpretowany przez

przeglądarki (rysunek 6.19). Po zastosowaniu dwóch kolejnych argumentów kod wysłany do przeglądarki pozwoli na wyświetlenie tekstu w liniach (rysunek 6.20).

Na cóż czekamy, zebrani na rynku? Dziś mają tu przyjść barbarzyńcy. Dlaczego taka bezczynność w senacie? Senatorowie siedzą

Rysunek 6.19. Brak interpretacji przez przeglądarkę znaku nowej linii

Przykład 6.61

```
<?php
$tekst = <<<TX
Na cóż czekamy, zebrani na rynku? Dziś mają tu przyjść barbarzyńcy.
Dlaczego taka bezczynność w senacie? Senatorowie siedzą –
czemuż praw nie uchwalą?
TX;
echo wordwrap($tekst, 30, "<br>\n");
?>
```

Na cóż czekamy, zebrani na
rynku? Dziś mają tu
przyjść barbarzyńcy.
Dlaczego taka bezczynność w
senacie? Senatorowie siedzą
– czemuż praw nie uchwalą?

Rysunek 6.20. Podział tekstu na linie o długości 30 znaków

Ostatni, czwarty argument funkcji będzie zastosowany, gdy w linii pojawi się słowo dłuższe niż zadeklarowana maksymalna długość linii. Domyślnie takie słowo nie zostanie podzielone między liniami. Użycie czwartego argumentu wymusi podział słowa między liniami. Aby się nim posłużyć, trzeba przypisać mu wartość logiczną `true`.

Przykład 6.62

```
<?php
$tekst = "Potrzebne informacje znajdują się pod adresem: ";
$tekst .= "http://helion.pl/kategorie/
podreczniki-szkolne/technik-informatyk";
echo wordwrap($tekst, 40, "<br>\n", true);
?>
```

Wynik interpretacji kodu został pokazany na rysunku 6.21.

Potrzebne informacje znajdują się pod
adresem:
http://helion.pl/kategorie/podreczniki-s
zkolne/technik-informatyk

Rysunek 6.21. Wymuszenie podziału długiego słowa między liniami

Funkcje zmiany wielkości liter

Częstym działaniem na ciągach znakowych jest zamiana wszystkich liter na duże lub małe. Do zamiany wszystkich liter na duże służy funkcja:

```
strtoupper(argument)
```

Ciąg podany w postaci argumentu zostanie zmodyfikowany w ten sposób, że wszystkie litery zostaną zamienione na duże.

Do zamiany wszystkich liter na małe służy funkcja:

```
strtolower(argument)
```

Ciąg podany w postaci argumentu zostanie zmodyfikowany w ten sposób, że wszystkie litery zostaną zamienione na małe.

Jeżeli ciąg znakowy użyty jako argument funkcji `strtoupper()` lub `strtolower()` zawiera znaki narodowe (na przykład literę ę lub ą), to nie zostaną one zamienione.

Funkcje, które przy zamianie liter na małe lub duże uwzględniają sposób ich kodowania, to `mb_strtoupper()` i `mb_strtolower()` zapisywane w postaci:

```
mb_strtoupper(argument, [kodowanie])
mb_strtolower(argument, [kodowanie])
```

gdzie `[kodowanie]` to sposób kodowania znaków.

Kolejnymi funkcjami o podobnym działaniu są:

```
ucfirst(argument)
```

i

```
ucwords(argument)
```

Funkcja `ucfirst()` zmodyfikuje podany argument w ten sposób, że pierwsza litera ciągu zostanie zamieniona na dużą literę.

Funkcja `ucwords()` zmodyfikuje ciąg podany jako argument w ten sposób, że wszystkie pierwsze litery wyrazów zostaną zamienione na duże litery.

Funkcje usuwania ciągu znaków

Do usunięcia białych znaków z początku lub końca ciągu można użyć jednej z trzech funkcji: `trim()`, `ltrim()`, `rtrim()`.

Wszystkie mają taką samą konstrukcję:

```
nazwa funkcji("ciąg_znakowy")
```

Usuwają one następujące znaki: znak spacji (kod `32`), znak tabulacji (kod `9`), znak nowej linii (kod `10`), znak powrotu karetki (kod `13`), znak tabulacji pionowej (kod `11`), znak o kodzie `0`.

- `trim()` — usuwa podane znaki z początku i końca ciągu,
- `ltrim()` — usuwa podane znaki z początku ciągu,
- `rtrim()` — usuwa podane znaki z końca ciągu.

Do usuwania innych znaków należy użyć podanych wyżej funkcji z dodatkowym parametrem w postaci:

```
nazwa funkcji("ciąg_znakowy", "znaki do usunięcia")
```

6.6.4. Funkcje analizowania ciągów znaków

Funkcja sprawdzająca długość ciągu

Do sprawdzenia długości ciągu znaków jest wykorzystywana funkcja `strlen()`. Funkcja ta zwraca prawidłowy wynik, jeśli ciąg znaków nie zawiera znaków narodowych. Funkcja, która uwzględnia sposób ich kodowania, to `mb_strlen()`.

Przykład 6.63

```
<?php
$napis = "Senatorowie siedzą - czemuż praw nie uchwalą?";
echo 'Tekst: "'.$napis.'"';
$dl = mb_strlen($napis);
echo " ma długość $dl znaków.<br>";
?>
```

W podanym przykładzie zostanie wyświetlony tekst oraz jego długość.

Indeksowanie ciągu znaków

Z utworzonego ciągu znaków podobnie jak z tablicy można pobierać pojedyncze znaki. Każdy znak ciągu ma swoje położenie, które można określić poprzez indeks. Znak na pierwszej pozycji ma indeks `0`, następny `1`. Wartość indeksu dla kolejnych znaków rośnie o 1.

Przykład 6.64

```
<?php
$napis = "Litwo, Ojczyzno moja!";
echo $napis[0];
echo $napis[10];
?>
```

W podanym przykładzie dla zmiennej `$napis` w nawiasach kwadratowych został podany indeks znaku, który powinien być wyświetlony. Zostanie wyświetlony znak

znajdujący się na pozycji o indeksie `0` (litera `L`) oraz znak znajdujący się na pozycji o indeksie `10` (litera `z`).

Znajdowanie podciągów

Funkcja strstr()

Do sprawdzenia, czy podany ciąg jest fragmentem innego ciągu, służy funkcja `strstr()` w postaci:

```
strstr(argument1, argument2)
```

Pierwszy argument jest przeszukiwanym ciągiem źródłowym, drugi poszukiwanym ciągiem. Funkcja zwróci wartość `false`, gdy podciąg nie zostanie znaleziony. Jeżeli szukany ciąg jest fragmentem ciągu źródłowego, to funkcja zwróci fragment ciągu źródłowego od znalezionego podciągu do jego końca. Funkcja `strstr()` rozróżnia wielkość liter. Jeżeli wielkość liter nie powinna mieć znaczenia, podczas przeszukiwania ciągu źródłowego należy użyć funkcji `stristr()`. Działa ona podobnie jak funkcja `strstr()`, ale nie rozróżnia wielkości liter.

Przykład 6.65

```
<?php
$dane = "Jan Kowalski, ul. Nowa 23, 80-874 Warszawa, tel. 693341678";
$tel = strstr($dane, "tel.");
echo $tel;
?>
```

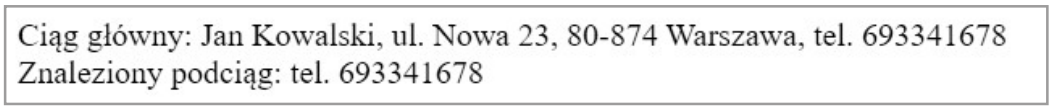
W podanym przykładzie zostanie wyświetlony tekst od znalezionego podciągu `tel.` do końca przeszukiwanego ciągu.

Funkcja zwróci wartość `false`, gdy ciąg nie zostanie znaleziony, zatem może być wykorzystana w instrukcjach warunkowych.

Przykład 6.66

```
<?php
$dane = "Jan Kowalski, ul. Nowa 23, 80-874 Warszawa, tel. 693341678";
echo "Ciąg główny: " . $dane . "<br>";
$tel = strstr($dane, "tel.");
if ($tel == false)
  echo "Brak numeru telefonu";
else
  echo "Znaleziony podciąg: " . $tel;
?>
```

Jeżeli w ciągu występuje podciąg `tel.`, wynikiem będzie wyświetlenie podciągu z numerem telefonu (rysunek 6.22), w przeciwnym razie wyświetli się komunikat o braku numeru telefonu.



Rysunek 6.22. Znaleziony podciąg w ciągu głównym

Funkcja strpos()

Podobne działanie do funkcji `strstr()` ma funkcja `strpos()`. Funkcja ta sprawdza, czy szukany podciąg znajduje się w ciągu źródłowym, a jeżeli tak, zwraca jego położenie. Ma trzy argumenty: pierwszy to przeszukiwany ciąg źródłowy, drugi to poszukiwany podciąg, trzeci to pozycja w ciągu źródłowym, od której rozpocznie się przeszukiwanie. Trzeci argument jest opcjonalny. Jeżeli podciąg nie zostanie znaleziony, zwrócona zostanie wartość `false`. Gdy podciąg zostanie znaleziony, funkcja zwróci indeks określający miejsce znalezienia podciągu. Indeksowanie pozycji w ciągu rozpoczyna się od `0`.

Funkcja substr()

Funkcja `substr()` zwraca część ciągu źródłowego. Ma trzy argumenty: pierwszy to ciąg źródłowy, drugi to indeks wskazujący miejsce początku zwracanego podciągu, trzeci, opcjonalny, określa długość pobranego ciągu. Jeżeli zostanie podany trzeci argument, zwrócona zostanie określona liczba znaków. Gdy ten argument zostanie pominięty, zostanie zwrócony podciąg od wskazanego indeksu do końca ciągu.

Przykład 6.67

```
<?php
$dane = "Jan Kowalski, ul. Nowa 23, 80-874 Warszawa, tel. 693341678";
echo substr($dane, 4, 8);
?>
```

W podanym przykładzie zostanie wyświetlony tekst `Kowalski`.

Jeżeli jako drugi argument funkcji (indeks) zostanie podana wartość ujemna, pozycja indeksu będzie liczona od końca ciągu źródłowego.

Przykład 6.68

```
<?php
$dane = "Jan Kowalski, ul. Nowa 23, 80-874 Warszawa, tel. 693341678";
echo substr($dane, 4, 8)."<br>";
echo substr($dane, -14, 14);
?>
```

Funkcja strtok()

Za pomocą funkcji `strtok()` można podzielić ciąg źródłowy na podciągi. Funkcja ma dwa argumenty: pierwszy to ciąg źródłowy, który zostanie podzielony, drugi to ciąg znaków rozdzielających podciągi. Funkcja ta przy pierwszym wywołaniu zwraca pierwszy wydzielony ciąg i zapamiętuje przeszukiwany ciąg w pamięci podręcznej. Przy kolejnym wywołaniu funkcja zwraca kolejne ciągi. Jeżeli zostanie osiągnięty koniec ciągu źródłowego, funkcja zwróci wartość `false`. Z powodu takiego działania funkcja ta najczęściej jest wywoływana w pętli.

Przykład 6.69

```
<?php
$dane = "Jan Kowalski, ul. Długa 23, 80-874 Gdańsk, tel. 693341678";
$znak = ",";
$ciag = strtok($dane, $znak);
while (is_string($ciag)) {
  if ($ciag) {
    echo "$ciag<br>";
  }
  $ciag = strtok($znak);
}
?>
```

Użyta w przykładzie funkcja `is_string()` sprawdza, czy typ zmiennej podanej jako argument jest ciągiem. Wywołana drugi raz funkcja `strtok()` nie zawiera jako argumentu ciągu źródłowego. Podanie tego argumentu spowodowałoby przeszukiwanie ciągu od początku. Wynik interpretacji kodu został pokazany na rysunku 6.23.

Jan Kowalski
ul. Długa 23
80-874 Gdańsk
tel. 693341678

Rysunek 6.23. Wynik podzielenia ciągu głównego. Znakiem rozdzielającym był przecinek

Porównania ciągów

Działaniem niezbędnym podczas pracy z ciągami jest ich porównywanie. Porównywanie ciągów znakowych można wykonywać za pomocą operatorów porównania lub funkcji porównujących.

Funkcja strcmp()

Funkcja `strcmp()` ma postać:

```
strcmp("ciąg1", "ciąg2")
```

- zwraca wartość mniejszą od `0`, jeżeli *`ciąg1`* jest mniejszy od *`ciąg2`*,
- zwraca wartość większą od `0`, jeżeli *`ciąg1`* jest większy od *`ciąg2`*,

- zwraca `0`, jeżeli ciągi są równe.

Funkcja rozróżnia wielkość znaków.

Funkcja strcasecmp()

Funkcja `strcasecmp()` działa jak funkcja `strcmp()`, ale nie uwzględnia wielkości liter. Ma postać:

```
strcasecmp("ciąg1", "ciąg2")
```

Ćwiczenie 6.12

Napisz skrypt, który będzie wyszukiwał określony wyraz w podanym ciągu i obliczał, ile razy ten wyraz w nim wystąpił.

Ćwiczenie 6.13

Napisz skrypt, który będzie zawierał tablicę wybranych wyrazów oraz ciąg źródłowy. Każde wystąpienie w tym ciągu wyrazu z tablicy powinno spowodować zamianę tego wyrazu na tekst `wyraz z tablicy`.

6.7. Funkcje obsługi plików

W języku PHP istnieją funkcje umożliwiające przeprowadzanie różnych operacji na plikach. Pozwalają one na odczytywanie informacji o strukturze plików i katalogów oraz na odczytywanie danych i zapisywanie ich do plików.

6.7.1. Dołączanie plików

Jeżeli skrypt PHP zawiera dużą ilość kodu, to można go podzielić i zapisać w kilku oddzielnych plikach. Do ponownego ich dołączenia można użyć polecenia `include` lub `require`. Obydwa polecenia wstawiają zawartość wskazanego pliku w miejscu, w którym wystąpią.

Dołączony skrypt zostanie wykonany tak, jakby był częścią kodu, w którym został wstawiony. Polecenie `include` ma postać:

```
include 'nazwa_pliku'
```

Polecenie `require` ma postać:

```
require 'nazwa_pliku'
```

Przykład 6.70

skrypt1.php

```
<?php
echo "<p>Plik został dołączony</p>";
?>
```

strona1.php

```
<!DOCTYPE HTML>
<html>
<head>
<title>Strona1</title>
</head>
<body>
<p>Witamy na pierwszej stronie</p>
<?php
include 'skrypt1.php';
?>
</body>
</html>
```

Jeżeli obydwa pliki zostaną umieszczone w tym samym folderze, to do uruchomionego pliku *strona1.php* zostanie dołączona zawartość pliku *skrypt1.php*. Ponieważ polecenie `include` jest częścią języka PHP, to w kodzie HTML został użyty znacznik `<?php`, aby przełączyć się na ten tryb. Po wykonaniu polecenia tryb PHP zostaje wyłączony i dalsza część skryptu będzie przetwarzana jako kod HTML.

Różnica między poleceniami `include` i `require` występuje tylko wtedy, gdy dołączany plik nie może zostać odczytany. Polecenie `include` wygeneruje ostrzeżenie, ale skrypt zawierający jego wywołanie będzie nadal działał. Natomiast użycie polecenia `require` spowoduje zgłoszenie błędu i zakończenie działania skryptu.

Plik do dołączenia najłatwiej wskazać przez podanie ścieżki dostępu do niego (względnej lub bezwzględnej).

Przykład 6.71

```
include './skrypty/skrypt1.php';
include '/moja_strona/skrypty/skrypt1.php';
```

W pierwszym przypadku dołączony zostanie plik znajdujący się w podkatalogu *skrypty* katalogu bieżącego. W drugim przypadku dołączony zostanie plik znajdujący się w podkatalogu *moja_strona/skrypty* katalogu głównego.

Dołączone pliki mogą zwracać wartość tak jak funkcje. Wywołanie instrukcji `return` wewnątrz dołączonego kodu kończy jego działanie.

6.7.2. Operacje na plikach

Sprawdzanie, czy plik istnieje

Do ustalenia, czy plik lub katalog istnieje, służy funkcja `file_exists()` zapisywana w postaci:

```
file_exists('nazwa_pliku')
```

Jako argument funkcji występuje nazwa pliku lub katalogu wraz ze ścieżką dostępu. Jeśli plik istnieje, funkcja zwraca wartość `true`, w przeciwnym razie zwraca wartość `false`.

Przykład 6.72

```
if (file_exists('skrypt.php')) {
  echo "Plik został znaleziony.";
}
```

Funkcja `is_file()` sprawdza, czy podany jako jej argument ciąg wskazuje na plik. Ma postać:

```
is_file(plik)
```

Przykład 6.73

```
<?php
$p = 'plik.txt';
if (is_file($p)) {
  echo "To jest plik";
}
?>
```

Funkcja zwraca wartość `true`, jeżeli argument funkcji wskazuje na plik.

Rozmiar pliku

Do określenia rozmiaru pliku służy funkcja `filesize()` w postaci:

```
filesize('nazwa_pliku')
```

Funkcja zwraca wartość typu `integer`, która określa wielkość pliku w bajtach. Wiedza na temat rozmiaru pliku jest potrzebna na przykład wtedy, gdy plik ma zostać dołączony jako załącznik do wiadomości elektronicznej lub gdy rozmiar pliku musi zostać wyświetlony, zanim zacznie się jego pobieranie.

Tworzenie i usuwanie pliku

Do tworzenia pliku używana jest funkcja `touch()` zapisywana w postaci:

```
touch('nazwa_pliku')
```

Funkcja tworzy pusty plik o podanej nazwie. Jeżeli istnieje już plik o takiej nazwie, nie ulegnie on zmianie. Zmieniona zostanie tylko data jego modyfikacji.

Do usuwania istniejącego pliku używana jest funkcja `unlink()` w postaci:

```
unlink('nazwa_pliku')
```

Funkcja zwraca wartość `true`, jeżeli plik został usunięty. W przeciwnym razie zwraca wartość `false`.

Otwieranie, zapisywanie i zamykanie pliku

Za pomocą funkcji `fopen()` można otwierać istniejące pliki. Funkcja ma postać:

```
fopen('nazwa_pliku', 'tryb_otwarcia')
```

Pierwszy argument funkcji określa ścieżkę do pliku, który ma zostać otwarty. Drugi określa tryb, w jakim plik zostanie otwarty. Najczęściej stosowane tryby otwarcia to:

- `r` — plik zostanie otwarty w trybie tylko do odczytu,
- `w` — plik zostanie otwarty w trybie tylko do zapisu,
- `a` — plik zostanie otwarty w trybie dopisywania.

Przykład 6.74

```
$p = fopen('dane.txt', 'r');
```

Plik zostanie otwarty tylko do odczytu.

Funkcja `fopen()` zwraca wartość `false`, jeżeli plik nie może zostać otwarty.

Po odpowiednim ustawieniu atrybutu *`tryb_otwarcia`* funkcja ta może zostać wykorzystana do zapisywania w pliku nowych danych lub dopisywania ich do niego.

Przykład 6.75

```
$p = fopen('dane.txt', 'w');
```

Po wykonaniu podanego kodu znajdujące się w pliku dane zostaną usunięte, a nowe dane zostaną zapisane na początku pliku. Jeżeli podany plik nie istnieje, zostanie utworzony.

Przykład 6.76

```
$p = fopen('dane.txt', 'a');
```

Po wykonaniu przedstawionego kodu nowe dane zostaną dopisane na końcu pliku.

Do zamykania pliku służy funkcja `fclose()` zapisywana w postaci:

```
fclose(deskryptor)
```

Deskryptor to wartość zwrócona przez funkcję `fopen()`. Gdy zakończą się operacje wykonywane na otwartym pliku, powinien on zostać zamknięty. Jeżeli nie zostanie to zrobione w skrypcie, plik zostanie zamknięty, gdy skrypt zakończy działanie.

Przykład 6.77

```
if ($p = fopen('dane.txt', 'w')) {
  // wykonanie działań na danych
  fclose($p);
}
```

Kolejną funkcją stosowaną do zapisywania pliku jest funkcja `fwrite()` w postaci:

```
fwrite(deskryptor, ciąg_znaków)
```

Pierwszy argument oznacza plik zwrócony za pomocą funkcji `fopen()`, drugi to ciąg znaków, które mają zostać zapisane. Funkcja zwraca wartość `false`, jeżeli zapisanie ciągu znaków w pliku się nie powiodło.

Przykład 6.78

```
<?php
$tekst = "Barbarzyńcy, gdy przyjdą, ustanowią prawa.\n";
if (!$p = fopen('dane.txt', 'a')) {
  echo "Nie można otworzyć pliku dane.txt";
} else {
  if (fwrite($p, $tekst) === false) {
    echo "Zapis do pliku nie powiódł się";
  } else {
    echo $tekst;
  }
  fclose($p);
}
?>
```

W podanym przykładzie tekst umieszczony w zmiennej `$tekst` będzie dopisywany na końcu pliku *dane.txt* przy każdym uruchomieniu skryptu. Jeżeli plik nie istnieje, zostanie utworzony przy pierwszym wywołaniu funkcji.

Odczyt danych

Dane z pliku mogą być odczytywane na wiele sposobów. W języku PHP istnieje duża liczba funkcji, które umożliwiają odczyt pojedynczych znaków, całych wierszy lub wybranych fragmentów pliku.

Po otwarciu pliku można odczytywać pojedyncze wiersze za pomocą funkcji `fgets()`. Ma ona postać:

```
fgets(deskryptor, ile_znaków)
```

Pierwszy argument oznacza plik otwarty za pomocą funkcji `fopen()`, drugi określa maksymalną liczbę znaków, które można odczytać. Funkcja zwraca odczytany ciąg znaków.

Funkcja `fgets()` często jest stosowana razem z funkcją `feof()`, która służy do sprawdzania, czy osiągnięty został koniec pliku. Jeśli tak, zwraca ona wartość `true`. Funkcja `feof()` ma postać:

```
feof(deskryptor)
```

Przykład 6.79

```
<?php
if (!$p = fopen('dane.txt', 'r')) {
  echo "Nie można otworzyć pliku dane.txt";
} else {
  while (!feof($p)) {
    $w = fgets($p,100);
    echo "$w<br>";
  }
  fclose($p);
}
?>
```

W podanym przykładzie za pomocą funkcji `fopen()` jest otwierany tylko do odczytu plik *dane.txt*. Jeśli otwarcie pliku się nie powiedzie (funkcja zwróci `false`), skrypt wyświetli komunikat i zakończy działanie. Gdy plik zostanie otwarty, w pętli będą odczytywane kolejne wiersze tekstu aż do osiągnięcia końca pliku. Przy każdym wykonaniu pętli sprawdzana jest wartość zwracana przez funkcję `feof()`. Gdy zostanie osiągnięty koniec pliku (funkcja `feof()` zwróci wartość `true`), pętla zakończy działanie. Wewnątrz pętli za pomocą funkcji `fgets()` pobierane są kolejne wiersze i są one przypisywane do zmiennej `$w`. Następnie zmienna ta jest wyświetlana z dodanym znacznikiem `<br>`.

Do odczytywania pojedynczych znaków służy funkcja `fgetc()`, zapisana w postaci:

```
fgetc(deskryptor)
```

Funkcja zwraca ciąg zawierający jeden znak. Po wykonaniu funkcji wskaźnik pliku jest przesuwany o jeden znak do przodu. Gdy zostanie osiągnięty koniec pliku, funkcja zwróci wartość `false`.

Przykład 6.80

```
<?php
if (!$p = fopen('dane.txt', 'r')) {
```

```
  echo "Nie można otworzyć pliku dane.txt";
} else {
  while (($z=fgetc($p)) !== false) {
    echo $z;
  }
  fclose($p);
}
?>
```

Skrypt przedstawiony w tym przykładzie jest bardzo podobny do skryptu z poprzedniego przykładu. Ponieważ funkcja `fgetc()` sama rozpoznaje koniec pliku, nie ma potrzeby stosowania funkcji `feof()`. Do sprawdzania, czy został osiągnięty koniec pliku, wykorzystany został operator `!==`.

Do odczytu bloków danych służy funkcja `fread()`, przyjmująca postać:

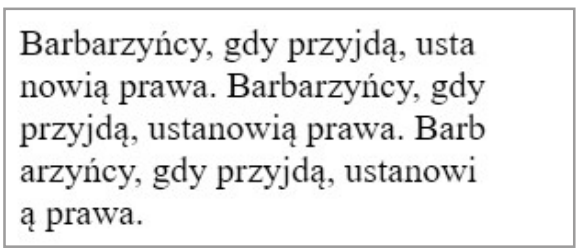
```
fread(deskryptor, ile_znaków)
```

Pierwszy argument funkcji to plik otwarty przez funkcję `fopen()`, drugi określa liczbę znaków, które należy odczytać. Odczytany fragment pliku jest zwracany przez funkcję w postaci ciągu znaków.

Przykład 6.81

```
<?php
if (!$p = fopen('dane.txt', 'r')) {
  echo "Nie można otworzyć pliku dane.txt";
} else {
  while (!feof($p)) {
    $b = fread($p, 32);
    echo "$b<br>";
  }
  fclose($p);
}
?>
```

W podanym przykładzie dane są odczytywane w blokach 32-bajtowych i wysyłane do przeglądarki (rysunek 6.24).



Rysunek 6.24.
Odczyt pliku dane.txt za pomocą funkcji fread()

Kolejna funkcja służąca do odczytu danych z pliku to `readfile()`, zapisywana w postaci:

```
readfile('nazwa_pliku')
```

Wysyła ona zawartość pliku podanego jako argument do przeglądarki. Zwraca jako wartość liczbę odczytanych bajtów lub wartość `false`, gdy wykonanie operacji się nie powiodło.

Podobnie działa funkcja `file_get_contents()`, zapisana w postaci:

```
file_get_contents('nazwa_pliku')
```

Zwraca ona odczytaną zawartość pliku podanego jako argument w postaci ciągu tekstowego lub wartość `false`, gdy wykonanie operacji się nie powiodło.

Funkcja `file()` służy do odczytywania zawartości plików tekstowych. Zapisywana jest w postaci:

```
file('nazwa_pliku')
```

Funkcja odczytuje całą zawartość pliku i zwraca tę zawartość w postaci tablicy. Każda komórka tablicy zawiera kolejny wiersz odczytanego tekstu. Argumentem jest nazwa odczytywanego pliku. Funkcja może mieć drugi opcjonalny argument, który może przyjmować jedną z wartości:

- `FILE_USE_INCLUDE_PATH` — po nazwie opcji powinny zostać wymienione katalogi, które zostaną przeszukane, jeżeli wskazanego pliku nie będzie w bieżącym katalogu.
- `FILE_IGNORE_NEW_LINES` — opcja ignoruje znaki końca linii.
- `FILE_SKIP_EMPTY_LINES` — opcja ignoruje puste linie.

6.7.3. Operacje na katalogach

Język PHP został wyposażony w zestaw funkcji, które umożliwiają wykonywanie różnych operacji na strukturze katalogów.

Za pomocą funkcji `mkdir()` można tworzyć nowe katalogi. Ma ona postać:

```
mkdir('nazwa_katalogu')
```

Argumentem funkcji jest nazwa tworzonego katalogu. Funkcja zwraca wartość `true`, jeżeli katalog został utworzony, lub wartość `false`, jeżeli katalogu nie udało się utworzyć.

Katalog można usunąć za pomocą funkcji `rmdir()`. Ma ona postać:

```
rmdir('nazwa_katalogu')
```

W wyniku wykonania funkcji zostanie usunięty katalog o nazwie podanej jako argument, pod warunkiem że jest on pusty. Funkcja zwraca wartość `true`, jeżeli katalog został usunięty, lub wartość `false`, jeżeli katalogu nie udało się usunąć.

Do otwierania katalogu służy funkcja `opendir()`, zapisana w postaci:

```
opendir('nazwa_katalogu')
```

Funkcja zwraca deskryptor katalogu lub wartość `false`, jeżeli katalogu o podanej nazwie nie uda się otworzyć.

Do odczytu zawartości katalogu stosowana jest funkcja `readdir()`, zapisana w postaci:

```
readdir(deskryptor)
```

Argumentem funkcji jest deskryptor uzyskany w wyniku wykonania funkcji `opendir()`. Każde wywołanie funkcji powoduje zwrócenie nazwy kolejnego elementu znajdującego się w katalogu. Po osiągnięciu końca katalogu funkcja zwróci wartość `false`.

Do zamykania katalogu służy funkcja `closedir()`, zapisana w postaci:

```
closedir(deskryptor)
```

Funkcja nie zwraca żadnej wartości. Jeżeli nie zostanie użyta, zamknięcie katalogu następuje po zakończeniu działania skryptu. Jeżeli funkcja zostanie wywołana bez podania argumentu, zostanie zamknięty ostatnio otwarty katalog.

Przykład 6.82

```
<?php
$katalog = "./";
if ($deskr = opendir($katalog)) {
  while (($plik = readdir($deskr)) !== false) {
    echo "$plik<br>";
  }
  closedir();
} else {
  echo "Nie można otworzyć katalogu";
}
?>
```

W podanym przykładzie ustawiona została zmienna `$katalog` zawierająca nazwę katalogu, którego zawartość należy odczytać (w przykładzie jest to katalog bieżący). Następnie sprawdzana jest możliwość otwarcia katalogu i jeżeli operacja przebiegła pomyślnie, za pomocą pętli `while` wyświetlane są nazwy wszystkich elementów umieszczonych we wskazanym katalogu.

Do odczytu zawartości katalogu może również zostać wykorzystana funkcja `scandir()`, która pobiera zawartość całego katalogu i zapisuje ją w tablicy. Dodatkowo sortuje odczytane nazwy plików i folderów. Funkcja ma postać:

```
scandir('nazwa_katalogu')
```

Przykład 6.83

```
<?php
$katalog = "./";
$tablica = scandir($katalog);
foreach($tablica as $plik) {
  echo "$plik<br>";
}
?>
```

Zarówno funkcja `readdir()`, jak i funkcja `scandir()`, odczytując zawartość wskazanego katalogu, nie rozróżnia w nim plików i podkatalogów.

Ćwiczenie 6.14

Napisz skrypt, który będzie odczytywał zawartość wskazanego katalogu, a odczytane elementy podzieli na katalogi i pliki — i wyświetli je w kolejności: wszystkie katalogi, następnie wszystkie pliki. W skrypcie wykorzystaj funkcje `is_file()` oraz `is_dir()`.

6.7.4. Praktyczne zastosowanie operacji na plikach

Omówione mechanizmy pracy z plikami zostaną wykorzystane do dodawania przez użytkowników na stronie internetowej komentarzy na wybrany temat.

Ćwiczenie 6.15

Utwórz skrypt, który będzie zapisywał opinie użytkowników w pliku tekstowym *opinie.txt*. Na stronie wyświetl formularz, który pozwoli na wpisanie opinii. Nowe opinie powinny być dopisywane do pliku i umieszczane na jego końcu. Dotychczasowe opinie zapisane w pliku tekstowym powinny zostać wyświetlone na stronie i powinny być dostępne dla innych jej użytkowników.

Rozwiązanie

skrypt1.php

```
<?php
if (isset($_POST['komentarz'])) {
  $tekst = substr($_POST['komentarz'], 0, 255);
  $tekst = strip_tags($tekst) . "\n";

  if (!$op = fopen('opinie.txt', 'a')) {
    echo "Błąd!. Nie można otworzyć pliku opinie.txt";
  } else {
```

```
    if (fwrite($op, $tekst) === false) {
      echo "Dodanie komentarza nie powiodło się";
    }
  }
}
?>

<html>
<head>
<title>Opinie użytkowników</title>
<meta charset="UTF-8">
</head>
<body>

<div>
<form action="skrypt1.php" method="post">
<p><b>Dodaj swój komentarz na temat globalnego ocieplenia</b><br>
(Maksymalnie 255 znaków)</p>

<textarea name="komentarz" rows="6" cols="50"
wrap="virtual"></textarea><br>
<input type="submit" value="Wyślij">
</div>
</form>

<b>Dodane opinie:</b>
<div>

<?php
$opinie = '';
if (file_exists('./opinie.txt')) {
  $opinie = file_get_contents('./opinie.txt');
  $opinie = nl2br($opinie);
}
```

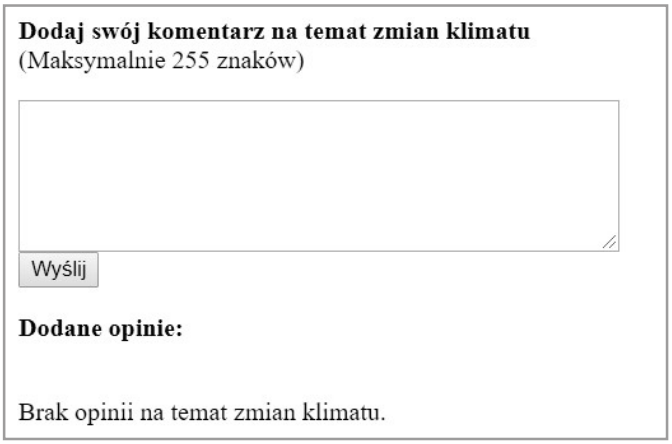
```
if ($opinie != '') {
  echo $opinie;
} else {
  echo "Brak opinii na temat zmian klimatu.";
}
?>
</div>
</body>
</html>
```

W pierwszej części kodu sprawdzane jest, czy za pomocą metody `POST` został przesłany z formularza parametr `komentarz` zawierający treść opinii użytkownika. Jeżeli tak, to wartość tego parametru jest odczytywana i zapisywana w zmiennej `$tekst`. Funkcja `substr()` skraca jego długość do 255 znaków. Następnie za pomocą funkcji `strip_tags()` ze zmiennej `$tekst` zostają usunięte znaczniki PHP i HTML (jeżeli się takie pojawią).

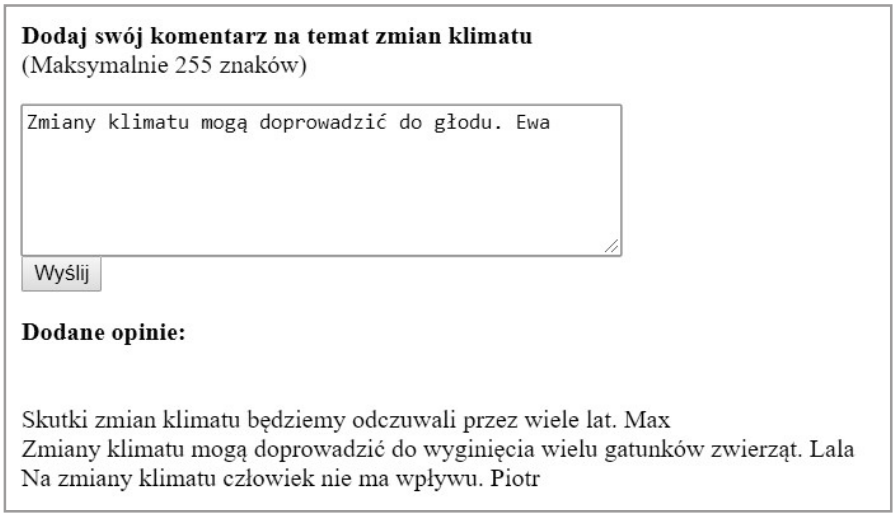
Kolejny blok instrukcji dołącza wartość zmiennej `$tekst` do pliku *opinie.txt*. Jeżeli pliku nie można otworzyć (`if (!$op = fopen())`) lub nie można dołączyć do niego nowej treści (`if (fwrite)`), zostaną wyświetlone odpowiednie komunikaty.

Następny blok to kod HTML tworzący formularz do wprowadzania tekstu za pomocą znacznika `<textarea>`. Dane z formularza będą przekazywane przy użyciu metody POST.

Ostatni blok to ponownie kod PHP, który spowoduje odczytanie wszystkich opinii umieszczonych w pliku *opinie.txt* i wyświetli je na stronie. Sprawdzane jest, czy plik istnieje, a jeżeli tak, to odczytywana treść jest zapisywana w zmiennej `$opinie`. Następnie za pomocą funkcji `nl2br()` występujące znaki końca linii są zamieniane na znaczniki `<br>`. Jeżeli w pliku nie było opinii, na stronie pojawia się komunikat o braku opinii (rysunek 6.25). Jeśli były, to ich treść wyświetla się na stronie (rysunek 6.26).



Rysunek 6.25. Brak opinii w pliku opinie.txt



Rysunek 6.26. Opinie zapisane w pliku opinie.txt

6.7.5. Funkcje wyjścia

Do natychmiastowego zakończenia wykonywania skryptu stosowane są instrukcje `exit()` i `die()`. Mogą występować w postaci:

```
exit([argument])
die([argument])
```

Instrukcje `exit()` i `die()` są elementami składni języka i mogą występować bez nawiasów, jeżeli nie mają argumentów. Argumentem może być komunikat, który wyświetli się przed wyjściem ze skryptu, lub liczba całkowita z przedziału od `0` do `254`. Jeżeli jest to liczba, określa ona stan wyjścia i nie jest wyświetlana. Stan równy `0` oznacza, że program w sposób prawidłowy zakończył działanie. Stan równy `255` jest zarezerwowany dla PHP i nie powinien być używany.

Przykład 6.84

```
<?php
if (!$p = fopen('dane.txt', 'r')) {
  exit("Nie można otworzyć pliku dane.txt");
}
else {
  // wykonanie działań na danych
}
?>
```

Podany kod sprawdza, czy plik *dane.txt* został otwarty. Jeżeli nie, wyświetla komunikat i kończy działanie.

Zadanie 6.1

Dodaj do swojej strony internetowej podstronę z informacjami na wybrany przez siebie temat zgodny z tematyką strony. Utwórz formularz, który pozwoli na wpisanie opinii dotyczącej zamieszczonych informacji. Opinie użytkowników powinny zostać zapisane w pliku tekstowym. Wszystkie opinie wpisane przez użytkowników powinny być widoczne na stronie.

Utwórz przycisk *Wyczyść komentarze*. Po kliknięciu tego przycisku wszystkie wpisane do tej pory komentarze powinny zostać usunięte.

Zadanie 6.2

Przygotuj plik *artykuł.txt* i umieść w nim informacje na wybrany temat. Przygotuj szablon strony internetowej. W menu z lewej strony umieść odnośnik do podstrony *Moje artykuły*. W górnej części tej podstrony wyświetl treść zapisaną w pliku *artykuł.txt* w liniach o długości 30 znaków.

Poniżej wyświetlonego tekstu utwórz pole tekstowe *Dodaj tekst*, w którym użytkownik będzie mógł wpisywać komentarze do wyświetlonej treści. Po wpisaniu komentarza i wciśnięciu przycisku *Wyślij* tekst powinien zostać przesłany na serwer i po dodaniu znaku przecinka dopisany do pliku *Nowe_teksty.txt*.

Utwórz pole tekstowe i wyświetl w nim zawartość pliku *Nowe_teksty.txt*. Wyświetlaną zawartość pliku sformatuj tak, aby każdy tekst zakończony przecinkiem był wyświetlany w oddzielnej linii.

6.8. Obsługa formularzy

Dynamiczna strona WWW to strona, która jest generowana na podstawie danych przekazanych między innymi przez użytkownika. Podstawowym sposobem przekazywania danych z przeglądarki internetowej do serwera jest korzystanie z formularzy HTML. Użytkownik za pomocą formularza wprowadza różne informacje, które są wysyłane do serwera. Dane te są przetwarzane na serwerze przy użyciu skryptów PHP.

6.8.1. Formularz HTML

Przykład 6.85

Prosty formularz HTML może mieć postać:

```
<!DOCTYPE HTML>
<html>
<head>
<title>Formularz rejestracyjny</title>
<meta charset="UTF-8">
<style>
```

```
p {
  font-weight: bold;
  font-size: 14pt;
}
#wyk {
  font-weight: bold;
  font-size: 11pt;
}
</style>
</head>
<body>
<form action="http://localhost/skrypt_form.php" method="post">
<p>Formularz kontaktowy:</p>
Nazwisko:<br>
<input type="text" name="nazw" value=" " size="30"><br>
Imię:<br>
<input type="text" name="im" value=" " size="30"><br>
Zawód:<br>
<input type="text" name="zaw" value=" " size="30"><br>
Adres e-mail:<br>
<input type="text" name="adr" value=" " size="30">

<p id="wyk">Wykształcenie:</p>
<input type="radio" value="podstawowe"
name="wykszt" checked> Podstawowe<br>
<input type="radio" value="średnie" name="wykszt"> Średnie<br>
<input type="radio" value="wyższe" name="wykszt"> Wyższe<br><br>

<input type="checkbox" name="opcje" maxlength="1">
Zgadzam się na przetwarzanie moich danych osobowych<br><br>
<input type="submit" value="Wyślij" name="wyslij">   
<input type="reset" value="Wyczyść" name="zeruj">
</form>
</body>
</html>
```

Podany w przykładzie formularz składa się z czterech pól do wprowadzania tekstu, pola wyboru typu `radio`, pola wyboru typu `checkbox` oraz przycisków typu `submit` (*Wyślij*) oraz `reset` (*Wyczyść*) (rysunek 6.27).



Rysunek 6.27. Przykładowy formularz

Po kliknięciu przycisku *Wyślij* dane z formularza powinny zostać przesłane na serwer za pomocą metody `POST`. Parametr `action` wskazuje adres serwera i nazwę skryptu PHP, który zostanie wywołany po zatwierdzeniu formularza. W tym przykładzie zostanie uruchomiony skrypt o nazwie *skrypt_form.php*, który znajduje się na serwerze lokalnym o adresie *http://localhost*. Dane wprowadzone do formularza zostaną przekazane do wybranego skryptu. Parametr `method` określa sposób (metodę) przesłania informacji do wskazanego skryptu. Może przyjąć jedną z dwóch wartości: `post` lub `get`. Wartością domyślną jest `get`, ale zaleca się stosowanie metody `post`.

6.8.2. Przekazywanie danych — metoda get i post

Każdy formularz zaczyna się od znacznika otwierającego `<form>` i kończy się znacznikiem zamykającym `</form>`. Pomiędzy tymi znacznikami znajdują się wszystkie polecenia dotyczące formularza. Jeżeli dane z formularza będą przesyłane na serwer, znacznik ten powinien mieć następujące atrybuty:

`action="adres"`. To atrybut, który ustala adres do wysyłki danych z formularza. Dane wprowadzone do formularza zostaną przekazane do podanego skryptu.

`method="metoda"` określa sposób (metodę) przekazania informacji do wskazanego skryptu lub na wskazany adres. Atrybut może przyjąć jedną z dwóch wartości: `post` lub `get`.

Metoda GET jest używana, kiedy parametrów jest niewiele. W tej metodzie parametry są przekazywane za pomocą adresu URL. Schematyczna postać URL wygląda następująco:

http://www.nazwa_serwera.pl/strona.php?parametr1=wartość1&parametr2=wartość2

Długość adresu URL jest ograniczona, a przekazywane parametry są widoczne w pasku adresu przeglądarki, dlatego metoda ta jest stosowana do przesyłania niedużych ilości danych. Adres skryptu PHP od przesyłanych parametrów oddzielany jest znakiem zapytania. Przesyłane pary *parametr*=*wartość* są oddzielane znakiem ampersand (&).

Metoda POST do przekazywania parametrów wykorzystuje nagłówek strony. Metoda ta umożliwia przekazywanie większej liczby parametrów, a parametry nie są widoczne w pasku adresu przeglądarki.

Przesłane wartości parametrów są zapisywane w odpowiednich tablicach asocjacyjnych. Dane przesłane metodą GET są zapisywane w tablicy $_GET, natomiast dane przesłane metodą POST w tablicy $_POST. Tablice $_GET i $_POST są superglobalne. Oznacza to, że są widoczne w każdym miejscu kodu PHP bez konieczności użycia składni globals.

Indeksami tablicy $_GET i $_POST są nazwy pól formularza (wartość atrybutu name w formularzu HTML), do których się odwołujemy. Odwołanie do elementów tablicy ma postać:

```
$zmienna = $_GET['nazwa_pola'];
```

lub

```
$zmienna = $_POST['nazwa_pola'];
```

Przykład 6.86

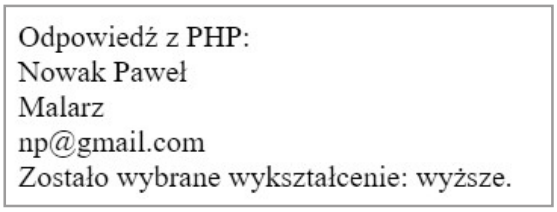
W celu sprawdzenia poprawności działania skryptu z przykładu 6.85 zostanie utworzony skrypt PHP (*skrypt_form.php*) przetwarzający dane przesłane z formularza.

```
<?php
echo "Odpowiedź z PHP: <br>".$_POST['nazw'];
echo $_POST['im']."<br>";
echo $_POST['zaw']."<br>";
echo $_POST['adr']."<br>";
echo "Zostało wybrane wykształcenie: {$_POST['wykszt']}.";
?>
```

Jeżeli podany kod zostanie zapisany w pliku *skrypt_form.php*, to po otwarciu formularza, wpisaniu danych (rysunek 6.28) i jego wysłaniu przeglądarka powinna wyświetlić przesłane dane (rysunek 6.29).



Rysunek 6.28. Formularz z wypełnionymi polami



Rysunek 6.29. Dane przesłane za pomocą formularza

Funkcja isset

Podczas przetwarzania formularza można wykorzystać funkcję `isset()`. Pozwala ona stwierdzić, czy określone pole formularza zostało wypełnione, a dokładniej, czy do skryptu została przekazana wartość danego pola. Funkcja zwraca `true`, gdy wartość została przekazana, w przeciwnym razie zwraca `false`.

Przykład 6.87

W przykładzie 6.85 formularz oprócz pól tekstowych zawierał również pola typu `radio`. Są to trzy pola wyboru, w których można zaznaczyć tylko jedną opcję z grupy opcji. Kliknięcie wartości powoduje zaznaczenie opcji, ale równocześnie usunięte zostaje zaznaczenie innej opcji. Każde z pól tego typu ma taką samą wartość atrybutu `name`. Atrybut `value` w tym przypadku jest konieczny i musi być inny dla każdego pola. Przez wartość atrybutu `value` pola będą identyfikowane w skrypcie.

Utworzymy skrypt PHP, który będzie rozpoznawał, jaka opcja pola typu `radio` została zaznaczona w formularzu (kod może zostać dołączony do pliku *skrypt_form.php*).

```
<?php
if (!isset($_POST['wykszt'])) {
  echo "<br>Proszę zaznaczyć pole <b>Wykształcenie</b>";
} else {
  echo "<br>Zostało wybrane wykształcenie: {$_POST['wykszt']}.";
}
?>
```

W podanym skrypcie do sprawdzenia, czy zaznaczone zostało pole opcji, zastosowano instrukcję warunkową `if` oraz funkcję `isset()`. W warunku `!isset($_POST['wykszt'])` został użyty operator logicznej negacji `!`, czyli gdy pole nie zostanie wypełnione, warunek zwróci wartość `true` i zostanie wysłany komunikat *Proszę zaznaczyć pole Wykształcenie*. W przeciwnym razie za pomocą składni `{$_POST['wykszt']}` zostanie odczytana z tablicy `$_POST` wartość opcji `wykszt` (będzie to wartość atrybutu `value` dla znacznika `<input type="radio">`).

Funkcja empty

Funkcja `empty()` sprawdza, czy istniejąca zmienna użyta jako argument jest pusta. Jeżeli zmienna jest pusta, to funkcja zwraca wartość `true`, w przeciwnym razie zwraca wartość `false`.

W formularzu może wystąpić pole typu `SELECT`, które służy do wyświetlania listy wartości i pozwala na wybranie jednej z nich lub kilku.

Przykład 6.88

```
<!DOCTYPE HTML>
<html>
<head>
<title>Formularz językowy</title>
<meta charset="UTF-8">
<style>
#wyb {
  font-weight: bold;
  font-size: 12pt;
}
</style>
</head>
<body>
<form action="skrypt_form1.php" method="post">
```

```
Nazwisko i imię:<br>
<input type="text" name="nazw" value=" " size="40"><br>

<p id="wyb">Wybór języka:</p>
<select name="języki[]" multiple>
<option value="Język angielski">Język angielski</option>
<option value="Język niemiecki">Język niemiecki</option>
<option value="Język francuski">Język francuski</option>
<option value="Język włoski">Język włoski</option>
<option value="Język rosyjski">Język rosyjski</option>
<option value="Język hiszpański">Język hiszpański</option>
</select>
<p><input type="submit" value="Wyślij" name="wyslij"></p>
</form>
</body>
</html>
```

Gdyby utworzona w formularzu lista wyboru wartości została zdefiniowana w postaci `<select name="języki" multiple>`, to skrypt obsługujący formularz miałby dostęp tylko do jednej wartości (`$_POST["języki"]`). Gdy kwadratowe nawiasy zostaną dodane w podany sposób: `<select name="języki[]">`, to elementy wybrane przez użytkownika w polu `języki[]` będą dostępne w tablicy `$_POST["języki"]`. Utworzony formularz został pokazany na rysunku 6.30.



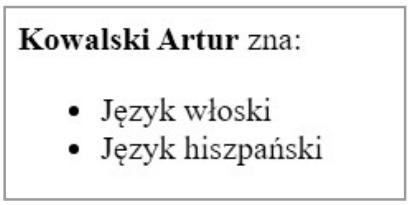
Rysunek 6.30.
Sposób wypełniania formularza

Należy jeszcze dołączyć skrypt PHP (zapisany w pliku *skrypt_form1.php*) w postaci:

```
<?php
if (!empty($_POST['języki'])) {
  echo "<p><b>".$_POST['nazw']."</b> zna:</p>";
  echo "<ul>";
  foreach ($_POST['języki'] as $wartosc) {
    echo "<li>$wartosc</li>";
  }
  echo "</ul>";
```

```
} else {
  echo "<p><b>".$_POST['nazw']."</b> nie zna żadnego języka.</p>";
}
?>
```

Instrukcja `if (!empty($_POST['języki']))` sprawdza, czy tablica `$_POST['języki']` jest pusta. Jeżeli nie, to nastąpi wyświetlenie zawartości każdego elementu tablicy (rysunek 6.31). W tablicy znajdują się wartości znacznika `<option value=...>` wybrane przez użytkownika.



Rysunek 6.31. Po wysłaniu formularza przeglądarka wyświetli przesłane dane

Ta technika może być stosowana do różnych elementów, nie tylko do pól `SELECT`. Jeżeli kilku polom nadamy taką samą nazwę i nazwa pola będzie kończyła się nawiasem kwadratowym, to PHP połączy wszystkie wartości o tej samej nazwie pola w jedną tablicę.

Przykład 6.89

Po wprowadzonych zmianach kod zawierający formularz HTML będzie wyglądał tak:

```
<!DOCTYPE HTML>
<html>
<head>
<title>Formularz rejestracyjny</title>
<meta charset="UTF-8">
<style>
#fo {
  font-weight: bold;
  font-size: 14pt;
}
p.wyk {
  font-weight: bold;
  font-size: 11pt;
}
</style>
</head>
<body>
<form action="skrypt_form2.php" method="post">
<p id="fo">Formularz kontaktowy:</p>
```

```
Nazwisko:<br>
<input type="text" name="nazw" value=" " size="30"><br>
Imię:<br>
<input type="text" name="im" value=" " size="30"><br>
Zawód:<br>
<input type="text" name="zaw" value=" " size="30"><br>
Adres e-mail:<br>
<input type="text" name="adr" value=" " size="30">

<p class="wyk">Wykształcenie:</p>
<input type="radio" value="podstawowe"
name="wykszt" checked> Podstawowe<br>
<input type="radio" value="średnie" name="wykszt"> Średnie<br>
<input type="radio" value="wyższe" name="wykszt"> Wyższe<br><br>

<p class="wyk">Wybór języka:</p>
<select name="języki[]" multiple>
<option value="Język angielski">Język angielski</option>
<option value="Język niemiecki">Język niemiecki</option>
<option value="Język francuski">Język francuski</option>
<option value="Język włoski">Język włoski</option>
<option value="Język rosyjski">Język rosyjski</option>
<option value="Język hiszpański">Język hiszpański</option>
</select>

<p><input type="checkbox" name="opcje" maxlength="1">
Zgadzam się na przetwarzanie moich danych osobowych</p>
<input type="submit" value="Wyślij" name="wyslij">   
<input type="reset" value="Wyczyść" name="zeruj">
</form>
</body>
</html>
```

A kod zawierający skrypt PHP (plik *skrypt_form2.php*) będzie wyglądał tak:

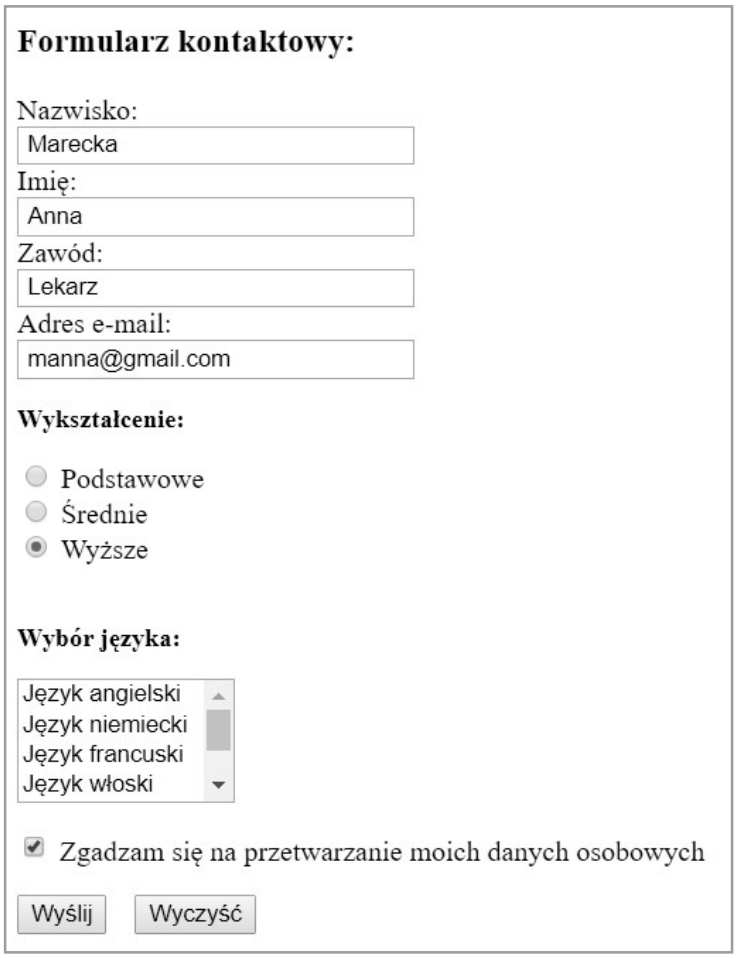
```
<?php
echo "Dziękujemy! Rejestracja przebiegła pomyślnie.<br>";
echo "Wprowadzone dane:<br>";
echo "Nazwisko: " . trim($_POST['nazw']) . "<br>";
echo "Imię: " . trim($_POST['im']) . "<br>";
echo "Zawód: " . trim($_POST['zaw']) . "<br>";
echo "Adres e-mail: " . trim($_POST['adr']) . "<br>";

if (!isset($_POST['wykszt'])) {
  echo "Proszę zaznaczyć pole Wykształcenie";
} else {
  echo "Wykształcenie: " . trim($_POST['wykszt']) . "<br>";
}

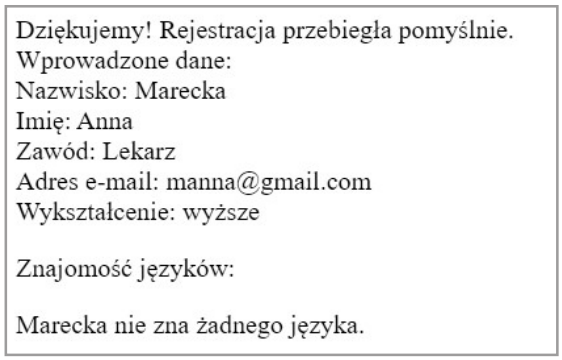
echo "<p>Znajomość języków:</p>";

if (!empty($_POST['języki'])) {
  echo "<ul>";
  foreach ($_POST['języki'] as $wartosc) {
    echo "<li>$wartosc</li>";
  }
  echo "</ul>";
} else {
  echo "<p>".$_POST['nazw']." nie zna żadnego języka.</p>";
}
?>
```

Po wprowadzeniu danych (rysunek 6.32) formularz zostanie wysłany, a w przeglądarce wyświetli się informacja o pomyślnym zarejestrowaniu użytkownika (rysunek 6.33).



Rysunek 6.32. Formularz z wprowadzonymi danymi



Rysunek 6.33. Wynik pobrania danych z formularza

Ćwiczenie 6.16

Opracuj formularz przeznaczony do rejestrowania uczniów zapisujących się do szkoły. Określ informacje, które powinny znaleźć się w formularzu, oraz sposób ich wprowadzania. Utwórz skrypt przetwarzający dane przesłane z formularza. W skrypcie powinno być sprawdzane wypełnienie wszystkich pól.

6.9. Pliki cookies i sesje

6.9.1. Pliki cookies w PHP

Pliki cookies

Pliki cookies to niewielkie pliki tekstowe wysyłane przez serwer lub skrypt do przeglądarki i umieszczane przez nią na dysku użytkownika. Pliki te są częścią specyfikacji protokołu HTTP i są wysyłane do przeglądarki w postaci nagłówka o nazwie `Set-Cookie`. Służą głównie do identyfikacji użytkownika. Plik cookie zawiera między innymi nazwę serwera, datę wygaśnięcia pliku oraz informacje na temat domeny i ścieżki dostępu do pliku. Rozmiar pliku nie może przekroczyć 4 kB.

Zasada działania plików cookies jest następująca:

- Po nawiązaniu połączenia serwer wysyła do przeglądarki nagłówek `Set-Cookie`, który zawiera plik cookie.
- Przeglądarka zapisuje plik cookie na dysku użytkownika.
- Przy kolejnym połączeniu z serwerem przeglądarka wysyła na serwer przechowywany na dysku plik cookie.

Pliki cookies są przechowywane na dysku do czasu wygaśnięcia. Jeżeli nie została podana data wygaśnięcia, to plik straci ważność po zamknięciu przeglądarki. Jeżeli zostały określone serwer i ścieżka do wysłania informacji zawartych w cookie, to po otwarciu adresu pasującego do podanej domeny i ścieżki zawartość pliku cookie zostanie wysłana na serwer.

Tworzenie pliku cookie

W skryptach PHP pliki cookies tworzone są za pomocą funkcji `setcookie()`, która ma postać:

```
setcookie(nazwa[, wartość[, czas_trwania[, ścieżka_dostępu[, domena[,
bezpieczeństwo[, tylko_http]]]]]])
```

Argumenty funkcji to:

- *nazwa* — nazwa pliku cookie.
- *wartość* — wartość, która będzie przechowana w pliku cookie.
- *czas_trwania* — czas, po jakim plik cookie zostanie usunięty przez przeglądarkę. Jest on podawany jako liczba sekund, które upłyną od 1 stycznia 1970 roku.
- *ścieżka_dostępu* — ścieżka dostępu na serwerze dla pliku cookie. Ustawienie tej wartości na `/` spowoduje, że plik cookie będzie dostępny w całej domenie. Ustawienie `/www/skrypty` sprawi, że plik będzie dostępny w podanym katalogu oraz w katalogach podrzędnych.
- *domena* — domena, w której plik cookie będzie dostępny.

- *bezpieczeństwo* — może przyjmować wartość `true` lub `false`. Ustawienie `true` spowoduje, że plik cookie będzie mógł być przesyłany tylko przez bezpieczne połączenia HTTPS. Domyślnym ustawieniem jest `false`.
- *tylko_http* — może przyjmować wartość `true` lub `false`. Ustawienie `true` spowoduje, że plik cookie będzie dostępny tylko przez protokół HTTP i nie będzie dostępny z poziomu skryptów osadzonych na stronie.

Jedynym argumentem, który musi wystąpić w funkcji, jest *nazwa*, pozostałe są opcjonalne.

Ponieważ pliki cookies są częścią nagłówka HTTP, funkcja `setcookie()` musi być wywołana, zanim jakiekolwiek dane zostaną wysłane do przeglądarki; powinna zatem zostać umieszczona w kodzie przed znacznikiem `<html>`.

Każdy wysłany plik cookie jest automatycznie umieszczany w superglobalnej tablicy `$_COOKIE`. Jej indeksami są nazwy plików cookies. Dostęp do pliku cookie jest możliwy po wpisaniu:

```
$_COOKIE['nazwa_pliku']
```

Przykład 6.90

```
<?php
setcookie("pismo", "Na skróty", time()+3600, "/", "localhost", false);
if (isset($_COOKIE["pismo"])) {
  echo "<p>Jesteś naszym stałym klientem.</p>";
} else {
  echo "<p>Witamy po raz pierwszy na naszej stronie.</p>";
}
?>
```

Utworzony w przykładzie plik cookie zostanie zapisany przez przeglądarkę pod nazwą `pismo`. Zawartością pliku będzie tekst `Na skróty`. Do określenia czasu trwania pliku została wykorzystana funkcja `time()`, która zwraca bieżący czas. Do czasu zwróconego przez tę funkcję został dodany wyrażony w sekundach czas ważności pliku (3600 s = 1 godz.). Ustawienie wartości / jako ścieżki dostępu oznacza, że plik będzie dostępny dla wszystkich stron na serwerze. Domena `localhost` wskazuje, że plik będzie dostępny na lokalnym serwerze. Ostatni argument ustawiony na `false` oznacza, że plik cookie może być przesyłany przez nieszyfrowane połączenia.

Usuwanie pliku cookie

Jeżeli dla pliku cookie został ustalony czas ważności, to zostanie on automatycznie usunięty przez przeglądarkę po określonym czasie. Jeśli czas ważności nie był ustalony, to plik zostanie usunięty przy zamykaniu przeglądarki. Aby usunąć samodzielnie plik cookie, można ustawić dla niego czas ważności, który już upłynął, albo ustawić jako

wartość ciąg pusty lub `false`. Pozostałe argumenty funkcji powinny być takie same jak argumenty przy tworzeniu pliku.

Przykład 6.91

```
setcookie("pismo", "Na skróty", time()-100, "/", "localhost", 0);
```

6.9.2. Zastosowania plików cookies

Jednym z zastosowań plików cookies może być przechowywanie na komputerze informacji o użytkowniku. Dzięki temu przy ponownej wizycie na stronie nie trzeba się powtórnie logować i podawać hasła. Pliki cookies mogą przechowywać również inne informacje, na przykład nazwisko i imię użytkownika, czas ostatniej wizyty, zawartość koszyka.

Data ostatnich odwiedzin na stronie

Przykład 6.92

Wykorzystując plik cookie, określimy datę ostatnich odwiedzin strony przez użytkownika.

Skrypt tworzący plik cookie:

```
<?php
$mies = 2592000 + time();
setcookie("wizyta", date("F jS - g:ia"), $mies);
?>
```

Zostanie utworzony plik cookie `wizyta`. Termin wygaśnięcia to 30 dni od chwili jego utworzenia (2 592 000 = 60 s × 60 min × 24 godz. × 30 dni).

Plik tworzący skrypt odczytujący zawartość pliku cookie:

```
<?php
if (isset($_COOKIE['wizyta'])) {
  $ostatnia = $_COOKIE['wizyta'];
  echo "Witamy ponownie! <br> Ostatni raz odwiedziłeś nas: " . $ostatnia;
} else {
  echo "Witamy na naszej stronie!";
}
?>
```

W podanym przykładzie sprawdzamy, czy istnieje plik cookie. Jeżeli tak, to wyświetli się informacja powitalna oraz data ostatniej wizyty.

Ćwiczenie 6.17

Utwórz skrypt, który będzie wyświetlał informacje o ostatnich odwiedzinach użytkownika na stronie przekazane do przeglądarki przez serwer.

Rozwiązanie

```
<!DOCTYPE html>
<?php
$czas = date("Y-m-d G:i:s");
setcookie("wizyta", $czas, time()+3600);
?>
<html>
<body>
<?php
if (isset($_COOKIE['wizyta'])) {
  $ostatnia = $_COOKIE['wizyta'];
  echo "Witamy ponownie! <br> Ostatni raz odwiedziłeś nas: " . $ostatnia;
} else {
  echo "Witamy na naszej stronie!";
}
?>
<p><strong>Uwaga:</strong> Potrzebne może być odświeżenie strony.</p>
</body>
</html>
```

Przesyłanie danych użytkownika

Ćwiczenie 6.18

Utwórz skrypt, który dane użytkownika przesłane za pomocą formularza będzie przekazywał do utworzonego pliku cookie. Formularz powinien zawierać imię i nazwisko użytkownika.

Rozwiązanie

Skrypt *rejestracja.php* realizujący podane zadanie:

```
<?php
if (!isset($_COOKIE['dane']) && !isset($_POST['nazw'])) {
?>
<!DOCTYPE HTML>
```

```
<html>
<head>
<title>Dane_user</title>
<meta charset="UTF-8">
</head>
<body>
<form action="rejestracja.php" method="post">
Podaj nazwisko i imię:<br>
<input type="text" name="nazw" value=" " size="35">
<p><input type="submit" value="Wyślij" name="wyslij"></p>
</form>
<?php
} else {
  if (isset($_POST['nazw'])) {
    setcookie('dane', $_POST['nazw'], time()+60*60*24*365);
    echo "<p>Dziękujemy za wprowadzenie danych.</p>";
  } else {
    echo "<p>Witamy po raz kolejny! " . $_COOKIE['dane'] . "</p>";
  }
}
?>
</body>
</html>
```

W podanym skrypcie obsługiwane są trzy sytuacje. W pierwszej plik cookie nie został utworzony i z formularza nie zostały przesłane dane — wtedy należy wywołać formularz rejestracyjny. W drugiej dane z formularza zostały przesłane, ale nie istnieje plik cookie — wówczas należy utworzyć taki plik. W trzeciej dane z formularza zostały przesłane oraz istnieje plik cookie — wtedy wyświetli się komunikat witający użytkownika po raz kolejny na stronie.

Zapisana w kodzie instrukcja `if (!isset ($_COOKIE ['dane']) && !isset($_POST['nazw']))` sprawdza, czy istnieje plik cookie i czy z formularza został przekazany parametr zawierający nazwisko i imię użytkownika. Jeżeli nie, to zostanie utworzona strona zawierająca formularz. Z formularza za pomocą metody `POST` zostanie przekazany parametr `nazw`. Użyty operator `!` oznacza negację, a operator `&&` iloczyn logiczny.

Druga instrukcja `if (isset($_POST['nazw']))` sprawdza, czy do skryptu zostały przesłane dane z formularza. Jeżeli tak, to za pomocą funkcji `setcookie()` do

przeglądarki jest wysyłany plik cookie o nazwie `dane`. Jego wartością jest wartość parametru `nazw` pobrana z tablicy `$_POST`. Czas ważności pliku został ustawiony na `365 dni`. Na stronie wyświetli się tekst z podziękowaniem za wprowadzenie danych.

Trzecia sytuacja wystąpi, jeśli istnieje plik cookie. Wtedy na stronie wyświetli się informacja o rozpoznaniu użytkownika.

Zliczanie liczby odwiedzin

Ćwiczenie 6.19

Utwórz skrypt, który będzie zliczał wizyty użytkownika na stronie.

Rozwiązanie

Skrypt *odwiedziny.php* realizujący podane zadanie:

```
<?php
if (!isset($_COOKIE ['odwiedz'])) {
  $odw = 1;
} else {
  $odw = intval($_COOKIE['odwiedz']) + 1;
}
setcookie("odwiedz", $odw, time()+60*60*24*365);
?>
<!DOCTYPE HTML>
<html>
<head>
<title>Odwiedziny</title>
<meta charset="UTF-8">
</head>
<body>
<?php
if ($odw == 1) {
  $wyraz = "raz";
} else {
  $wyraz = "razy";
}
echo "W ciągu ostatniego roku odwiedziłeś naszą stronę $odw $wyraz.<br>";
?>
</body>
</html>
```

W podanym kodzie za pomocą funkcji `isset()` sprawdzane jest, czy istnieje plik cookie o nazwie `odwiedz`. Jeżeli nie, to zmienna `$odw` przechowująca liczbę odwiedzin ustawiana jest na `1`. Jeżeli tak, to z tablicy `$_COOKIE` odczytywana jest wartość pliku `odwiedz`. Odczytana wartość za pomocą funkcji `intval()` jest zamieniana na liczbę całkowitą i zwiększana o `1`. Funkcja `intval()` jest niezbędna, ponieważ wartości odczytywane z tablicy `$_COOKIE` są ciągami znaków.

Po wykonaniu tych czynności do przeglądarki wysyłany jest plik cookie o nazwie `odwiedz`. Jego wartością jest liczba odwiedzin zapisana w zmiennej `$odw`. Na stronie wyświetla się informacja o liczbie odwiedzin (rysunek 6.34). Dodatkowo sprawdzana jest wartość zmiennej `$odw`. Jeżeli wynosi ona `1`, to wyświetlane zdanie będzie kończyło się słowem `raz`, dla pozostałych wartości będzie kończyło się słowem `razy`.



Rysunek 6.34. Odczyt pliku cookie z przechowywaną liczbą odwiedzin

Ćwiczenie 6.20

Napisz skrypt, w którym będzie tworzony plik cookie zapamiętujący wprowadzoną przez formularz datę urodzin użytkownika. Skrypt powinien wyświetlać informację, za ile dni użytkownik będzie obchodził urodziny.

6.9.3. Sesje

Mechanizm sesji dostępny w języku PHP pozwala na identyfikację użytkownika i śledzenie jego poczynań na stronie internetowej. Sesja to czas, w którym użytkownik przegląda stronę internetową. W momencie pierwszego wejścia na stronę tworzony jest dla użytkownika specjalny identyfikator (losowa, unikatowa liczba wygenerowana przez PHP). Identyfikator jest przechowywany na komputerze użytkownika i na serwerze. Na komputerze użytkownika jest najczęściej zapisywany w pliku cookie, a jeżeli nie jest możliwe utworzenie takiego pliku, identyfikator jest dodawany automatycznie do adresu URL. Jeżeli identyfikator jest przekazywany jako element adresu, to jego nazwa i wartość są zapisywane w stałej `SID`.

Wszystkie zmienne związane z użytkownikiem są przechowywane na serwerze w tablicy superglobalnej `$_SESSION`.

Do rozpoczęcia sesji najczęściej jest wykorzystywana funkcja `session_start()`. Funkcja zwykle nie przyjmuje żadnych argumentów, a jej wywołanie ma postać:

```
session_start();
```

W każdym skrypcie, który będzie korzystał z sesji, należy wywołać tę funkcję.

Gdy do zapisania identyfikatora wykorzystywany jest plik cookie, funkcja musi zostać wywołana, zanim do przeglądarki zostaną wysłane dane. Jej wywołanie powinno znaleźć się na początku pliku.

Do zakończenia sesji należy wykorzystać funkcję `session_destroy()`. Usuwa ona zasoby związane z sesją, nie usuwa natomiast zmiennych zapisanych na serwerze oraz pliku cookie na komputerze użytkownika. Nie przyjmuje argumentów, a jej wywołanie ma postać:

```
session_destroy();
```

Identyfikator sesji można odczytać za pomocą funkcji `session_id()`. Wywołując tę funkcję, można zarówno odczytać wartość identyfikatora, jak i zmienić ją.

Przykład 6.93

```
<?php
session_start();
echo "<p>Identyfikator sesji to: " . session_id() . "</p>";
?>
```

Podany kod wyświetli informację na temat wartości identyfikatora. Przy pierwszych odwiedzinach strony PHP wygeneruje dla użytkownika nowy identyfikator. Przy kolejnych odwiedzinach przydzieli mu ten sam identyfikator, a sesja zostanie wznowiona.

Zmienne sesji

Po rozpoczęciu sesji w tablicy `$_SESSION` mogą zostać zapisane dowolne zmienne związane z sesją. Aby zapisać zmienną, należy użyć konstrukcji:

```
$_SESSION['nazwa_zmiennej'] = wartość;
```

Do odczytania wartości zmiennej trzeba użyć indeksu odnoszącego się do nazwy zmiennej w tablicy `$_SESSION`:

```
$zmienna = $_SESSION['nazwa_zmiennej'];
```

Do sprawdzenia, czy określona zmienna istnieje, można zastosować funkcję `isset()` w postaci:

```
isset($_SESSION['nazwa_zmiennej'])
```

Funkcja `isset()` zwraca wartość `true`, jeśli zmienna została zarejestrowana, w przeciwnym razie zwraca wartość `false`. Zmienne zarejestrowane w czasie sesji powinny zostać wyrejestrowane przed jej zakończeniem. Służy do tego funkcja `unset()` w postaci:

```
session_unset()
```

która usuwa wszystkie zmienne sesji, lub

```
unset($_SESSION['nazwa_zmiennej'])
```

która usuwa podaną zmienną sesji.

Zmienne utworzone w sesji są przechowywane przez PHP. Zapisane dane mogą zostać odczytane przez inne skrypty w obrębie tej samej sesji.

Przykład 6.94

```
<?php
session_start();
if (!isset($_SESSION['ile'])) {
  $_SESSION['ile'] = 0;
}
$_SESSION['ile']++;
echo "<p>Odwiedziłeś " . $_SESSION['ile'] . " razy nasze strony.</p>";
?>
```

Skrypt pokazuje, jak za pomocą zmiennej sesji `ile` można obliczać liczbę odwiedzin na stronie. Po otwarciu sesji sprawdzana jest zawartość tablicy `$_SESSION`. Jeżeli zmienna sesji `ile` nie jest zarejestrowana, zostanie jej przypisana początkowa wartość `0`. Jeżeli jest zarejestrowana, jej wartość zwiększy się o `1`. Następnie wyświetli się komunikat o liczbie odwiedzin.

6.9.4. Praktyczne zastosowania sesji

Uwierzytelnienie użytkownika

Ćwiczenie 6.21

Przyjmij, że do strony internetowej mają dostęp tylko zalogowani użytkownicy. Wykorzystaj mechanizm sesji do przeprowadzenia autoryzacji użytkownika.

Rozwiązanie

Przygotuj trzy pliki. Plik *strona.php* będzie wyświetlał zawartość strony głównej witryny. Plik *loguj.php* będzie zawierał procedury autoryzacji użytkownika i formularz logowania, plik *wyloguj.php* — procedury wylogowania. W tworzonym rozwiązaniu będzie sprawdzane uwierzytelnienie tylko jednego użytkownika o nazwie `janek` i haśle `jan23`.

Pierwszym krokiem będzie weryfikacja danych użytkownika. Po ich sprawdzeniu użytkownik będzie mógł zalogować się i korzystać ze strony. Przed opuszczeniem strony użytkownik będzie musiał się wylogować.

Skrypt *loguj.php*:

```
<?php
session_start();
if (isset($_SESSION['log'])) {
  header('location: strona.php');
  exit();
} elseif (isset($_POST['nazwa']) && isset($_POST['haslo'])) {
```

```
  if ($_POST['nazwa'] == 'janek' && $_POST['haslo'] == 'jan23') {
    $_SESSION['log'] = $_POST['nazwa'];
    header('location: strona.php');
    exit();
  } else {
    echo "Nieprawidłowe dane logowania<br>";
  }
}
?>
<!DOCTYPE HTML>
<html>
<head>
<title>Autoryzacja danych</title>
<meta charset="UTF-8">
<style>
p.fo {
  font-weight: bold;
  font-size: 11pt;
}
#log {
  font-weight: bold;
  font-size: 14pt;
}
</style>
</head>
<body>
<div>
<form action="http://localhost/loguj.php" method="post">
<p id="log">Logowanie</p>
<p class="fo">Nazwa użytkownika:</p>
<input type="text" name="nazwa" value="" size="25"><br>
<p class="fo">Hasło:</p>
<input type="password" name="haslo" value="" size="25">
<input type="submit" value="Zaloguj się">
</form>
</div>
</body>
</html>
```

Skrypt autoryzacji użytkownika zaczyna się od wywołania funkcji `session_start()`, która rozpoczyna sesję. Następnie sprawdzane jest, czy istnieje zmienna sesji `$log`. Jest to zmienna, która powinna zawierać nazwę zalogowanego użytkownika. Jeżeli zmienna istnieje, to znaczy, że stronę próbuje otworzyć zalogowany użytkownik. Zostanie on przekierowany do strony głównej.



Rysunek 6.35.
Próba logowania do witryny

Jeżeli zmienna `$log` nie została ustawiona, ale w tablicy `$_POST` znajdują się dane (nazwa użytkownika i hasło), należy zweryfikować poprawność wprowadzonych danych. Następuje sprawdzenie, czy dane wprowadzone w czasie logowania są prawidłowe. Jeżeli weryfikacja przebiegła pomyślnie, oznacza to, że niezalogowany użytkownik wypełnił poprawnie formularz logowania i należy go zalogować (rysunek 6.35). Zmiennej sesji `$log` zostaje przypisana nazwa zalogowanego użytkownika i użytkownik zostaje przekierowany do strony głównej.



Rysunek 6.36.
Niepoprawne dane logowania

Jeżeli weryfikacja danych jest negatywna, wyświetla się komunikat informujący o nieprawidłowych danych logowania, a następnie wyświetla się formularz logowania (rysunek 6.36).

Jeżeli w tablicy `$_POST` nie zostały ustawione nazwa użytkownika i hasło, to znaczy, że na stronę próbuje dostać się niezalogowany użytkownik. Wyświetli się formularz logowania.

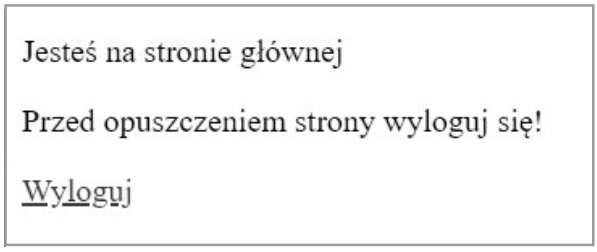
Kod strony głównej witryny został zapisany w skrypcie *strona.php*.

Skrypt *strona.php*:

```
<?php
session_start();
if (!isset($_SESSION['log'])) {
  header('location: loguj.php');
  exit;
}
?>
```

```
<!DOCTYPE HTML>
<html>
<head>
<title>Strona główna</title>
<meta charset="UTF-8">
</head>
<body>
<?php
$imie = ucfirst($_SESSION['log']);
echo "Witaj " . $imie;
?>
<p>Jesteś na stronie głównej.</p>
<p>Przed opuszczeniem strony wyloguj się!</p>
<a href="wyloguj.php">Wyloguj</a>
</body>
</html>
```

Po otwarciu sesji sprawdzane jest, czy istnieje zmienna logowania `$_SESSION['log']`. Jeżeli zmienna nie istnieje, to znaczy, że stronę próbuje otworzyć niezalogowany użytkownik — zostanie on przekierowany do strony logowania. Na stronie dostępnej tylko dla zalogowanych użytkowników wyświetlana jest informacja, że przed jej opuszczeniem użytkownik powinien się wylogować (rysunek 6.37).



Rysunek 6.37. Prawidłowo zalogowany użytkownik ma dostęp do strony głównej

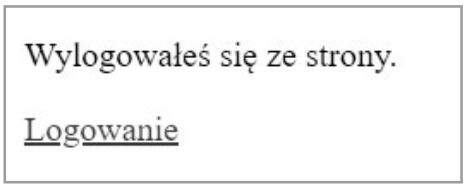
Próba wylogowania powoduje wywołanie kolejnego skryptu — *wyloguj.php*. Skrypt wylogowania zawiera polecenia zakończenia sesji oraz usunięcia jej zmiennych.

Skrypt *wyloguj.php*:

```
<?php
session_start();
if (isset($_SESSION['log'])) {
  unset($_SESSION['log']);
} else {
  header('location: loguj.php');
  exit;
```

```
}
$s = session_destroy();
?>
<!DOCTYPE HTML>
<html>
<head>
<title> Koniec_sesji</title>
<meta charset="UTF-8">
</head>
<body>
<p>Wylogowałeś się ze strony.</p>
<a href="loguj.php">Logowanie</a>
</body>
</html>
```

Pierwszym poleceniem skryptu jest wywołanie funkcji `session_start()`. Następnie jest sprawdzane, czy użytkownik wywołujący procedurę wylogowania był zalogowany. Jeżeli tak, to następuje usunięcie pliku cookie i zmiennej sesji `$_SESSION['log']` oraz zakończenie sesji przy użyciu funkcji `session_destroy` (rysunek 6.38).



Rysunek 6.38. Zakończenie sesji. Wylogowanie

Ćwiczenie 6.22

Zmodyfikuj skrypty z ćwiczenia 6.21 tak, aby weryfikacja danych dotyczyła wielu użytkowników. Utwórz tabelę *użytkownicy*, która będzie zawierała loginy oraz hasła użytkowników korzystających z danej strony internetowej. Zdefiniuj funkcję, która będzie przeszukiwała tablicę i zwracała wartość `true`, jeśli użytkownik o podanym loginie i haśle istnieje. Wykorzystaj ją do zdefiniowania nowych funkcji skryptów.

Ćwiczenie 6.23

Przy założeniu, że Twoja witryna internetowa składa się z kilku stron, napisz kod, który pozwoli na dostęp do podstron tylko ze strony głównej. Możliwość wejścia bezpośrednio na wybraną stronę (z pominięciem strony głównej) powinna zostać zablokowana.

Koszyk zakupów

Ćwiczenie 6.24

Wykorzystaj mechanizm sesji do utworzenia listy wybranych przez klienta produktów w sklepie internetowym.

Przygotuj dwa skrypty. W pierwszym skrypcie (*lista.php*) utwórz formularz, za pomocą którego możliwe będzie wybranie z listy różnych produktów. Wybrane produkty przechowuj w zmiennych sesji zapisanych w postaci tablicy. W drugim skrypcie (*koszyk.php*) wyświetl listę produktów wybranych przez użytkownika i umieszczonych w koszyku.

Rozwiązanie

Skrypt *lista.php*:

```
<?php
session_start();
?>
<!DOCTYPE HTML>
<html>
<head>
<title>Koszyk zakupów</title>
<meta charset="UTF-8">
</head>
<body>
<p><b>Lista artykułów</b></p>
<?php
if (isset($_POST['lista'])) {
  if (!empty($_SESSION['koszyk'])) {
    $koszyk = array_unique(
      array_merge(
        unserialize($_SESSION['koszyk']),
        $_POST['lista']
      )
    );
    $_SESSION['koszyk'] = serialize($koszyk);
  } else {
    $_SESSION['koszyk'] = serialize($_POST['lista']);
  }
  echo "<p>Wybrane produkty zostały umieszczone w koszyku</p>";
}
```

```
?>
<form action="lista.php" method="post">
<p>Wybór produktu:</p>
<p>
<select name="lista[]" multiple="multiple" size = "9">
<option value="Monitor"> Monitor</option>
<option value="Drukarka"> Drukarka</option>
<option value="Klawiatura"> Klawiatura</option>
<option value="Myszka"> Myszka</option>
<option value="Głośniki"> Głośniki</option>
<option value="Kamera internetowa"> Kamera internetowa</option>
<option value="Słuchawki"> Słuchawki</option>
<option value="Stacja DVD"> Stacja DVD</option>
<option value="Dysk twardy"> Dysk twardy</option>
<option value="Pendrive"> Pendrive</option>
<option value="Komputer"> Komputer</option>
</select></p>
<p>Wybierz produkty, trzymając wciśnięty klawisz Ctrl.</p>
<p><input type="submit" value="Wyślij"></p>
</form>
<p><a href="koszyk.php">Przejdź do koszyka</a></p>
</body>
</html>
```

W skrypcie zostały użyte funkcje działające na tablicach:

- `array_unique(tablica)` — pobiera tablicę i zwraca nową tablicę bez powtarzających się wartości,
- `array_merge(tablica1, tablica2, ...)` — łączy elementy wybranych tablic i zwraca tablicę wynikową,
- `unserialize(łańcuch)` — pobiera łańcuch znaków i przekształca go na wartość.

Po otwarciu lub wznowieniu sesji (funkcja `session_start()`) sprawdzamy, czy formularz z listą produktów został wysłany (tablica `$_POST['lista']`). Jeżeli tak, to sprawdzamy, czy istnieje tablica `$_SESSION['koszyk']`. Jeżeli taka tablica istnieje, to znaczy, że pewne produkty zostały już wybrane i przesłane do koszyka. Zostaną one połączone (funkcja `array_merge()`) z listą produktów znajdujących się w tablicy `$_POST['lista']`, a wynik będzie zapisany w zmiennej `$koszyk`.

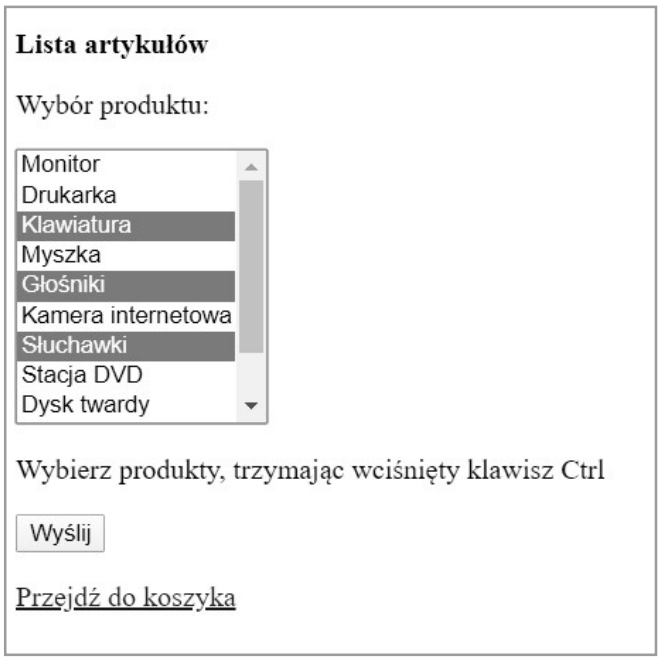
Dane umieszczane w sesji w tablicy `$_SESSION` muszą być ciągiem znaków (w formacie tablicy), dlatego przed ich zapisaniem należy za pomocą funkcji `serialize()` zamienić

je na taki ciąg znaków. Podobnie po pobraniu w sesji danych z tablicy `$_SESSION` należy zamienić je na wartości (funkcja `unserialize()`).

Przed połączeniem tablic dane pobrane z tablicy `$_SESSION['koszyk']` zostaną przekształcone na wartości (funkcja `unserialize()`). Po połączeniu tablic funkcja `array_unique()` usunie powtarzające się produkty umieszczone w koszyku. Następnie tablicy `$_SESSION['koszyk']` zostanie przypisana wartość zmiennej `$koszyk`.

Jeżeli nie istnieje tablica `$_SESSION['koszyk']`, to znaczy, że koszyk jest zapisywany po raz pierwszy. Po utworzeniu tablicy zostaną jej przypisane wartości tablicy `$_POST['lista']`.

Jeżeli nie istnieje tablica `$_POST['lista']`, to znaczy, że nie został wypełniony i wysłany formularz z listą produktów. Wyświetli się formularz, za pomocą którego użytkownik będzie mógł wybrać produkty do koszyka (rysunek 6.39).



Rysunek 6.39. Formularz z listą dostępnych produktów

Skrypt *koszyk.php*:

```
<?php
session_start();
?>
<!DOCTYPE HTML>
<html>
<head>
<title>Koszyk</title>
<meta charset="UTF-8">
</head>
```

```
<body>
<p><b>Zawartość koszyka</b></p>
<?php
if (isset($_SESSION['koszyk'])) {
  foreach(unserialize($_SESSION['koszyk']) as $produkt) {
    echo "<li>" . $produkt . "</li>";
  }
} else {
  echo "brak sesji";
}
?>
<p><a href="lista.php">Przejdź do listy produktów</a></p>
</body>
</html>
```

Podany kod otwiera sesję (funkcja `session_start()`), następnie sprawdza, czy istnieje tablica `$_SESSION['koszyk']`. Jeżeli istnieje, w pętli wyświetlana jest lista wartości tablicy `$_SESSION['koszyk']`. Wynik działania kodu został pokazany na rysunku 6.40.



Rysunek 6.40. Lista zamówionych produktów

Ćwiczenie 6.25

Zmodyfikuj kod skryptu z ćwiczenia 6.24 tak, aby była wyświetlana informacja o liczbie produktów w koszyku, oraz utwórz nowy skrypt, który będzie wyświetlał zawartość koszyka i umożliwiał usunięcie z niego wybranych produktów.

6.10. Bazy danych w PHP

Gdy tworzymy aplikacje internetowe, niezbędne dane możemy przechowywać w plikach. Ale kiedy liczba danych zwiększa się, rozwiązaniem jest zastosowanie bazy danych. Pliki nadal mogą być wykorzystywane, na przykład do przechowywania podstawowych ustawień aplikacji, ale do gromadzenia i przetwarzania większych ilości danych używa się relacyjnych baz danych. Do ich obsługi wykorzystywane są systemy zarządzania bazami danych. Najpopularniejszym systemem zarządzania bazami danych, który współpracuje z PHP, jest MySQL. Bazy danych można obsługiwać w sposób proceduralny lub obiektowy.

Aby możliwa była współpraca języka PHP z systemem MySQL, musi istnieć baza, z którą można się połączyć.

Użytkownik, który chce połączyć się z taką bazą, musi mieć login i hasło, znać nazwę oraz adres tej bazy danych.

UWAGA

We wszystkich skryptach tworzonych w tym rozdziale wykorzystywana jest baza danych *ksiegarnia* opisana w podręczniku *Kwalifikacja INF.03. Tworzenie i administrowanie stronami i aplikacjami internetowymi oraz bazami danych. Część 2. Projektowanie i administrowanie bazami danych. Podręcznik do nauki zawodu technik informatyk i technik programista*.

Współpraca PHP z MySQL może odbywać się przy użyciu:

- MySQLi (proceduralnie lub obiektowo),
- PDO (PHP Data Objects).

MySQLi obsługuje proceduralny i obiektowy paradygmat programowania. Interfejs proceduralny jest podobny do rozszerzenia MySQL.

6.10.1. MySQLi proceduralny

Połączenie z bazą danych

Nawiązanie połączenia

Do połączenia z bazą danych służy funkcja `mysqli_connect()`, która ma postać:

```
mysqli_connect('nazwa_hosta', 'nazwa_użytkownika', 'hasło', 'baza')
```

Nazwa_hosta oznacza nazwę lub adres IP serwera, na którym została umieszczona baza danych. Funkcja zwraca identyfikator połączenia lub wartość `false`, jeżeli połączenie nie zostało zrealizowane. Jeśli argumenty *nazwa_hosta*, *nazwa_użytkownika* i *hasło* zostaną pominięte, interpreter zastosuje wartości domyślne:

- *nazwa_hosta* — localhost:3306,
- *nazwa_użytkownika* — użytkownik domyślny,
- *hasło* — pusty ciąg znaków,
- *baza* — określa bazę, do której będą wysyłane zapytania.

Przykład 6.95

```
<?php
$serwer = 'localhost';
$login = 'root';
$haslo = '';
$baza = 'ksiegarnia';
```

```
$do_bazy = mysqli_connect($serwer, $login, $haslo, $baza);
if (mysqli_connect_errno()) {
  exit("Błąd połączenia z serwerem MySQL:" . mysqli_connect_error());
} else {
  echo "Połączono z serwerem baz danych . <br>";
}
?>
```

W podanym kodzie po wywołaniu funkcji `mysqli_connect()` sprawdzane jest, czy połączenie zostało nawiązane. Jeżeli tak, to wyświetli się komunikat o uzyskanym połączeniu, a w przeciwnym razie wyświetli się komunikat o braku tego połączenia i program zakończy swoje działanie. Funkcja `mysqli_connect_errno()` zwraca kod błędu połączenia, funkcja `mysqli_connect_error()` zwraca opis błędu połączenia.

Kończenie połączenia

Do zakończenia połączenia z bazą danych używana jest funkcja `mysqli_close()` w postaci:

```
mysqli_close([identyfikator])
```

Podany jako argument funkcji identyfikator jest identyfikatorem połączenia, które ma zostać zamknięte. Jeżeli identyfikator będzie pominięty, zamknięte zostanie połączenie ostatnio otwarte.

Jeżeli funkcja `mysqli_close()` nie zostanie wywołana, połączenie zostanie automatycznie zamknięte, gdy skrypt zakończy swoje działanie. Funkcja zwróci wartość `true`, jeśli operacja zamykania połączenia zakończy się prawidłowo, a w przeciwnym razie zwróci wartość `false`.

Przykład 6.96

```
<?php
$do_bazy = mysqli_connect('localhost', 'root', '', 'ksiegarnia');
if (mysqli_connect_errno()) {
  exit("Błąd połączenia z serwerem MySQL:" . mysqli_connect_error());
} else {
  echo "Połączono z serwerem baz danych";
  mysqli_close($do_bazy);
}
?>
```

Zapytania do bazy danych

Po nawiązaniu połączenia z wybraną bazą danych można wysyłać do niej zapytania. Są one przesyłane za pomocą funkcji `mysqli_query()` zapisanej w postaci:

```
mysqli_query(połączenie, 'zapytanie', stan_wyniku)
```

- *połączenie* — określa połączenie z bazą,
- *zapytanie* — zawiera treść zapytania,
- *stan_wyniku* — opcjonalne, określa sposób przetwarzania wyniku (domyślnie `MYSQLI_STORE_RESULT`).

Zwracane przez funkcję wartości zależą od typu zapytania. Jeżeli zapytanie pobierało dane (na przykład `SELECT`), to zwracany jest obiekt `mysqli_result`. Jeżeli zapytanie nie pobierało danych (na przykład `INSERT`), zwracana jest wartość `true`. Jeżeli wykonanie zapytania się nie powiodło, zwracana jest wartość `false`.

Przykład 6.97

Pobieranie danych:

```
$wynik = mysqli_query($do_bazy, 'SELECT * FROM Klient');
```

Przykład 6.98

Wstawianie danych:

```
$wstaw = mysqli_query($do_bazy, "INSERT INTO Klient VALUES (NULL, 'Anna',
'Polak', '87-100', 'Toruń')";
```

Zapytanie pobierające dane

Dla zapytań typu `SELECT` pobierających dane zwracany jest identyfikator zasobów. Może on zostać wykorzystany do odczytania tych danych. Można użyć w tym celu funkcji `mysqli_fetch_row()` lub `mysqli_fetch_array()`.

Funkcja `mysqli_fetch_row()` po wywołaniu odczytuje jeden wiersz tabeli i zapisuje go w kolejnym wierszu tablicy. Aby odczytać całą zawartość tabeli, należy wywołać tę funkcję w pętli. Gdy wszystkie dane zostaną odczytane, funkcja zwróci wartość `false`.

Przykład 6.99

```
while ($wiersz = mysqli_fetch_row($wynik)) {
 // przetwarzanie danych z zapytania
}
```

W podanym przykładzie zmienna `$wynik` zawiera identyfikator zasobów zwróconych przez zapytanie. W pętli zostaną odczytane kolejne wiersze tabeli zwrócone przez zapytanie i umieszczone w tablicy `$wiersz`.

Funkcja `mysqli_num_rows()` zwraca jako wartość liczbę wierszy znajdujących się w wyniku zapytania. Może być użyta do zbudowania pętli odczytującej kolejne wiersze zwrócone przez zapytanie.

Przykład 6.100

```
$ile = mysqli_num_rows($wynik);
for ($i = 0; $i < $ile; $i++) {
  $wiersz = mysqli_fetch_row($wynik);
  // przetwarzanie danych z zapytania
}
```

W pętli zostaną odczytane kolejne wiersze tabeli zwrócone przez zapytanie i umieszczone w tablicy `$wiersz`.

Funkcja `mysqli_fetch_array()` po wywołaniu odczytuje całą tabelę i zwraca tablicę asocjacyjną. Kluczami tej tablicy są nazwy kolumn zwróconej przez zapytanie tabeli.

Przykład 6.101

```
$tab = mysqli_fetch_array($wynik);
```

Ćwiczenie 6.26

Dysponujesz utworzoną w MySQL bazą danych *ksiegarnia*. Odczytaj dane z tabeli `Klient`.

Rozwiązanie

```
<!DOCTYPE HTML>
<html>
<head>
<title>Odczyt danych z tabeli Klient</title>
<meta charset="UTF-8">
</head>
<body>
<?php
$do_bazy = mysqli_connect('localhost', 'root', '', 'ksiegarnia');

if (!$do_bazy) {
  echo "Błąd połączenia z serwerem MySQL.<br>";
  ?>
  </body>
  </html>
```

```
    <?php
    exit;
} else {
    $zapytanie = mysqli_query($do_bazy, 'SELECT * FROM Klient');
    if (!$zapytanie) {
      mysqli_close();
      echo "Błąd w zapytaniu<br>";
      ?>
      </body>
      </html>
      <?php
      exit;
    } else {
      ?>
      <table>
      <tr>
      <td>Nazwisko</td>
      <td>Imię</td>
      <td>Kod pocztowy</td>
      <td>Miejscowość</td>
      <td>Ulica</td>
      <td>Nr domu</td>
      <td>PESEL</td>
      <td>Telefon</td>
      <td>Adres e-mail</td>
      <td>Id klienta</td>
      </tr>
      <?php
      while ($wiersze = mysqli_fetch_row($zapytanie)) {
        echo "<tr>";
        echo "<td>$wiersze[0]</td>";
        echo "<td>$wiersze[1]</td>";
        echo "<td>$wiersze[2]</td>";
        echo "<td>$wiersze[3]</td>";
        echo "<td>$wiersze[4]</td>";
```

```
            echo "<td>$wiersze[5]</td>";
            echo "<td>$wiersze[6]</td>";
            echo "<td>$wiersze[7]</td>";
            echo "<td>$wiersze[8]</td>";
            echo "<td>$wiersze[9]</td>";
            echo "</tr>";
        }
        ?>
        </table>
        <?php
        mysqli_close($do_bazy);
    }
}
?>
</body>
</html>
```

W podanym przykładzie po połączeniu z serwerem i wybraniu bazy danych zostaje wywołana funkcja `mysqli_query()`, która wysyła do serwera zapytanie SQL (`SELECT * FROM Klient`). Wynik zwrócony przez funkcję zostaje przypisany do zmiennej `$zapytanie`. Jeżeli wynik ma wartość `false`, to znaczy, że w zapytaniu wystąpił błąd, i skrypt kończy działanie. Jeśli wszystko przebiegło poprawnie, w HTML tworzona jest tabela w celu sformatowania wyświetlanych danych otrzymanych z zapytania. Następnie w kodzie PHP wyniki zapytania są pobierane w pętli za pomocą funkcji `mysqli_fetch_row()`, która pobiera kolejne wiersze tabeli i umieszcza je w tablicy. Indeks `0` tablicy zawiera wartość pobraną z pierwszej kolumny wiersza tabeli, indeks `1` wartość z drugiej kolumny itd. Wynik działania kodu został pokazany na rysunku 6.41.

Nazwisko	Imię	Kod pocztowy	Miejscowość	Ulica	Nr domu	PESEL	Telefon	Adres e-mail	Id klienta
Kowalski	Jan	05-120	Warszawa	Mickiewicza	1	67010123456	456793456	jan@o2.pl	1
Nowak	Andrzej	87-100	Toruń	Szeroka	34	78021203121	44444	an@gmail.com	2
Kowalski	Maciej	05-120	Warszawa	Sowia	25	43562902821			3

Rysunek 6.41. Wyświetlanie danych otrzymanych z zapytania do bazy

Klauzula WHERE

Klauzula `WHERE` jest dodawana do instrukcji `SELECT` wtedy, gdy należy wybrać tylko te wiersze, które spełniają określone kryterium.

Ćwiczenie 6.27

Utwórz skrypt, który zwróci z bazy dane osób, które mają nazwisko `Kowalski`.

Rozwiązanie

```
<?php
$do_bazy = mysqli_connect('localhost', 'root', '', 'ksiegarnia');
if (!$do_bazy) {
  exit("Błąd połączenia z serwerem MySQL.");
}
$zapytanie = mysqli_query($do_bazy, "SELECT * FROM Klient WHERE Nazwisko
= 'Kowalski'");

if (!$zapytanie === true) {
  mysqli_close();
  exit("Błąd w zapytaniu<br>");
}
while ($tab = mysqli_fetch_array($zapytanie)) {
  echo $tab['nazwisko'] . " " . $tab['imie'];
  echo "<br>";
}
mysqli_close($do_bazy);
?>
```

W wyniku wykonania podanego skryptu z bazy danych zwrócone zostaną dane osób, które mają nazwisko `Kowalski`. Dane zostały pobrane za pomocą funkcji `mysqli_fetch_array()`, która zapisała je w tablicy asocjacyjnej `$tab`. Kluczami tablicy `$tab` są nazwy kolumn tabeli SQL. Konstrukcje `$tab['nazwisko']` i `$tab['imie']` odwołują się do wartości zapisanych w tablicy i wyświetlają nazwiska i imiona klientów wybranych instrukcją `SELECT`.

Zapytanie wstawiające dane

Dla zapytań modyfikujących zawartość tabel funkcja `mysqli_query()` zwraca wartość `true` lub `false`. Natomiast za pomocą funkcji `mysqli_affected_rows()` można odczytać, ile wierszy zostało zmodyfikowanych przez zapytanie. Funkcja ma postać:

```
mysqli_affected_rows([identyfikator])
```

Argument *`identyfikator`* jest opcjonalny i zawiera identyfikator połączenia z serwerem. Jeżeli zostanie pominięty, działania będą dotyczyć ostatnio otwartego połączenia.

Zapytanie dodające nowe dane do tabeli jest realizowane przez polecenie `INSERT INTO`.

Ćwiczenie 6.28

Do tabeli `Klient` w bazie danych *ksiegarnia* dodaj nowego klienta o następujących danych: Anna Lisek, miejscowość: 34-100 Wadowice, ulica Lwowska, 11, PESEL: 95031203267, tel. 936789453, adres e-mail: ania@gmail.com.

Rozwiązanie

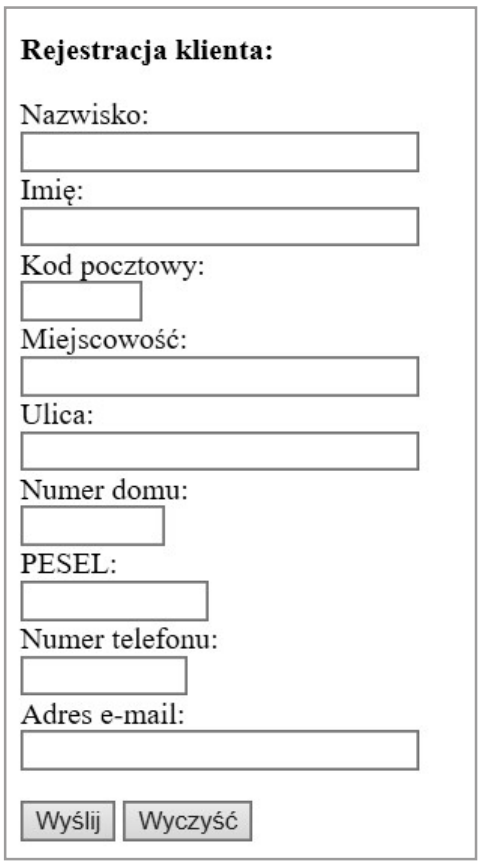
```
<?php
$do_bazy = mysqli_connect('localhost', 'root', '', 'ksiegarnia');
if (!$do_bazy) {
  exit("Błąd połączenia z serwerem MySQL.");
}
$dodaj = mysqli_query($do_bazy, "INSERT INTO Klient VALUES('Lisek',
'Anna', '34-100', 'Wadowice', 'Lwowska', '11', '95031203267',
'936789453', 'ania@gmail.com', NULL)");
if (!$dodaj === true) {
  mysqli_close($do_bazy);
  exit("Błąd w zapytaniu<br>");
}
$ile = mysqli_affected_rows($do_bazy);
echo "Liczba rekordów dodanych do tabeli Klient wynosi: $ile<br>";
mysqli_close($do_bazy);
?>
```

W podanym kodzie po prawidłowym połączeniu z serwerem i wybraniu bazy danych zostaje utworzona zmienna `$dodaj`, w której jest przechowywany wynik zapytania SQL (`INSERT INTO`). Za pomocą funkcji `mysqli_query()` jest ono wysyłane do serwera. Instrukcja `if` sprawdza, czy funkcja `mysqli_query()` zwróciła wartość `false`, co oznacza, że zapytanie nie zostało obsłużone. Wtedy skrypt kończy działanie. Jeżeli zapytanie zostało przyjęte, to zmiennej `$ile` przypisywana jest wartość zwrócona przez funkcję `mysqli_affected_rows()`, określająca, ile wierszy w bazie danych zostało zmienionych. Ta informacja zostanie wyświetlona w przeglądarce.

Inną metodą dodawania nowych danych do tabeli jest wprowadzanie ich za pomocą formularza.

Ćwiczenie 6.29

Utwórz skrypt, który dane pobrane z formularza (rysunek 6.42) będzie dodawał w bazie *ksiegarnia* do tabeli `Klient`.



Rysunek 6.42. Formularz do wprowadzania danych do bazy

Rozwiązanie

Utwórz dwa skrypty. Skrypt HTML (*form_klient.html*) będzie zawierał formularz do wprowadzania danych klienta. W tym skrypcie dane wpisane do formularza zostaną przesłane do pliku *dodaj_klienta.php*.

Skrypt PHP (*dodaj_klienta.php*) doda otrzymane z formularza dane do tabeli `Klient`.

Skrypt *form_klient.html*:

```
<!DOCTYPE HTML>
<html>
<head>
<title>Rejestracja klienta</title>
<meta charset="UTF-8">
<style>
#wyb {
  font-weight: bold;
```

```
  font-size: 12pt;
}
</style>
</head>
<body>
<form action="dodaj_klienta.php" method="post">
<p id="wyb">Rejestracja klienta:</p>
Nazwisko:<br>
<input type="text" name="nazw" value="" size="30"><br>
Imię:<br>
<input type="text" name="im" value="" size="30"><br>
Kod pocztowy:<br>
<input type="text" name="kod" value="" size="5"><br>
Miejscowość:<br>
<input type="text" name="mia" value="" size="30"><br>
Ulica:<br>
<input type="text" name="ul" value="" size="30"><br>
Numer domu:<br>
<input type="text" name="nr" value="" size="7"><br>
PESEL:<br>
<input type="text" name="pe" value="" size="11"><br>
Numer telefonu:<br>
<input type="text" name="tel" value="" size="9"><br>
Adres e-mail:<br>
<input type="text" name="adr" value="" size="30"><br>

<p><input type="submit" value="Wyślij" name="wyslij">
<input type="reset" value="Wyczyść" name="zeruj"></p>
</form>
</body>
</html>
```

Skrypt *dodaj_klienta.php*:

```
<?php
$do_bazy = mysqli_connect('localhost', 'root', '', 'ksiegarnia');
if (!$do_bazy) {
  exit("Błąd połączenia z serwerem MySQL.");
}
$nazwisko = $_POST['nazw'];
$imie = $_POST['im'];
$kod = $_POST['kod'];
$miasto = $_POST['mia'];
$ulica = $_POST['ul'];
$nr = $_POST['nr'];
$pesel = $_POST['pe'];
$tel = $_POST['tel'];
$mail = $_POST['adr'];
$dodaj = "INSERT INTO Klient VALUES ('$nazwisko', '$imie', '$kod',
'$miasto', '$ulica', '$nr', '$pesel', '$tel', '$mail', NULL)";
$zapytanie = mysqli_query($do_bazy, $dodaj);
if (!$zapytanie === true) {
  echo "Nowy klient nie został dodany do bazy!";
} else {
  echo "Klient " . $nazwisko . " " . $imie . " został dodany do bazy.";
}
mysqli_close($do_bazy);
?>
```

Podany skrypt jest podobny do skryptu z ćwiczenia 6.28. Jedyna zmiana to użycie w instrukcji `INSERT INTO` zmiennej `$_POST['nazwa_pola']` w miejscu, gdzie poprzednio wstawione były wartości danych.

Aktualizowanie danych

Dane można aktualizować za pomocą instrukcji `UPDATE`.

Ćwiczenie 6.30

Zaktualizuj w tabeli `Klient` dane klienta `Nowak Andrzej`. Dane do aktualizacji pobierz z formularza dostępnego na stronie internetowej.

Rozwiązanie

Aby zaktualizować dane klienta, należy wykonać kilka czynności — wyszukać jego dane w bazie danych, wyświetlić je za pomocą formularza na stronie internetowej, pobrać przesłane z formularza zaktualizowane dane i zapisać je w bazie.

Pierwszy skrypt będzie pobierał dane z bazy i wyświetlał je na stronie w postaci formularza.

Skrypt *szukaj_klienta.php*:

```
<?php
$do_bazy = mysqli_connect('localhost', 'root', '', 'ksiegarnia');
if (!$do_bazy) {
  exit("Błąd połączenia z serwerem MySQL.");
}

$zapytanie = mysqli_query($do_bazy, "SELECT * FROM Klient
WHERE Nazwisko = 'Nowak' and Imie = 'Andrzej'");
$ile = mysqli_num_rows($zapytanie);

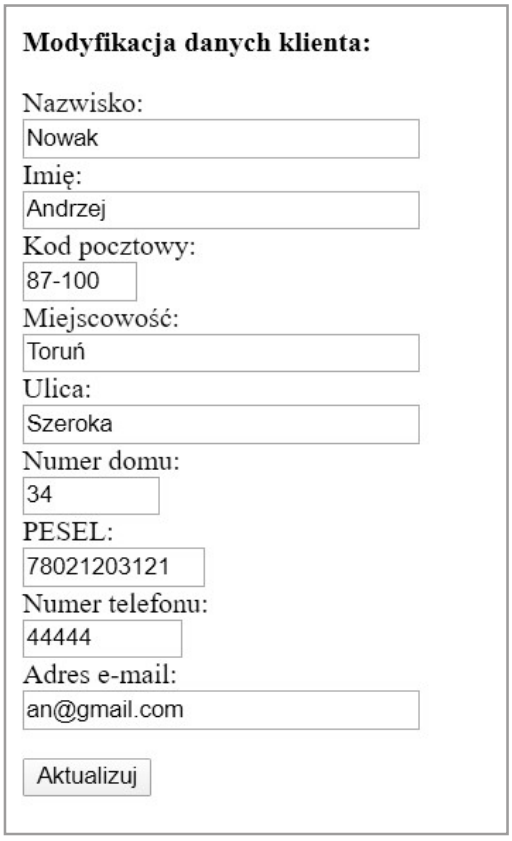
if (!$zapytanie === true) {
  mysqli_close($do_bazy);
  exit("Błąd w zapytaniu.");
}
if ($ile = 0) {
  exit("Brak danych");
}
if ($ile > 1) {
  exit("Nie można wyodrębnić danych");
}
$wiersz = mysqli_fetch_array($zapytanie);
$nazw = $wiersz['nazwisko'];
$im = $wiersz['imie'];
$kod = $wiersz['kod_pocztowy'];
$mia = $wiersz['miejscowosc'];
$ul = $wiersz['ulica'];
$nr = $wiersz['nr_domu'];
$pe = $wiersz['PESEL'];
```

```
$tel = $wiersz['telefon'];
$adr = $wiersz['adres_e_mail'];
?>

<!DOCTYPE HTML>
<html>
<head>
<title>Modyfikacja danych</title>
<meta charset="UTF-8">
<style>
p {
  font-weight: bold;
  font-size: 12pt;
}
</style>
</head>
<body>
<form action="aktualizuj.php" method="post">
<p>Modyfikacja danych klienta:</p>
Nazwisko:<br>
<input type="text" name="nazw" value="<?php
echo $nazw; ?>" size="30"><br>
Imię:<br>
<input type="text" name="im" value="<?php echo $im; ?>" size="30"><br>
Kod pocztowy:<br>
<input type="text" name="kod" value="<?php echo $kod; ?>" size="5"><br>
Miejscowość:<br>
<input type="text" name="mia" value="<?php echo $mia; ?>" size="30"><br>
Ulica:<br>
<input type="text" name="ul" value="<?php echo $ul; ?>" size="30"><br>
Numer domu:<br>
<input type="text" name="nr" value="<?php echo $nr; ?>" size="7"><br>
PESEL:<br>
<input type="text" name="pe" value="<?php echo $pe; ?>" size="11"><br>
Numer telefonu:<br>
```

```
<input type="text" name="tel" value="<?php echo $tel; ?>" size="9"><br>
Adres e-mail:<br>
<input type="text" name="adr" value="<?php echo $adr; ?>" size="30"><br>
<p><input type="submit" value="Aktualizuj" name="akt"></p>
</form>
</body>
</html>
```

Po połączeniu z serwerem i wybraniu bazy danych zostaje wywołana funkcja `mysqli_query()`, która wysyła do serwera zapytanie SQL (`SELECT * FROM Klient WHERE Nazwisko = 'Nowak' and Imie = 'Andrzej'`). W wyniku zostaną zwrócone dane wybranego klienta. Zwrócone dane zostają przypisane do zmiennej `$zapytanie`. Następnie za pomocą funkcji `mysqli_num_rows()` sprawdza się, ile wierszy zostało odczytanych z bazy. Jeżeli `0`, to brak danych do dalszego przetwarzania i następuje zakończenie działania skryptu. Jeżeli więcej niż `1`, to nie można określić, czyje dane powinny być aktualizowane, i również następuje zakończenie działania skryptu. Jeśli zostały odczytane dane jednego klienta, to będą one przypisane do zmiennych i nastąpi przejście do kodu HTML. W kodzie HTML nastąpi wyświetlenie formularza z bieżącymi danymi pobranymi z utworzonych zmiennych (rysunek 6.43). Użytkownik może zaktualizować swoje dane i przesłać je do skryptu *aktualizuj.php*.



Rysunek 6.43. Bieżące dane pobrane z bazy danych do aktualizacji

Kolejnym krokiem będzie aktualizacja danych w bazie.

Skrypt *aktualizuj.php*:

```
<?php
$do_bazy = mysqli_connect('localhost', 'root', '', 'ksiegarnia');
if (!$do_bazy) {
  exit("Błąd połączenia z serwerem MySQL.");
}
$nazwisko = $_POST['nazw'];
$imie = $_POST['im'];
$kod = $_POST['kod'];
$miasto = $_POST['mia'];
$ulica = $_POST['ul'];
$nr = $_POST['nr'];
$pesel = $_POST['pe'];
$tel = $_POST['tel'];
$mail = $_POST['adr'];

$aktualny = "UPDATE Klient SET nazwisko = '$nazwisko', imie = '$imie',
kod_pocztowy = '$kod', miejscowosc = '$miasto', ulica = '$ulica',
nr_domu = '$nr', PESEL = '$pesel', telefon = '$tel', adres_e_mail
= '$mail' WHERE Nazwisko = 'Nowak' and Imie = 'Andrzej'";

$zapytanie = mysqli_query($do_bazy, $aktualny);
if (!$zapytanie === true) {
  echo "Błąd zapytania! Dane nie zostały zaktualizowane!";
} else {
  echo "Dane zostały zaktualizowane.";
}
mysqli_close($do_bazy);
?>
```

Skrypt instrukcją `UPDATE` aktualizuje dane przesłane z formularza i przekazane za pomocą zmiennej `$_POST`.

Tworzenie bazy danych

Do tworzenia bazy danych jest używana instrukcja `SQL CREATE DATABASE`. Należy dołączyć ją do funkcji `mysqli_query()`.

Przykład 6.102

```
<?php
$do_bazy = mysqli_connect('localhost', 'root', '');
if (!$do_bazy) {
  exit("Błąd połączenia z serwerem MySQL.");
}
$baza_sql = "CREATE DATABASE moje_kino";
if (mysqli_query($do_bazy, $baza_sql)) {
  echo "Baza została utworzona";
} else {
  echo "Błąd! Nie można utworzyć bazy";
}
mysql_close($do_bazy);
?>
```

W przykładzie został pokazany fragment kodu PHP tworzący bazę danych *moje_kino*.

Tworzenie tabeli

Do tworzenia tabeli w istniejącej bazie danych jest używana instrukcja SQL `CREATE TABLE`. Należy dołączyć ją do funkcji `mysqli_query()`.

Przykład 6.103

```
<?php
$do_bazy = mysqli_connect('localhost', 'root', '', 'moje_kino');
if (!$do_bazy) {
  exit("Błąd połączenia z serwerem MySQL.");
}
$tab = "CREATE TABLE Film (Tytul_filmu VARCHAR(30), Rezyser
VARCHAR(30), Rok_prod  VARCHAR(4))";
if (mysqli_query($do_bazy, $tab)) {
  echo "Tabela Film została utworzona";
} else {
  echo "Błąd! Nie można utworzyć tabeli Film";
}
mysql_close($do_bazy);
?>
```

W przykładzie został pokazany fragment kodu PHP tworzący tabelę `Film`.

6.10.2. MySQLi zorientowany obiektowo

Połączenie z bazą danych

Nawiązanie połączenia

Do połączenia z bazą danych służy funkcja `mysqli()`, która ma postać:

```
new mysqli('nazwa_hosta', 'nazwa_użytkownika', 'hasło', 'baza')
```

Przykład 6.104

```
<?php
$do_bazy = new mysqli('localhost', 'root', '', 'ksiegarnia');
if (!$do_bazy->connect_error) {
  exit("Błąd połączenia z serwerem MySQL:" . $do_bazy->connect_error);
} else {
  echo "Połączono z serwerem baz danych.";
}
?>
```

W podanym kodzie podobnie po wywołaniu funkcji `mysqli()` sprawdzane jest, czy połączenie zostało nawiązane. Jeżeli tak, to wyświetli się komunikat o uzyskanym połączeniu, w przeciwnym razie wyświetli się komunikat o braku tego połączenia i program zakończy swoje działanie.

Kończenie połączenia

Do zakończenia połączenia z bazą danych używana jest funkcja `close()` w postaci:

```
identyfikator->close()
```

Przykład 6.105

```
<?php
$do_bazy = new mysqli('localhost', 'root', '', 'ksiegarnia');
if (!$do_bazy->connect_error) {
  exit("Błąd połączenia z serwerem MySQL:" . $do_bazy->connect_error);
} else {
  echo "Połączono z serwerem baz danych.";
}
$do_bazy->close();
?>
```

Jeżeli funkcja `close()` nie zostanie wywołana, połączenie zostanie automatycznie zamknięte, gdy skrypt zakończy swoje działanie.

Zapytania do bazy danych

Po nawiązaniu połączenia z wybraną bazą danych można wysyłać do niej zapytania. Są one przesyłane za pomocą funkcji `query()` zapisanej w postaci:

```
identyfikator->query('zapytanie')
```

Argumentem funkcji jest treść zapytania. Zwracane przez funkcję wartości zależą od typu zapytania. Jeżeli zapytanie pobierało dane (na przykład `SELECT`), to zwracany jest obiekt `mysqli_result`. Jeżeli zapytanie nie pobierało danych (na przykład `INSERT`), zwracana jest wartość `true`. Jeśli wykonanie zapytania się nie powiodło, zwracana jest wartość `false`.

Zapytanie pobierające dane

Dla zapytań typu `SELECT` pobierających dane zwracany jest identyfikator zasobów. Może on zostać wykorzystany do odczytania tych danych. Do odczytania danych pobranych zapytaniem `SELECT` można użyć funkcji `fetch_assoc()`.

Przykład 6.106

```
while ($wiersze = $wynik->fetch_assoc()) {
  // przetwarzanie danych z zapytania
}
```

W podanym przykładzie zmienna `$wynik` zawiera identyfikator zasobów zwróconych przez zapytanie. W pętli zostaną odczytane kolejne wiersze tabeli zwrócone przez zapytanie i umieszczone w tablicy `$wiersze`.

Funkcja `num_rows()` zwraca jako wartość liczbę wierszy znajdujących się w wyniku zapytania. Może być użyta do zbudowania pętli odczytującej kolejne wiersze zwrócone przez zapytanie.

Funkcja `fetch_assoc()` po wywołaniu odczytuje całą tabelę i zwraca tablicę asocjacyjną. Kluczami tej tablicy są nazwy kolumn zwróconej przez zapytanie tabeli.

Przykład 6.107

Dysponujemy utworzoną w MySQL bazą danych *ksiegarnia*. Odczytamy dane z tabeli `Klient`.

```
<!DOCTYPE HTML>
<html>
<head>
<title>Odczyt danych z tabeli Klient</title>
<meta charset="UTF-8">
</head>
<body>
```

```
<?php
$do_bazy = new mysqli('localhost', 'root', '', 'ksiegarnia');
if ($do_bazy->connect_error) {
  exit("Błąd połączenia z serwerem MySQL:");
}
$zapytanie = 'SELECT * FROM Klient';
$wynik = $do_bazy->query($zapytanie);
if ($wynik->num_rows > 0) {
  while ($wiersze = $wynik->fetch_assoc()) {
    echo $wiersze['nazwisko'] . " " . $wiersze['imie'] . "<br>";
  }
} else {
  echo "0 wierszy";
}
$do_bazy->close();
?>
</body>
</html>
```

W podanym przykładzie po połączeniu z serwerem i wybraniu bazy danych zostaje wywołana funkcja `query()`, która wysyła do serwera zapytanie SQL (`SELECT * FROM Klient`). Wynik zwrócony przez funkcję zostaje przypisany zmiennej `$wynik`. Jeżeli wynik wynosi `0`, to znaczy, że zapytanie zwróciło zero wierszy, i skrypt kończy działanie. Jeśli wszystko przebiegło poprawnie, to w kodzie PHP wyniki zapytania są pobierane w pętli za pomocą funkcji `fetch_assoc()`, która pobiera kolejne wiersze tabeli i umieszcza je w tablicy asocjacyjnej.

Klauzula WHERE

Klauzula `WHERE` służy do wybierania rekordów, które spełniają określone kryterium.

Przykład 6.108

```
<?php
$do_bazy = new mysqli('localhost', 'root', '', 'ksiegarnia');
if ($do_bazy->connect_error) {
  exit("Błąd połączenia z serwerem MySQL:");
}
$zapytanie = "SELECT * FROM Klient WHERE Nazwisko = 'Kowalski'";
$wynik = $do_bazy->query($zapytanie);
```

```
if (!$wynik === true) {
  $do_bazy->close();
  exit("Błąd w zapytaniu<br>");
}
while ($tab = $wynik->fetch_assoc()) {
  echo $tab['nazwisko'] . " " . $tab['imie'];
  echo "<br>";
}
$do_bazy->close();
?>
```

W wyniku wykonania podanego skryptu z bazy danych zwrócone zostaną nazwiska i imiona osób, które mają nazwisko `Kowalski`. Dane zostały pobrane za pomocą funkcji `fetch_assoc()`, która zapisała je w tablicy asocjacyjnej `$tab`. Kluczami tablicy `$tab` są nazwy kolumn tabeli SQL. Konstrukcje `$tab['nazwisko']` i `$tab['imie']` odwołują się do wartości zapisanych w tablicy i wyświetlają nazwiska i imiona klientów wybranych instrukcją `SELECT`.

Zapytanie wstawiające dane

Dla zapytań modyfikujących zawartość tabel funkcja `query()` zwraca wartość `true` lub `false`.

Zapytanie dodające nowe dane do tabeli jest realizowane przez polecenie `INSERT INTO`.

Przykład 6.109

Do tabeli `Klient` w bazie danych *ksiegarnia* zostanie dodany nowy wpis.

```
<?php
$do_bazy = new mysqli('localhost', 'root', '', 'ksiegarnia');
if ($do_bazy->connect_error) {
  exit("Błąd połączenia z serwerem MySQL:");
}
$dodaj = "INSERT INTO Klient (imie, nazwisko, miejscowosc, adres_e_mail)
VALUES('Hanna', 'Nowicka', 'Kraków', 'hania@gmail.com')";
$wynik = $do_bazy->query($dodaj);
if ($wynik === true) {
  echo "Rekord został dodany";
} else {
  echo "Błąd w zapytaniu. Rekord nie został dodany";
}
$do_bazy->close();
?>
```

Inną metodą dodawania nowych danych do tabeli jest wprowadzanie ich za pomocą formularza.

Ćwiczenie 6.31

Utwórz podobnie jak w ćwiczeniu 6.29 skrypt, który dane pobrane z formularza będzie dodawał w bazie *ksiegarnia* do tabeli `Klient`. W skrypcie zastosuj polecenia `mysqli` zorientowanego obiektowo.

Aktualizowanie danych

Aktualizowanie danych jest realizowane za pomocą instrukcji `UPDATE`.

Przykład 6.110

W tabeli `Klient` zostaną zaktualizowane dane klienta `Nowicka Hanna`.

```
<?php
$do_bazy = new mysqli('localhost', 'root', '', 'ksiegarnia');
if ($do_bazy->connect_error) {
  exit("Błąd połączenia z serwerem MySQL:");
}
$mody = "UPDATE Klient SET ulica = 'Franciszkańska',
nr_domu = 2 WHERE Nazwisko = 'Nowicka' and Imie = 'Hanna'";
$wynik = $do_bazy->query($mody);
if ($wynik === true) {
  echo "Dane zostały zaktualizowane.";
} else {
  echo "Błąd zapytania! Dane nie zostały zaktualizowane";
}
$do_bazy->close();
?>
```

Ćwiczenie 6.32

Utwórz podobnie jak w ćwiczeniu 6.30 skrypt, który zaktualizuje w tabeli `Klient` dane wybranego klienta. Dane do aktualizacji pobierz z formularza dostępnego na stronie internetowej. W skrypcie zastosuj polecenia `mysqli` zorientowanego obiektowo.

Tworzenie bazy danych

Do tworzenia bazy danych jest używana instrukcja `CREATE DATABASE`. Należy dołączyć ją do funkcji `query()`.

Przykład 6.111

```
<?php
$do_bazy = new mysqli('localhost', 'root', '');
if ($do_bazy->connect_error) {
  exit("Błąd połączenia z serwerem MySQL:");
}

$baza = "CREATE DATABASE szkola";
if ($do_bazy->query($baza) ==== true) {
  echo "Baza została utworzona";
} else {
  echo "Błąd! Nie można utworzyć bazy";
}
$do_bazy->close();
?>
```

W przykładzie został pokazany fragment kodu PHP tworzący bazę danych *szkola*.

Tworzenie tabeli

Do tworzenia tabeli w istniejącej bazie danych jest używana instrukcja SQL `CREATE TABLE`. Należy dołączyć ją do funkcji `query()`.

Przykład 6.112

```
<?php
$do_bazy = new mysqli('localhost', 'root', '');
if ($do_bazy->connect_error) {
  exit("Błąd połączenia z serwerem MySQL:");
}
$tab = "CREATE TABLE Uczen (nazwisko VARCHAR(30),
imie VARCHAR(30), email VARCHAR(40))";
if ($do_bazy->query($tab) === true) {
  echo "Tabela Uczen została utworzona";
} else {
  echo "Błąd! Nie można utworzyć tabeli Uczen";
}
$do_bazy->close();
?>
```

W przykładzie został pokazany fragment kodu PHP tworzący tabelę `Uczen`.

6.11. Biblioteka PDO

PDO (ang. *PHP Data Objects*) jest rozszerzeniem języka PHP, które udostępnia jednolity, uniwersalny interfejs do komunikacji z bazami danych w technice obiektowej. Od wersji 5.1 jest elementem środowiska PHP i poza dodaniem sterowników dla wykorzystywanych baz danych nie wymaga innych działań. Sterowniki różnych baz danych udostępniają takie same metody, chociaż mogą również dodawać własne, specyficzne metody.

Zaletą PDO jest to, że tworzone skrypty mogą używać tych samych metod niezależnie od wykorzystywanej bazy. Kolejną zaletą jest walidacja danych wysyłanych w zapytaniach oraz filtrowanie tych danych pod kątem zabezpieczeń przed atakami.

Nawiązanie połączenia

Aby nawiązać połączenie z bazą danych, należy za pomocą konstruktora utworzyć nowy obiekt klasy PDO w postaci:

```
PDO(źródło_danych, nazwa_użytkownika, hasło, opcje)
```

- *źródło_danych* — inaczej DSN (ang. *Data Source Name*), jest to specjalny ciąg znaków opisujący rodzaj bazy danych i sposób połączenia z bazą. Zawiera takie informacje jak nazwa sterownika, adres serwera, nazwa bazy, numer portu.
- *opcje* — to tablica zawierająca dodatkowe opcje związane z połączeniem.

Parametrem wymaganym jest *źródło_danych*. Pozostałe mogą być pominięte.

Wersja uproszczona źródła danych dla MySQL wygląda następująco:

```
mysql:host=nazwa_serwera;port=numer_portu;dbname=nazwa_bazy
```

- *nazwa_serwera* — określa nazwę lub adres serwera baz danych, na przykład localhost, 127.0.0.1.
- *numer_portu* — port, przez który nastąpi połączenie z bazą danych. Jeżeli parametr zostanie pominięty, przyjęta będzie wartość standardowa.
- *nazwa_bazy* — nazwa bazy, z którą nastąpi połączenie.

Przykład 6.113

```
mysql:host=localhost;port=3306;dbname=moja_baza
```

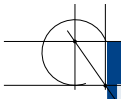
W podanym przykładzie źródło danych pozwoli na nawiązanie połączenia z bazą *moja_baza* znajdującą się na serwerze MySQL pracującym na lokalnym komputerze. Połączenie będzie realizowane na porcie 3306.

Przykład 6.114

```
<?php
$user = "root";
$pass = "";
```

```
$dsn = "mysql:host=localhost;dbname=ksiegarnia";
try {
  $pdo = new PDO($dsn, $user, $pass);
  echo 'Połączenie nawiązane!';
} catch (PDOException $e) {
  echo 'Błąd połączenia: ' . $e->getMessage();
  exit;
}
?>
```

W podanym przykładzie ciąg określający źródło danych został przypisany do zmiennej `$dsn`. Utworzony obiekt klasy PDO został przypisany do zmiennej `$pdo`. Jeżeli podczas nawiązywania połączenia wystąpi błąd, zostanie zgłoszony wyjątek `PDOException`. Musi on zostać przechwycony, ponieważ domyślny komunikat o błędzie, który wygeneruje PHP, wyświetli nazwę użytkownika i hasło do bazy danych.



UWAGA

PDO ma obsługę błędów opartą na wyjątkach. Jeżeli zgłoszone wyjątki nie zostaną obsłużone, to skrypt zakończy działanie z komunikatem PHP, a w przypadku błędu podczas łączenia z bazą komunikat będzie zawierał login i hasło do bazy danych.

Kończenie połączenia

Połączenie, które zostało nawiązane za pomocą obiektu PDO, istnieje aż do momentu usunięcia tego obiektu z pamięci. Obiekt zostanie usunięty automatycznie po zakończeniu działania skryptu. Można go również usunąć, przypisując zmiennej obiektowej przechowującej odwołanie do obiektu wartość `null`.

Przykład 6.115

```
$pdo = null;
```

Pobieranie danych

Zapytanie pobierające dane jest wysyłane za pomocą metody `query()`. Treść zapytania jest przekazywana do tej metody w postaci argumentu.

```
$obiekt_PDO->query("Treść_zapytania")
```

Metoda `query()` zwraca obiekt klasy `PDOStatement`, który zawiera wynik wykonania zapytania.

Przykład 6.116

```
<?php
try {
  $pdo = new PDO('mysql:host=localhost;dbname=ksiegarnia', 'root');
  $zapytanie = $pdo->query('SELECT nazwisko, imie, pesel FROM Klient');
  foreach ($zapytanie as $wiersz) {
    echo $wiersz['nazwisko'] . ', '.$wiersz['imie']
. ', ' . $wiersz['pesel'] . '<br>';
  }
  $pdo = null;
} catch (PDOException $e) {
  echo "Błąd połączenia:" . $e->getMessage();
}
?>
```

W podanym przykładzie metoda `query()` zwraca wynik wykonania zapytania. Następnie za pomocą pętli `foreach` kolejne wiersze wyniku zostają zapisane do tablicy asocjacyjnej `$wiersz` i będą wyświetlone instrukcją `echo`. Po zakończeniu wyświetlania wyników połączenie zostanie zamknięte.

Aktualizacja danych

Zapytania typu `INSERT` czy `UPDATE` służące do aktualizacji danych i inne, które nie zwracają wyników, są wysyłane za pomocą metody `exec()`. Po wykonaniu zapytania metoda ta zwraca liczbę określającą, ile wierszy zostało zmodyfikowanych.

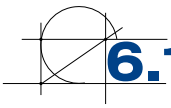
Przykład 6.117

```
<?php
try {
  $pdo = new PDO('mysql:host=localhost;dbname=ksiegarnia', 'root');
  $dodaj = $pdo->exec("INSERT INTO Klient (nazwisko, imie, pesel)
VALUES('Górski', 'Jakub', '92120203229')");
  if ($dodaj > 0) {
    echo "Dodano: " . $dodaj . " wierszy<br>";
  } else {
    echo "Błąd podczas dodawania danych!<br>";
  }
  $pdo = null;
} catch (PDOException $e) {
  echo "Błąd połączenia:" . $e->getMessage();
}
?>
```

W podanym przykładzie metoda `exec()` wysyła zapytanie dodające nowe wiersze do tabeli oraz zwraca liczbę dodanych wierszy. Następnie informacja ta jest wyświetlana. Jeżeli operacja nie została wykonana poprawnie, pojawią się odpowiednie komunikaty.

Ćwiczenie 6.33

Zmodyfikuj skrypt z przykładu 6.117 tak, aby dane były wprowadzane do tabeli za pomocą formularza wyświetlanego na stronie internetowej.



6.12. Pytania i zadania

6.12.1. Pytania

1. Wymień obszary zastosowań języka PHP.
2. W jaki sposób w kodzie HTML oznaczane są skrypty PHP?
3. Jakiego rodzaju informacje przechowują zmienne predefiniowane?
4. Czym różni się operator inkrementacji `++$x` od operatora `$x++`?
5. Jakie jest przeznaczenie **stałych magicznych**?
6. Gdzie stosowana jest pętla `foreach`?
7. Czym różnią się zmienne statyczne od zmiennych lokalnych?
8. Jaką wartość w języku PHP przyjmuje *`znacznik_czasu`* (`timestamp`)?
9. Czym różni się przesyłanie danych z formularza na serwer za pomocą metody `POST` od przesyłania danych przy użyciu metody `GET`?
10. Jakie jest przeznaczenie plików cookies?
11. Jak działa dostępny w języku PHP mechanizm sesji?
12. Jakie parametry należy podać w kodzie PHP, aby nawiązać połączenie z bazą danych?
13. Jakie jest przeznaczenie biblioteki PDO dostępnej w języku PHP?

6.12.2. Zadania

Zadanie 1.

1. W MySQL utwórz bazę danych *Moja_szkola* lub wykorzystaj bazę danych *Moja_szkola* tworzoną w części II podręcznika. Baza danych powinna zawierać tabelę `Uczen` z danymi uczniów oraz tabelę `Rejestracja` z dwoma polami: `login` i `haslo`. Zostanie ona wykorzystana do rejestracji loginów i haseł użytkowników strony.
2. Utwórz stronę internetową, która za pomocą skryptów PHP będzie komunikowała się z tą bazą danych. Strona główna ma zawierać formularz logowania oraz formularz rejestracji nowych uczniów.